



ÉCOLE
POLYTECHNIQUE
DE BRUXELLES

07. Encoding Geometry

INFO-H502

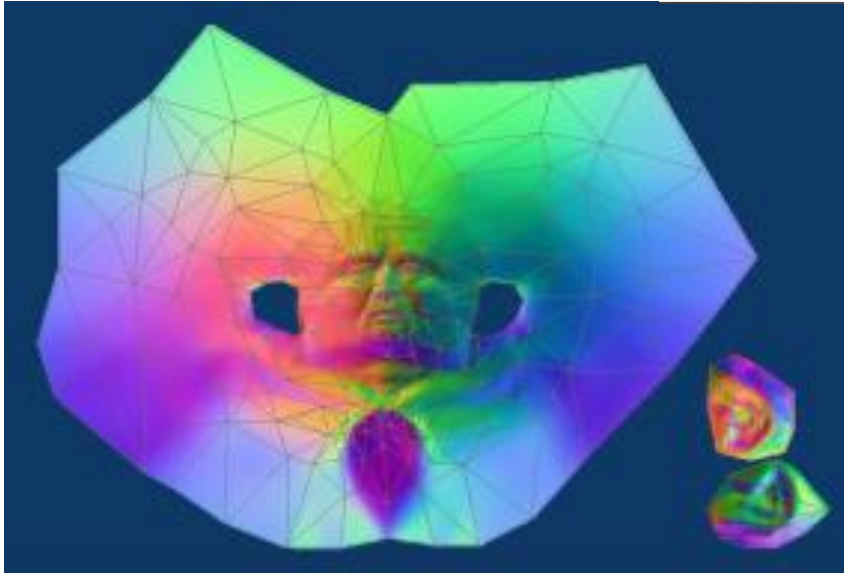
Daniele Bonatto

LISA-VR

2024

Previously

- Texturing and
- Normal mapping



Object space normal map



High poly model

Simplified model

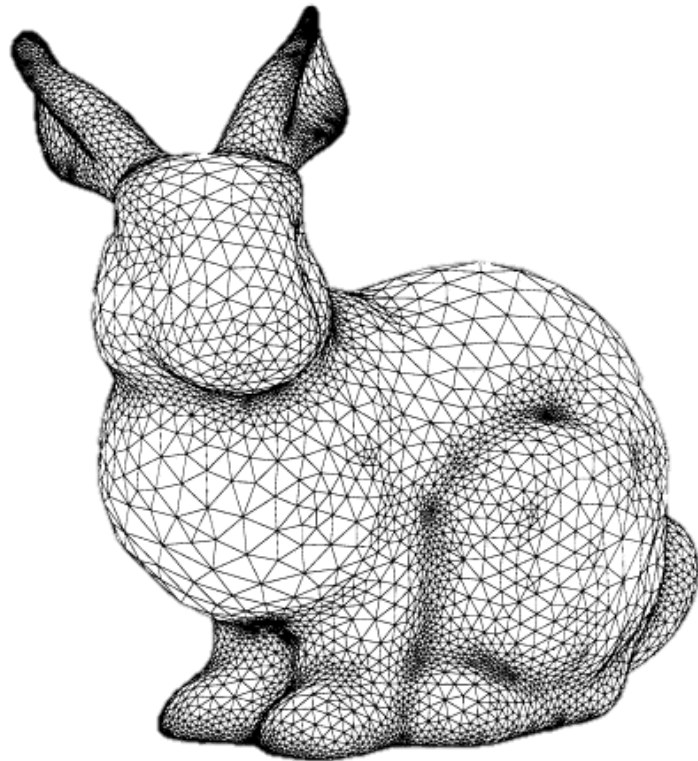
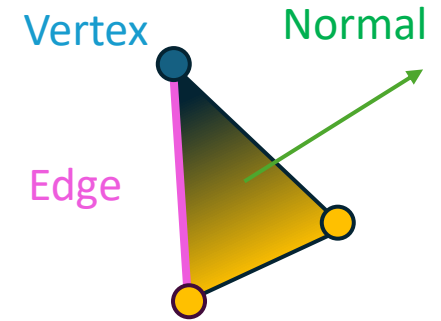
Rendered model

What is Geometry

- Geometry is the study of shapes.
- What mathematicians say:
 - Euclid: Euclidean geometry (2D, 3D)
 - (later) Klein: A Geometry is the study of its associated group of symmetries
- Hum.. That does not help us.
- And worse.. How do we encode geometry for a computer ?

What is Geometry

- How do we encode geometry for a computer ?
- You already saw a way, meshes



	Vertex	Normal	Attributes (eg: vertex color)
1	4876		
2	0.00273012 -0.75114 0.445136	-0.0934739 -0.751342 0.42608	-0.0161479 -0.704623 0.359556 1 1 1
3	0.19653 -0.748571 0.439299	0.127738 -0.739661 0.449994	0.190269 -0.690756 0.389317 1 1 1
4	0.275476 -0.730451 0.400454	0.269782 -0.69383 0.331669	0.370418 -0.689443 0.341495 1 1 1
5	0.370418 -0.689443 0.341495	0.453109 -0.690704 0.301089	0.466236 -0.742544 0.349141 1 1 1
6	0.370418 -0.689443 0.341495	0.381449 -0.749945 0.385336	0.336162 -0.750407 0.40263 1 1 1
7	0.00273012 -0.75114 0.445136	-0.0161479 -0.704623 0.359556	0.0478671 -0.694524 0.383326 1 1 1
8	0.336162 -0.750407 0.40263	0.275476 -0.730451 0.400454	0.370418 -0.689443 0.341495 1 1 1
9	0.370418 -0.689443 0.341495	0.466236 -0.742544 0.349141	0.381449 -0.749945 0.385336 1 1 1
10	0.127738 -0.739661 0.449994	0.00273012 -0.75114 0.445136	0.0478671 -0.694524 0.383326 1 1 1

3 vertices
= one triangle

- Meshes are composed by vertices
- their order creates triangles
- We can store other attributes such as colors
- Multiple formats exists for mesh data

Wavefront Object files (.obj)

- One of the simplest format is a Wavefront object file

```
# comment

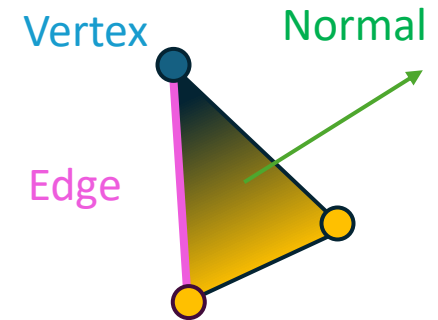
# Vertices
v 0.123 0.234 0.345 1.0
v ...
...

# Texture coordinates
vt 0.500 1 [0]
vt ...
...

# Normals
vn 0.707 0.000 0.707
vn ...
...

# Faces
f 1 2 3
f 3/1 4/2 5/3
f 6/4/1 3/5/3 7/6/5
f 7//1 8//2 9//3
f ...
```

- We can specify vertices
- Texture coordinates
- Normals
- Faces of our triangle (in term of the vertex number)
- Faces can also use the texture coordinate and normals



Polygon File Format (.ply)

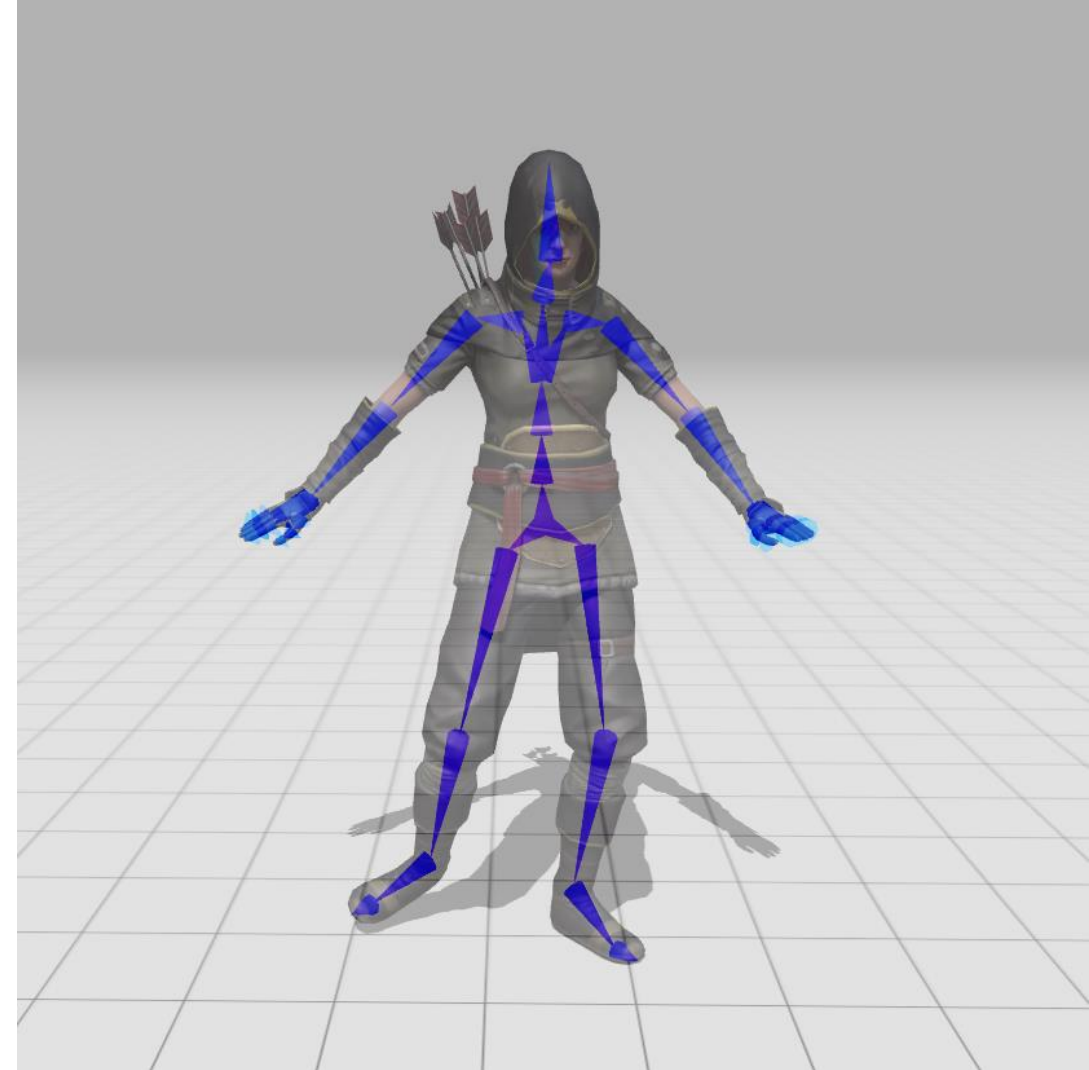
- Another very common file format is the Polygon File Format.

```
ply
format ascii 1.0
comment Created in Blender version 4.0.2
element vertex 5
property float x
property float y
property float z
property float nx
property float ny
property float nz
property float s
property float t
element face 5
property list uchar uint vertex_indices
end_header
1 1 1 0.5773503 0.5773503 0.5773503 0.625 0.5
-1 1 1 -0.5773503 0.5773503 0.5773503 0.875 0.5
-1 -1 1 -0.5773503 -0.5773503 0.5773503 0.875 0.75
1 -1 1 0.5773503 -0.5773503 0.5773503 0.625 0.75
1 -1 -1 0.5773503 -0.5773503 -0.5773503 0.375 0.75
-1 -1 1 -0.5773503 -0.5773503 0.5773503 0.625 1
```

- Its a useful format because we can specify and kind of property
- And number of elements per face
- And types of data
- A useful viewer for .ply files is **MeshLab**

Problems with .obj and .ply

- There are several problems with .obj and .ply files:
 - It is not compressed, so files tend to grow big
 - There is no description of the rest of the scene !
 - Only one object
 - No lights
 - No camera
 - No skeleton (for animations)
 - Etc.
- Newer formats try to solve this
- There are many proprietary formats
 - FBX, 3ds, Maya files, dwf
- And open-source ones
 - Blender files, COLLADA, glTF, STL

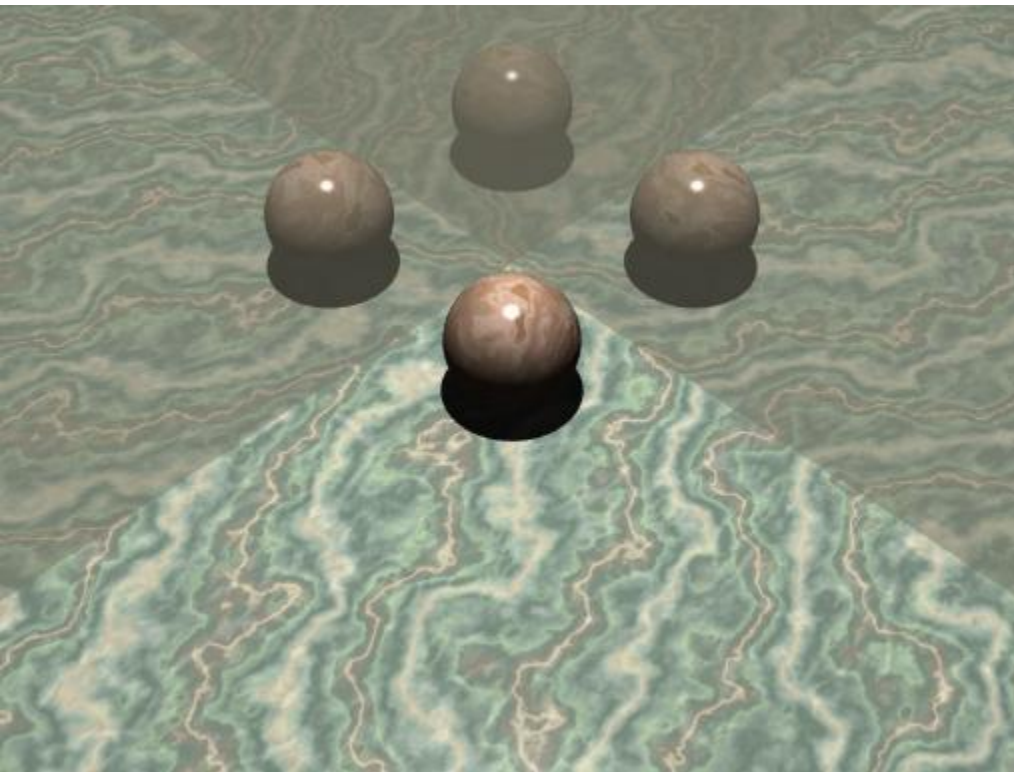


Graphics Library Transmission Format (glTF)

- The Graphics Library Transmission Format (.glTF) is a recent file format that is very tries to be as complete as possible
- It is maintained by the Khronos Group
 - The group responsible for OpenGL, Vulkan, WebGL and OpenCL
- It represents scenes as json files (with possibly blobs of data)
- Allows to store ressources directly inside
- You can also store matrices for transformations, animations, etc.
- Okay, we know how to store meshes..
- Are there other ways of encoding geometry ?

POV-Rays

- POV-Ray is a way to encode scenes for raytracing
- Its renderer allows very realistic images



```
//Files with predefined colors and textures
#include "colors.inc"
#include "glass.inc"
#include "golds.inc"
#include "metals.inc"
#include "stones.inc"
#include "woods.inc"

//Place the camera
camera {
  sky <0,0,1>           //Don't change this
  direction <-1,0,0>    //Don't change this
  right <-4/3,0,0>      //Don't change this
  location <10,0,10>    //Camera location
  look_at <0,0,0>       //Where camera is pointing
  angle 30              //Angle of the view--increase to see more, decrease to see less
}

//Ambient light to "brighten up" darker pictures
global_settings { ambient_light White }

//Place a light--you can have more than one!
light_source {
  <0,0,10>              //Change this if you want to put the light at a different point
  color White*2          //Multiplying by 2 doubles the brightness
}

//Set a background color
background { color White }

//Create a "floor"
plane {
  <0,0,1>, 0             //This represents the plane 0x+0y+z=0
  texture { T_Silver24 } //The texture comes from the file "metals.inc"
}

//Define two mildly reflective planes meeting at right angles
//-x+y+0z=0 translated 1 unit in the direction <-1,1,0>
#declare mirror1=plane {
  <-1,1,0>, 1
  texture {T_Silver_1C}
};

//-x-y+0z=0 translated 1 unit in the direction <-1,-1,0>
#declare mirror2=plane {
  <-1,-1,0>, 1
  texture {T_Silver_1C}
};

//Display the mirrors
mirror1
mirror2

sphere{<0,0,.5>,.5 texture{T_Stone1}}
```

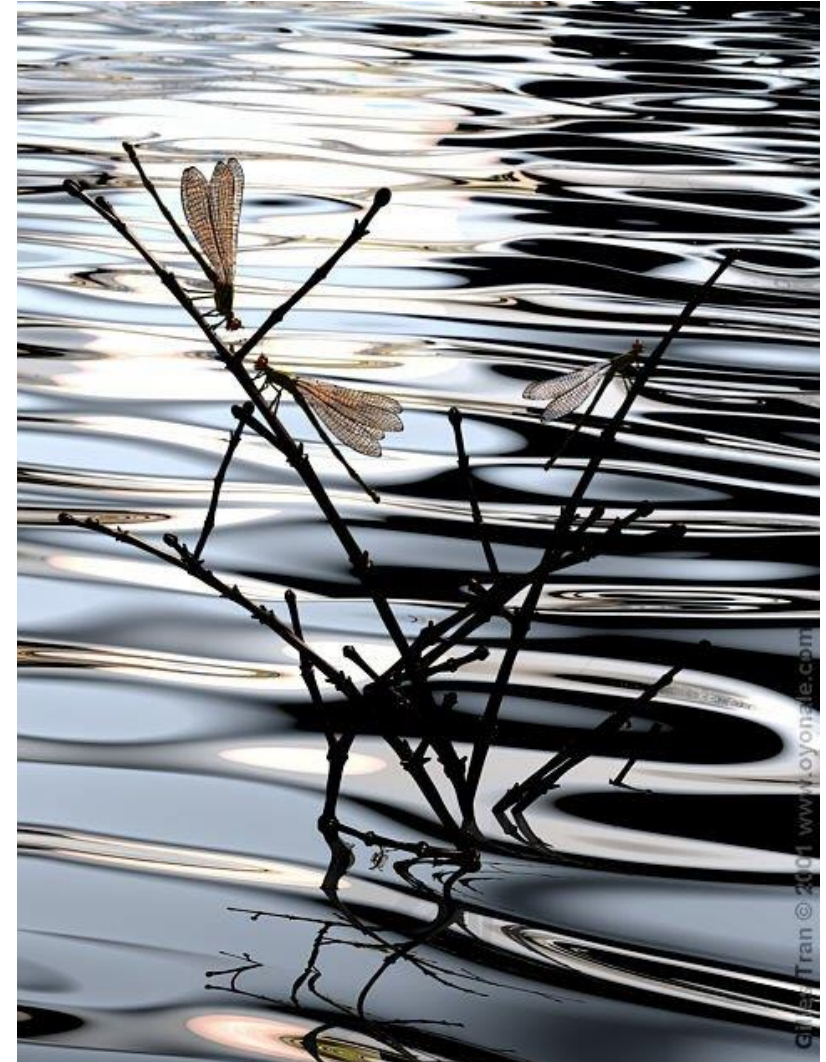
We use Euclidean Geometry in POV-Ray

```
sphere { <0,0,1.5>, 1 texture {T_Stone1} }
```

POV-Rays

! Not widely used format

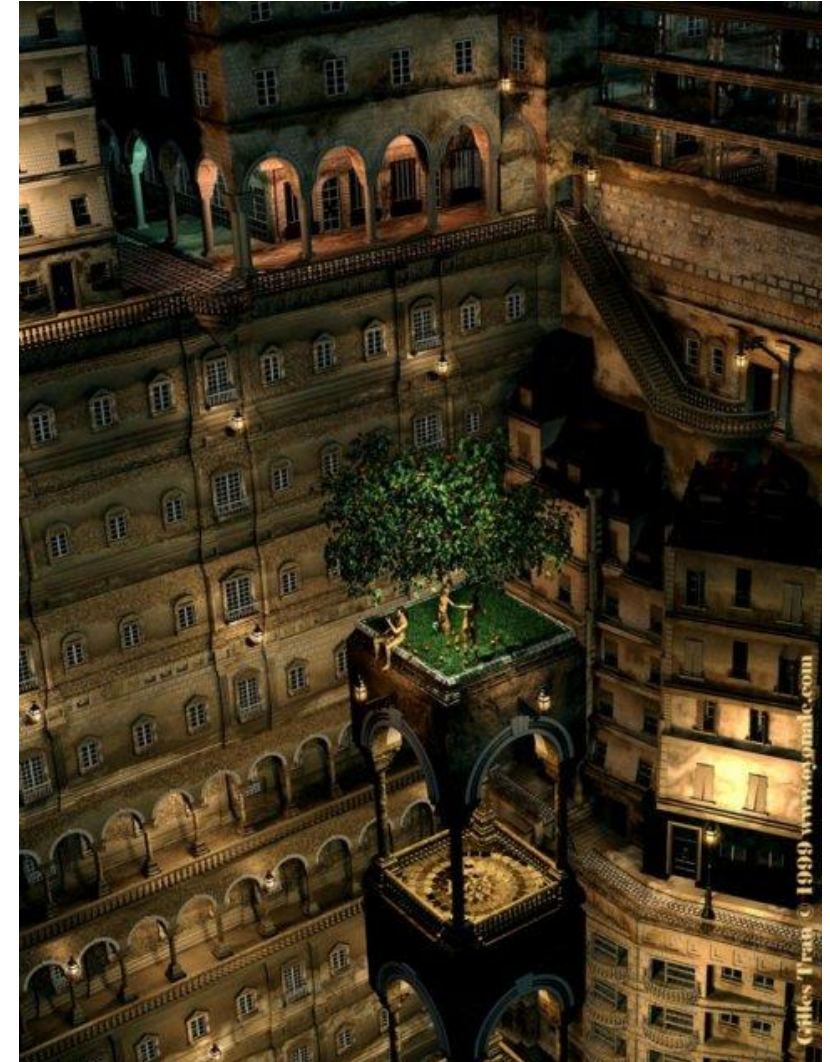
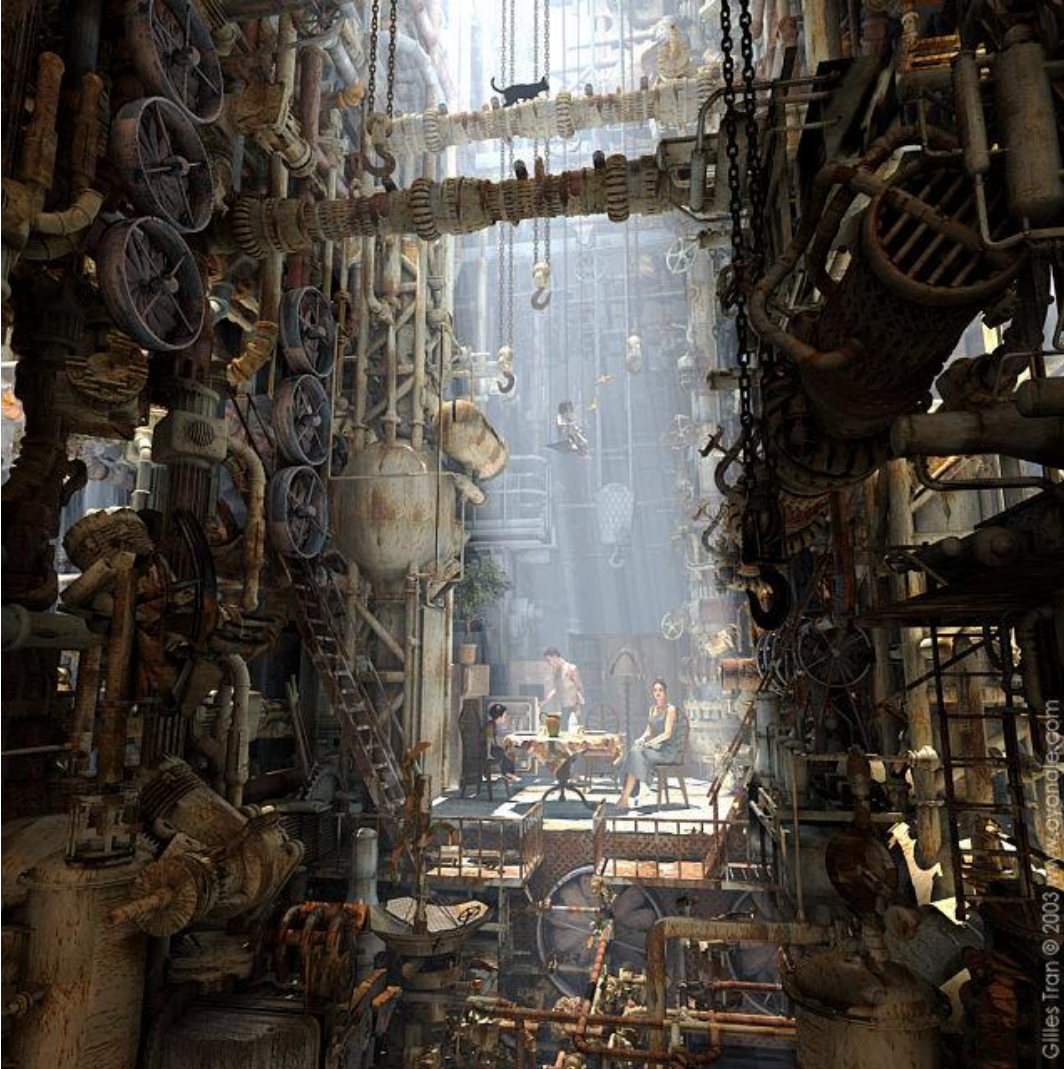
- POV-Ray is a way to encode scenes for raytracing
- Its renderer allow very realistic images



POV-Rays

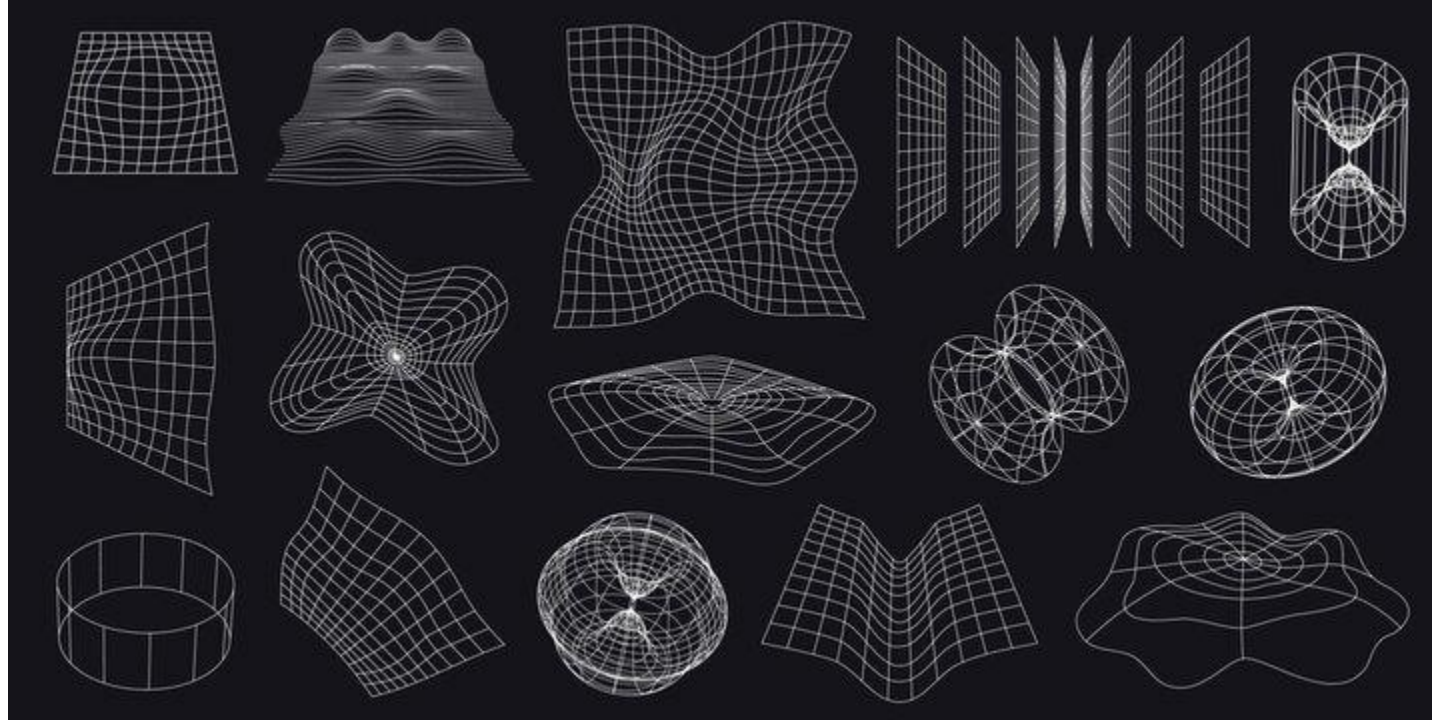
! Not widely used format

- POV-Ray is a way to encode scenes for raytracing
- Its renderer allow very realistic images



Other ways of Encoding Geometry

- Are there other ways of encoding geometry ?



Other ways of Encoding Geometry

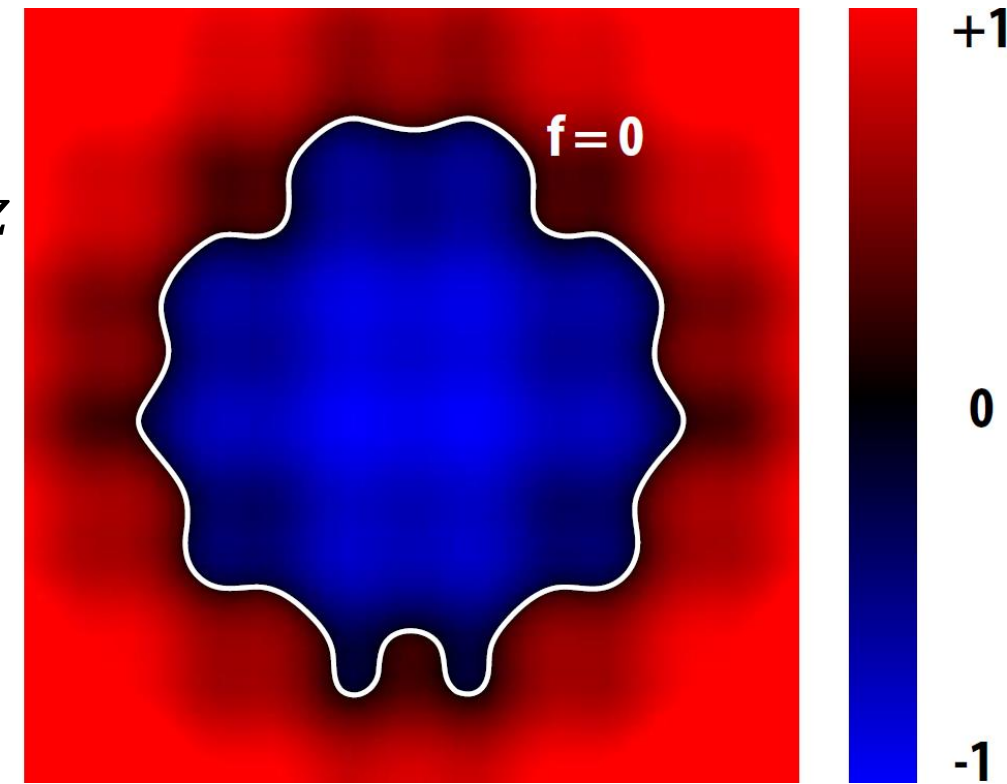
- Are there other ways of encoding geometry ?
- It turns out that meshes are only one way to encode geometry
- But geometrical objects can be described in many ways
- In this course we were mainly limited to the interaction with the vertices, which led to sometimes difficult operations
 - E.g., intersection between a ray and all the vertices of a scene is a lot of computations
- Some ways have better operations on them
 - E.g., Easier to do intersections with the geometry
- But, they make it harder to do other operations !
- Let's explore some of them.

What is Geometry

- Geometrical objects can be separated in two categories, implicit and explicit
 - Implicit:
 - We have an equation that describe our object, closer to mathematics
 - The advantage is that they offers a continuous representation of our object, e.g.:
 - $x^2 + y^2 = 1$ (Euclidean Geometry)
 - $\gamma: t \mapsto (\varphi_1(t), \varphi_2(t), \varphi_3(t))$ (a parametric curve)
 - Level set
 - Algebraic surface (the zeros of some function)
 - $(\cos \theta, \sin \theta)$
 - Etc..
 - Explicit:
 - The objects of interest are described by physical quantities such as
 - Meshes
 - Point clouds
 - Light Fields
 - NURBS
 - Etc..

Algebraic Surface (Implicit)

- Surface is the zero set of a polynomial in x, y, z
- $x^2 + y^2 + z^2 = 1$
- $\left(R - \sqrt{x^2 + y^2}\right)^2 + z^2 = r^2$

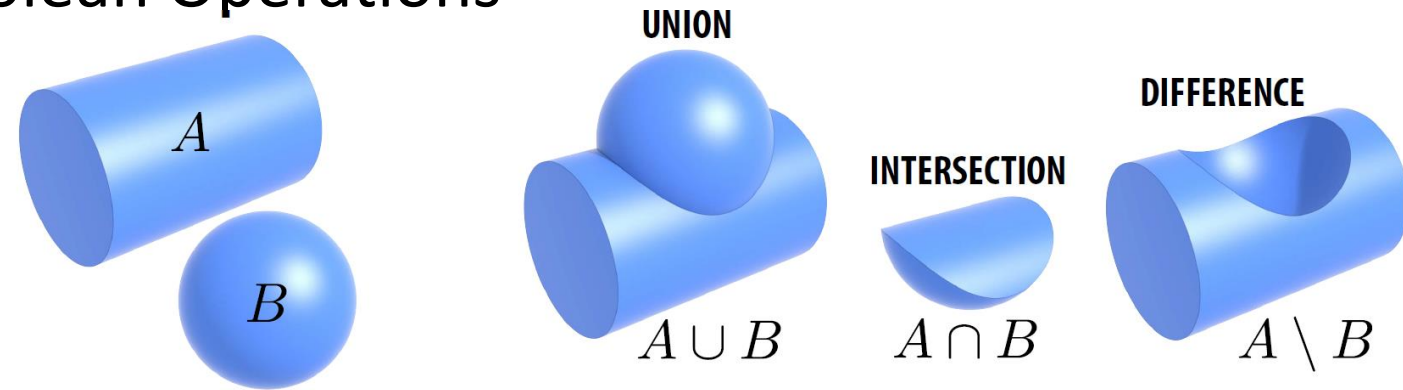


- Real shapes are very difficult to create like this
- How can we create complex shapes ?

Constructive Solid Geometry (Implicit)

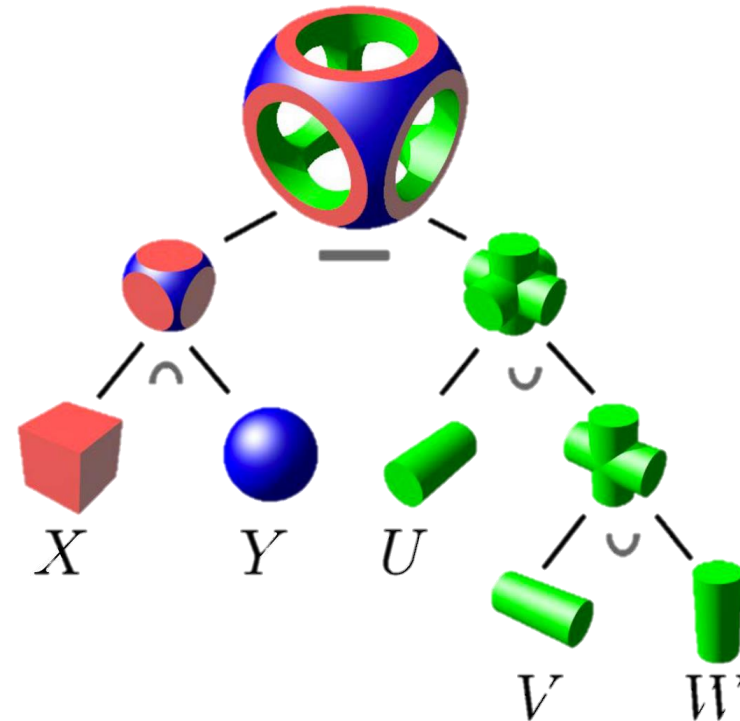
- Build complicated shapes via Boolean Operations

- $A, B; A \cup B; A \cap B; A \setminus B$



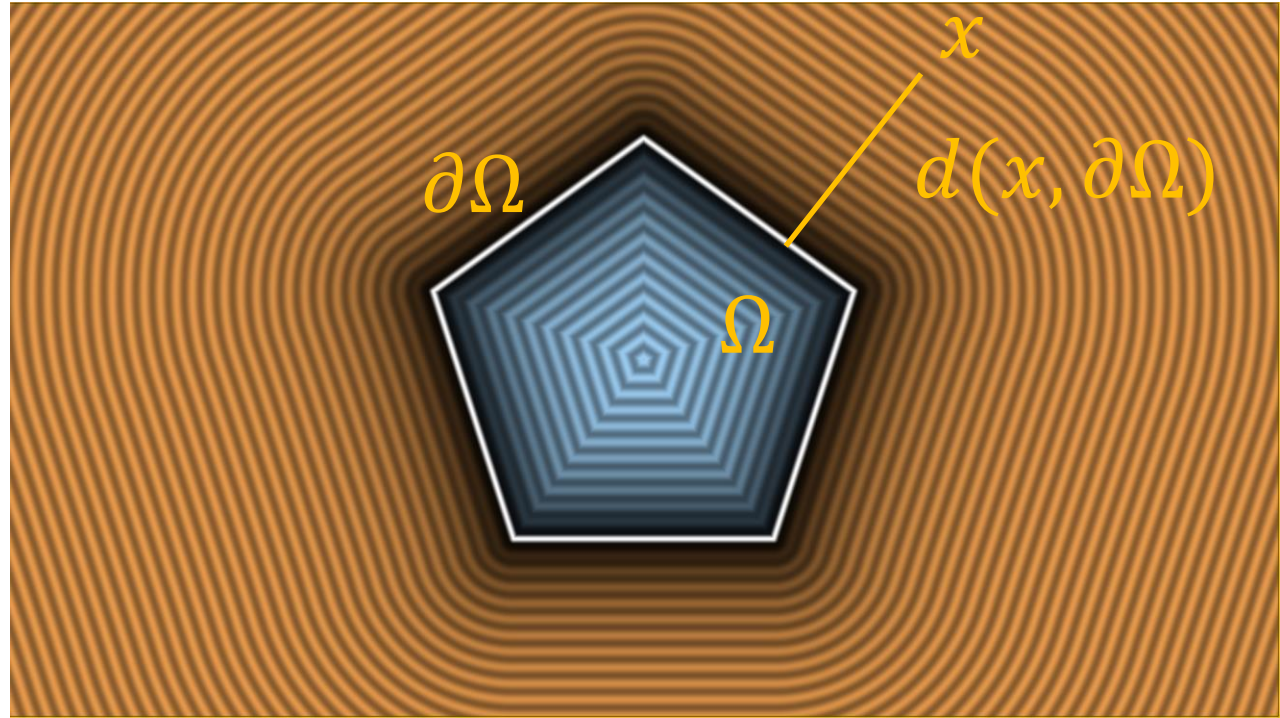
- Chain together expressions:

- $(X \cap Y) \setminus (U \cup V \cup W)$



Signed Distance Fields (Implicit)

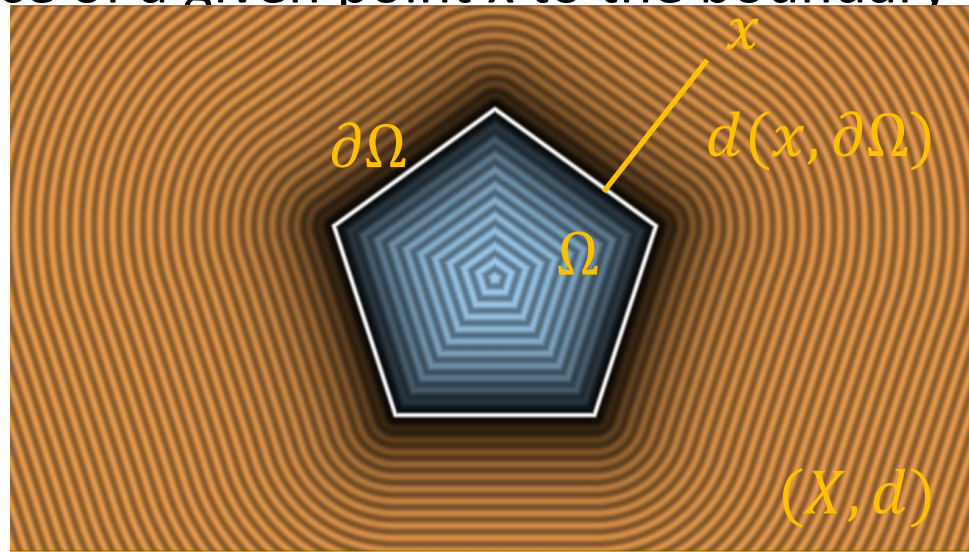
- the orthogonal distance of a given point x to the boundary of a set Ω in a metric space
- The sign depends on if $x \in \Omega$ or not:
 - + if inside of Ω
 - - if outside of Ω



- The value decreases as we approach the border of Ω ($\partial\Omega$)
- The value is zero at the border.

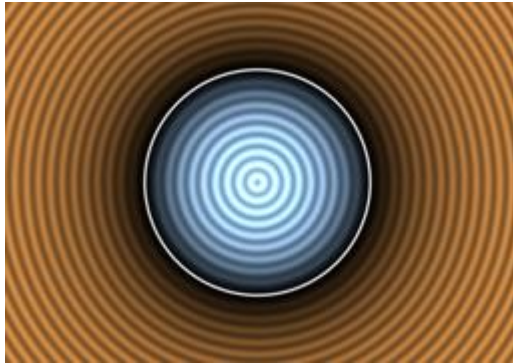
Signed Distance Fields (Implicit)

- Formally:
 - Let Ω be a subset of a metric space X with metric d .
 - Let $\partial\Omega$ be its boundary.
 - Distance between a point $x \in X$ and the subset $\partial\Omega$ is defined as:
 - $d(x, \partial\Omega) = \inf_{y \in \partial\Omega} d(x, y)$
 - The signed distance function from a point $x \in X$ to Ω is:
 - $f(x) = \begin{cases} d(x, \partial\Omega) & \text{if } x \in \Omega \\ -d(x, \partial\Omega) & \text{if } x \notin \Omega \end{cases}$
 - the orthogonal distance of a given point x to the boundary of a set Ω in a metric space



Examples of SDF

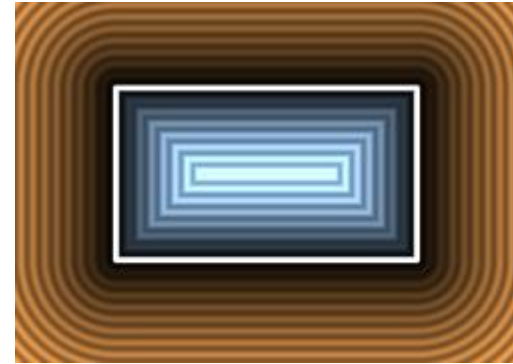
- The function f defines the distance at every point in space to the surface
- It is different than an algebraic surface that define the border of the object !
- Examples:



```
float sdCircle( vec2 p, float r )
{
    return length(p) - r;
}
```

Circle (exact)

<https://www.shadertoy.com/view/3ltSW2>



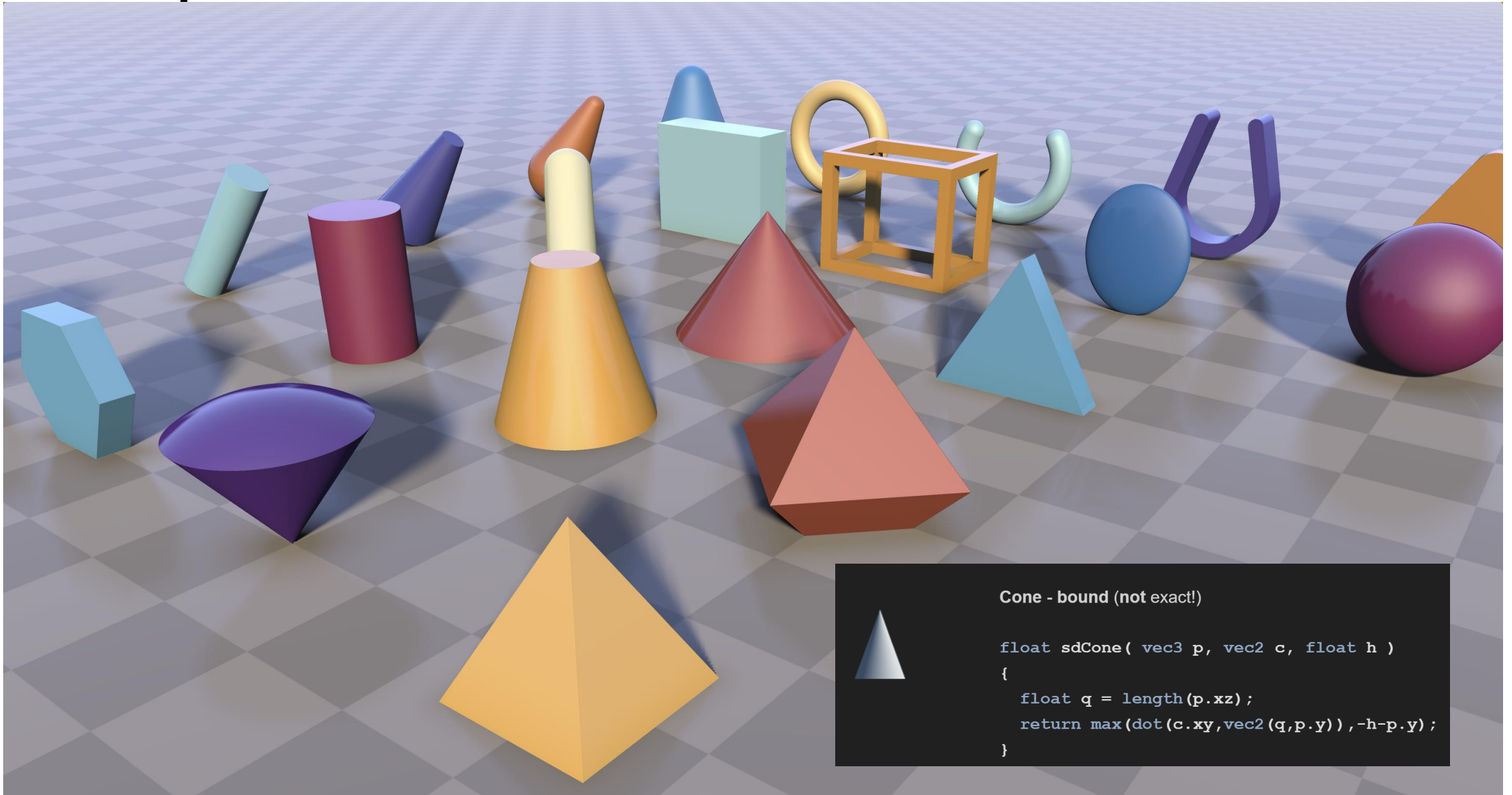
```
float sdBox( in vec2 p, in vec2 b )
{
    vec2 d = abs(p)-b;
    return length(max(d,0.0)) + min(max(d.x,d.y),0.0);
}
```

Box (exact)

<https://www.youtube.com/watch?v=62-pRVZuS5c>

Examples of SDF in 3D

<https://www.shadertoy.com/view/Xds3zN>



Why SDF ?

- Ok, this representation is interesting..
- But.. What could we create with it ?
- Why is it used ?

Signed Distance Fields (Implicit)

- We can do a lot with SDF!



<https://www.shadertoy.com/view/WsSBzh>

<https://www.youtube.com/watch?v=8--5LwHRhjk> ²³

Signed Distance Fields (Implicit)

- We can do a lot with SDF!



<https://www.shadertoy.com/view/WsSBzh>

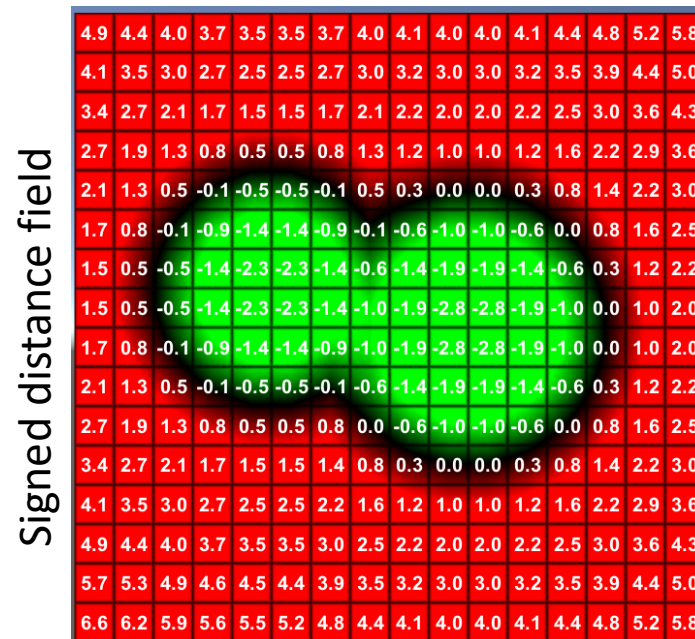
<https://www.youtube.com/watch?v=8--5LwHRhjk> 25

Signed Distance Fields (Implicit)

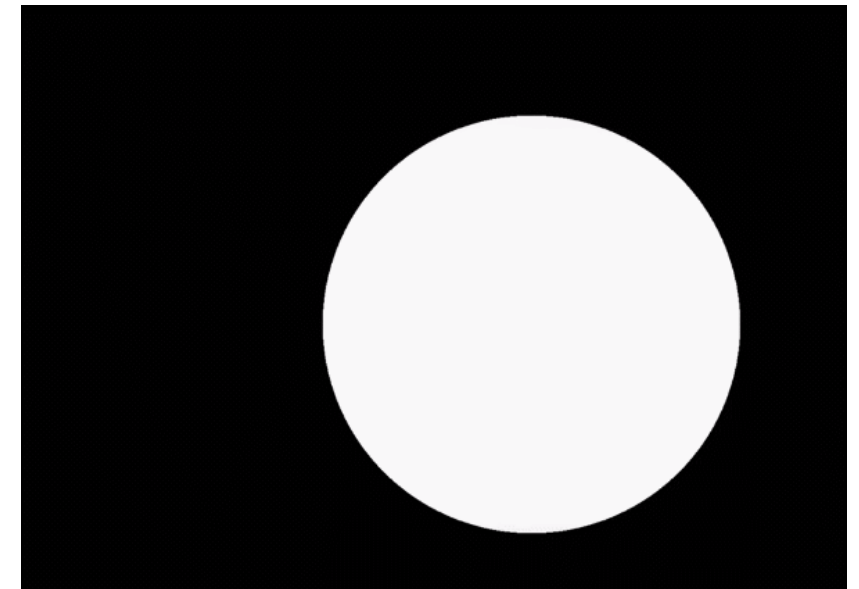
- The previous image was generated with a unique mathematical function which output the color given a position in space.

$$f(x) = \text{very_complicated_sdf_function}()$$

- We can imagine this function as a discrete grid, containing the distance from the surface at each point
- Now suppose you want to render a scene described in SDF
- How to do that ?

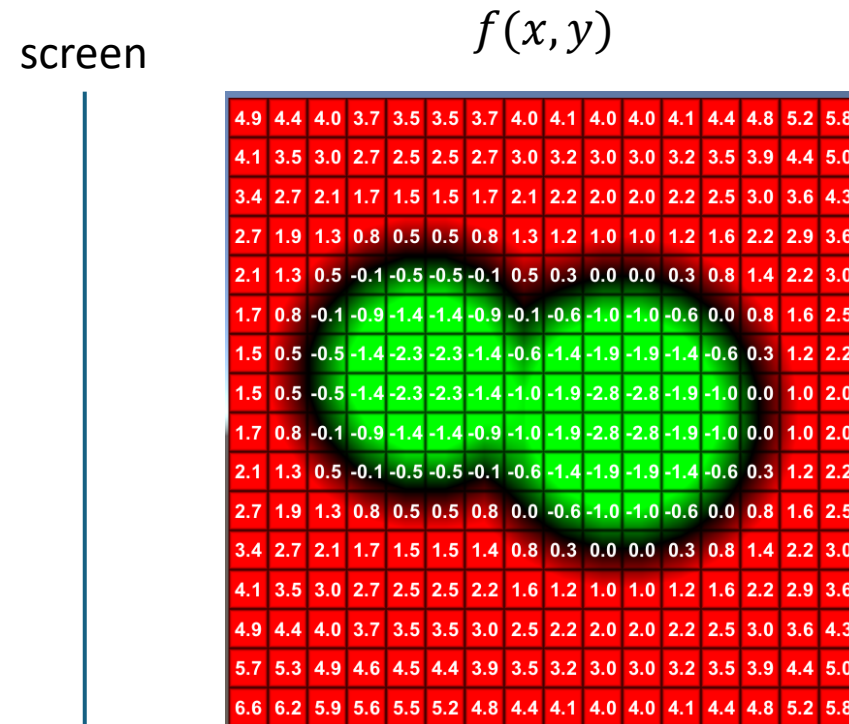


$f(x)$



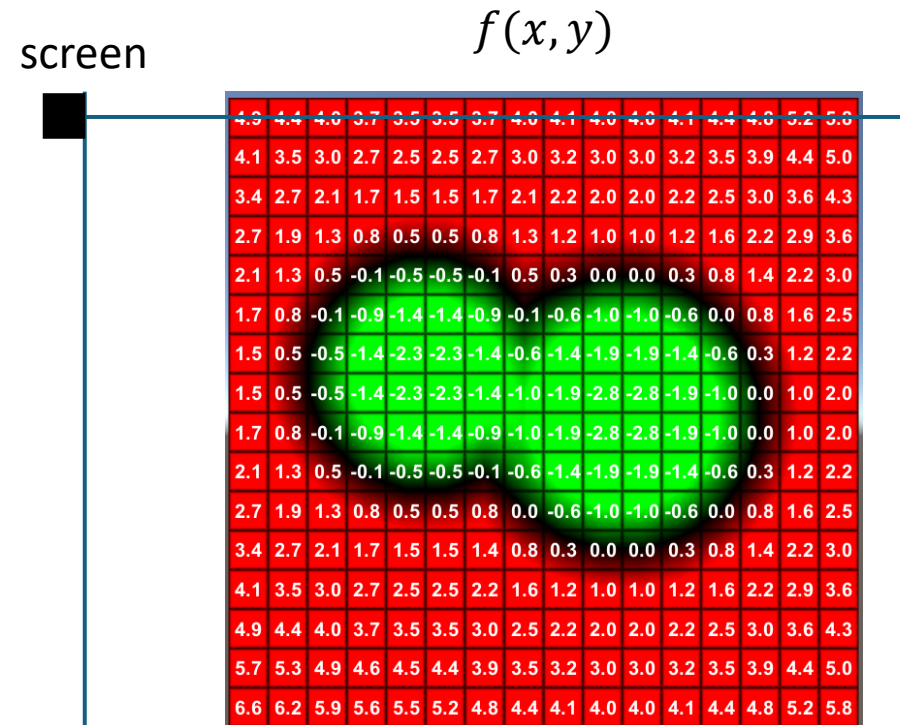
Rendering SDFs

- How can we render it if the screen is there:



Rendering SDFs

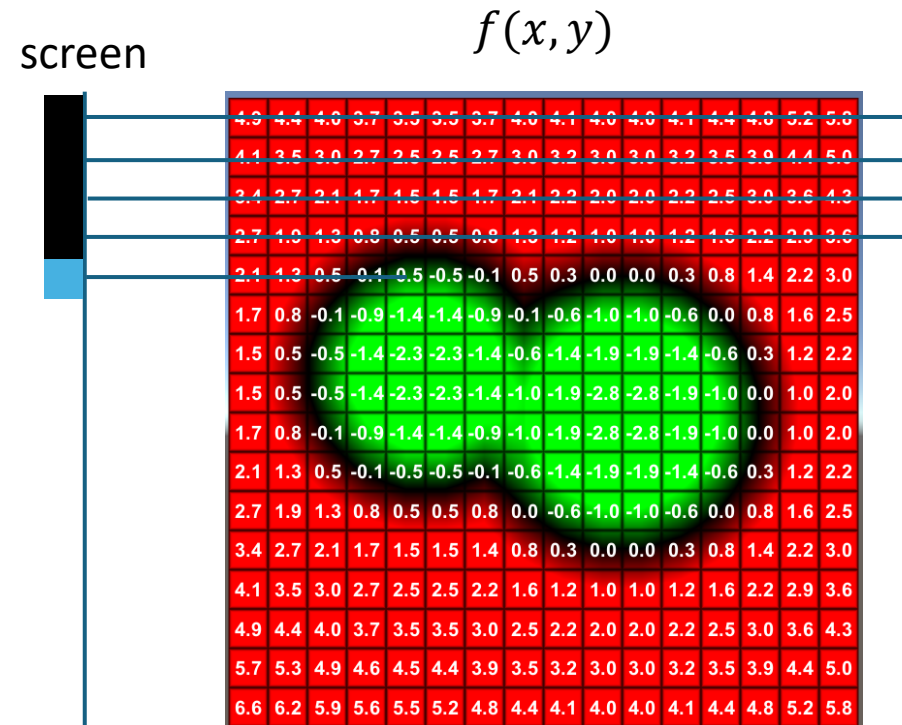
- How can we render it if the screen is there:



- We can launch a ray from a pixel (raytracing), check if we find a value near 0
- We set its color depending on the distance, no luck here, pure black.

Rendering SDFs

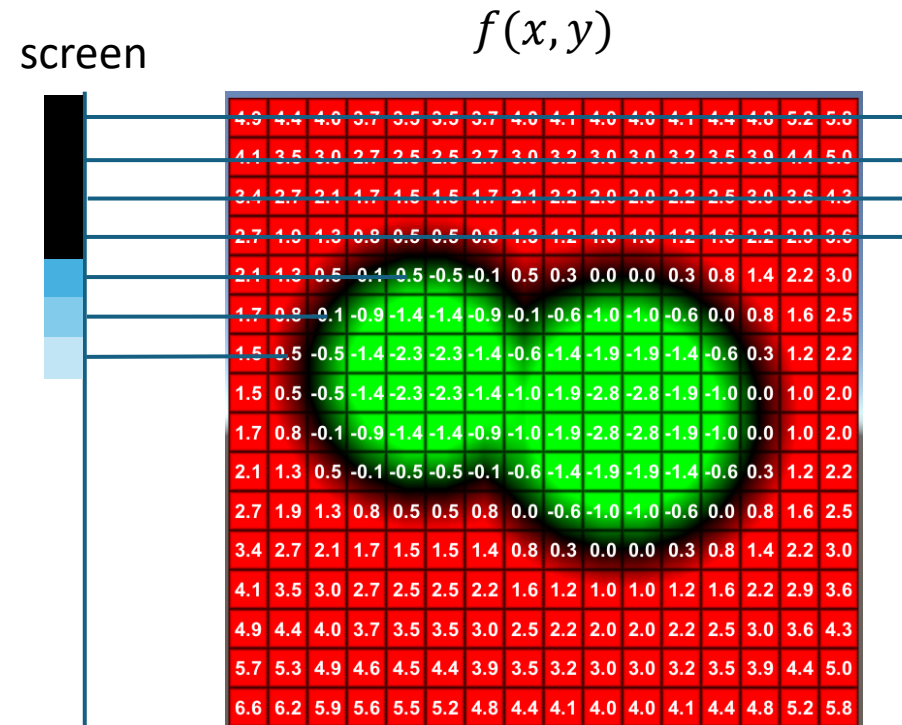
- How can we render it if the screen is there:



- We can launch a ray from a pixel (raytracing), check if we find a value near 0
- We start to see a border

Rendering SDFs

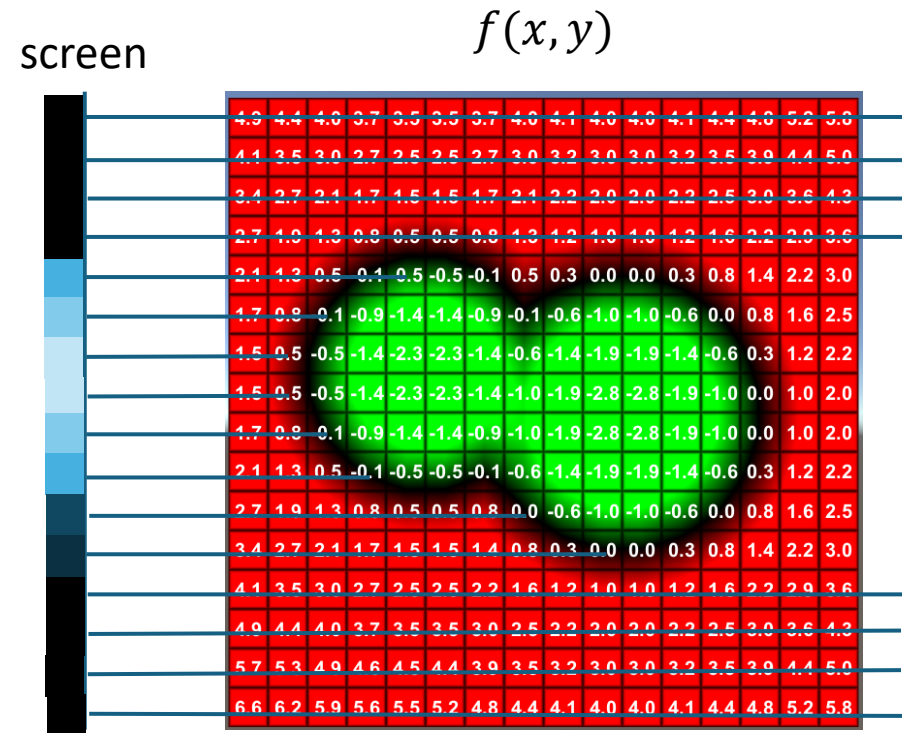
- How can we render it if the screen is there:



- We can launch a ray from a pixel (raytracing), check if we find a value near 0
- Let's colorize the rest

Rendering SDFs

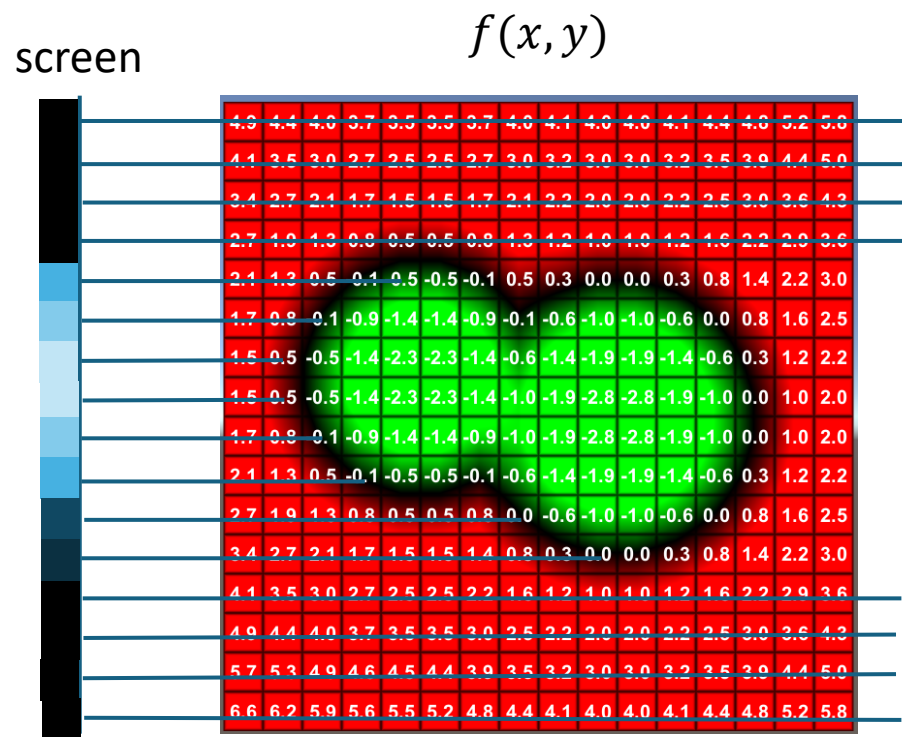
- How can we render it if the screen is there:



- We can launch a ray from a pixel (raytracing), check if we find a value near 0
- When we launch a ray, how many positions we should check ?

Rendering SDFs

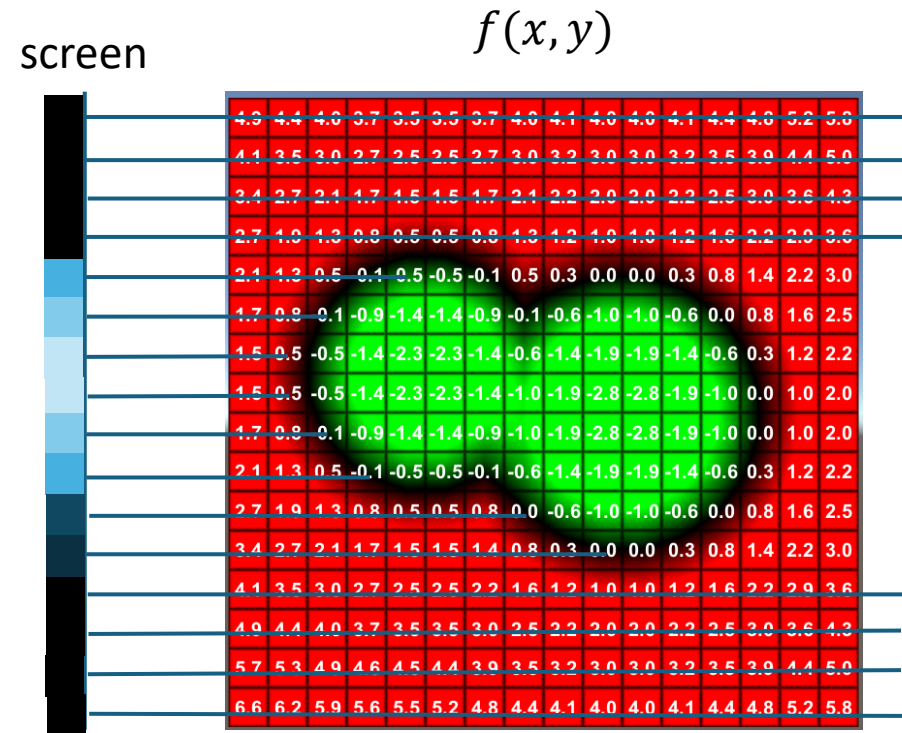
- How can we render it if the screen is there:



- When we launch a ray, how many positions we should check ?
- If we evaluate at N positions, we have N checks to perform
 - If the value is not exactly 0, its even worse, we need to compare with previous position.

Rendering SDFs

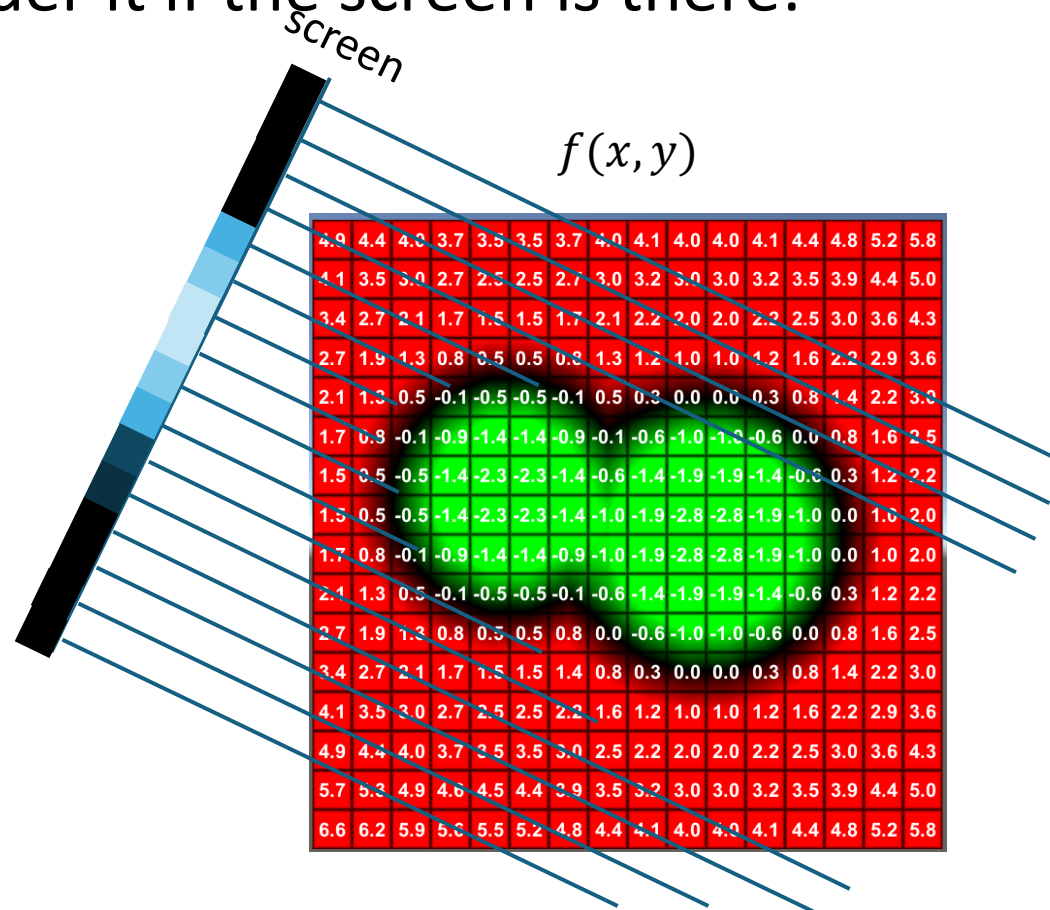
- How can we render it if the screen is there:



- Thankfully, the checks are *memory aligned* (fast for a computer)
- But what about..

Rendering SDFs

- How can we render it if the screen is there:

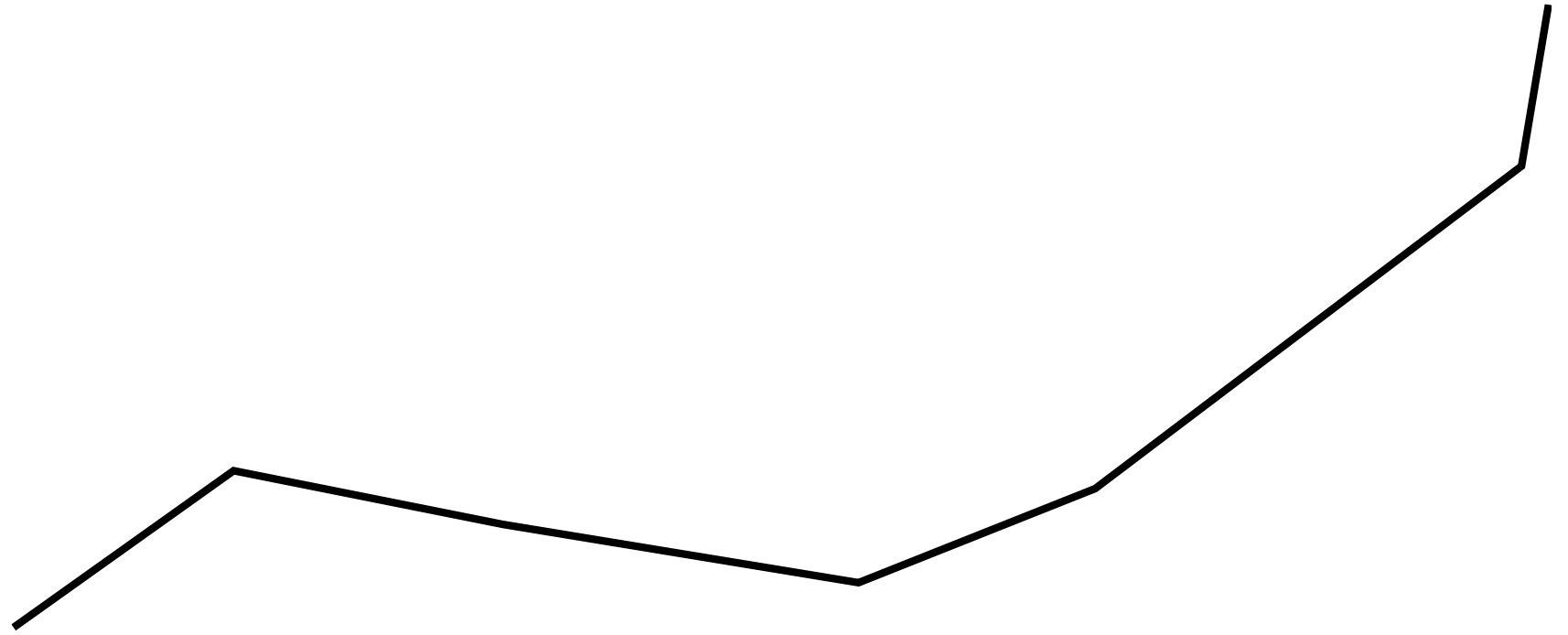


- Not memory aligned at all, very expensive 😞
- what could we do?

Raymarching

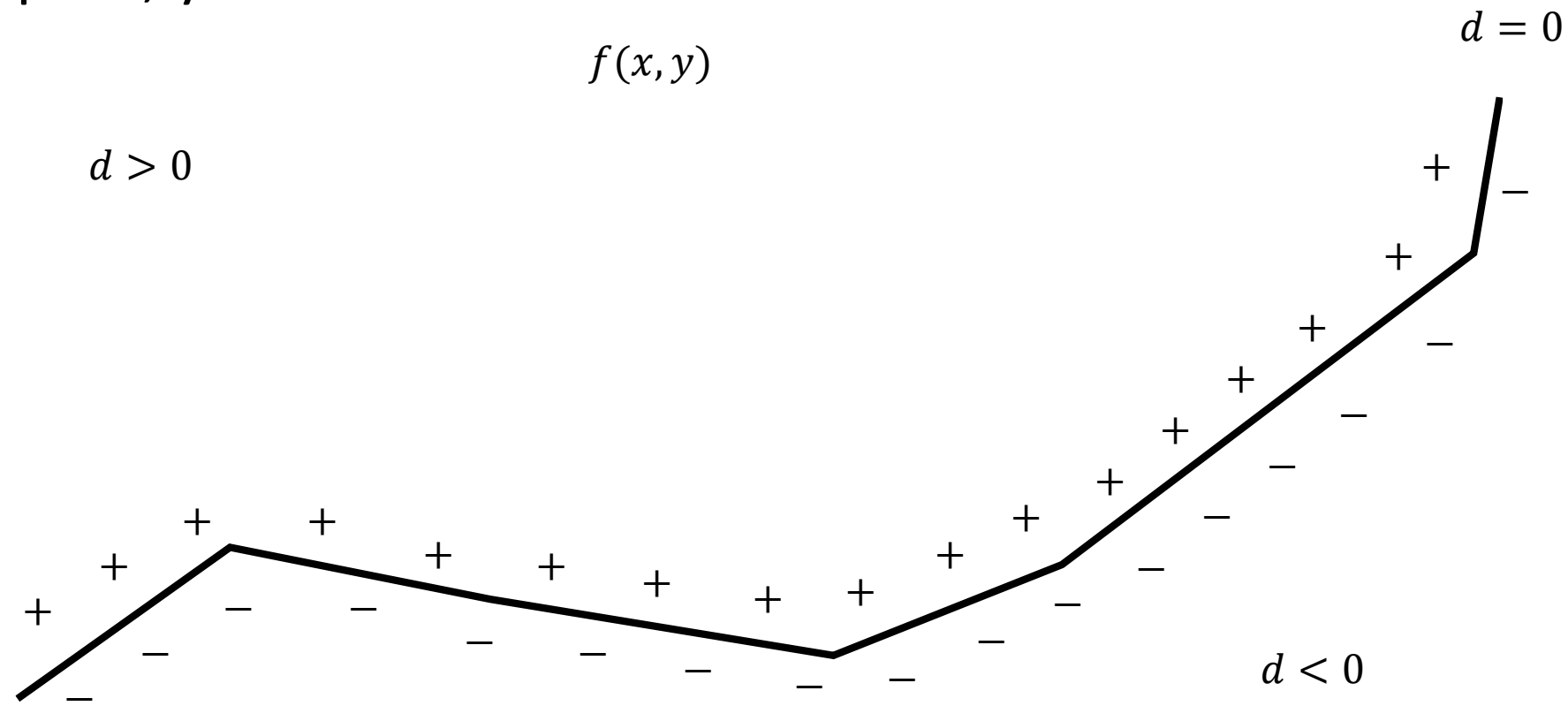
- Suppose this is our surface (we don't have an equation for it!)
- We have a function f that describe the distance from it at each point

$$f(x, y)$$



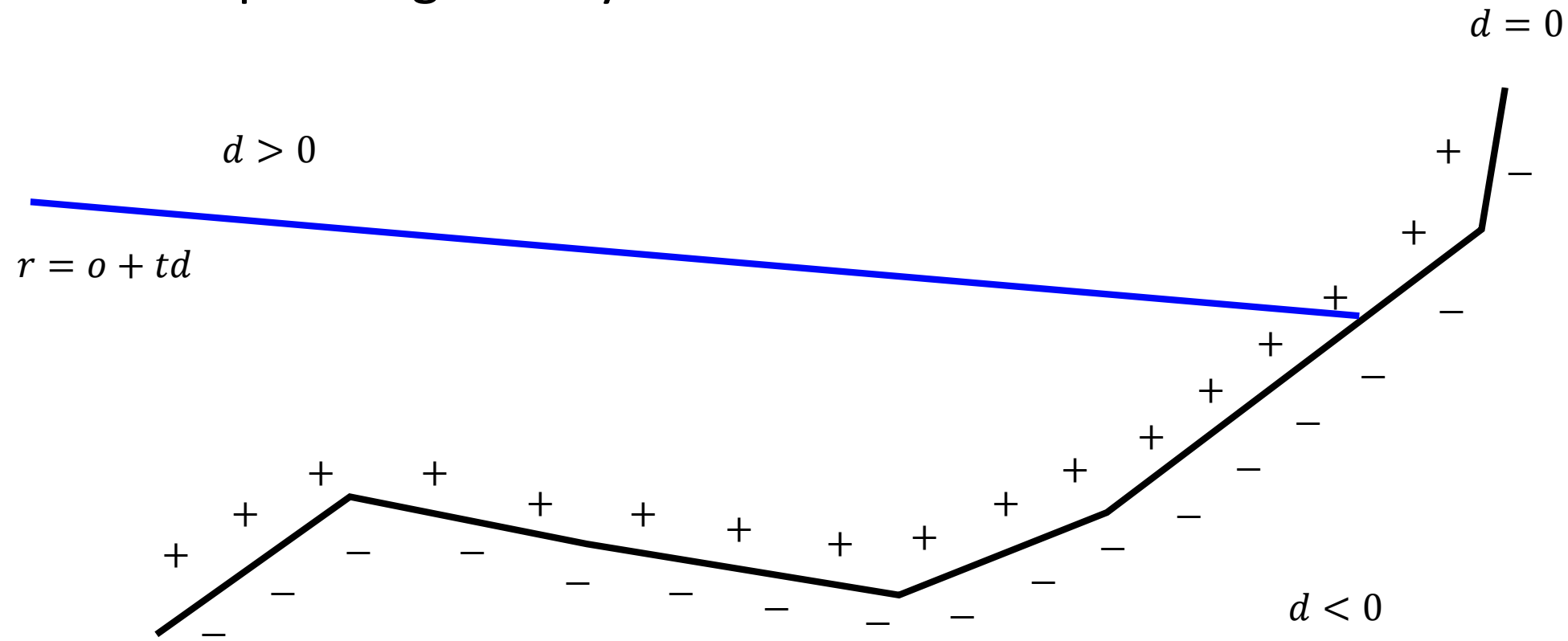
Raymarching

- Suppose this is our surface
- We have a function f that describe the distance from it at each point
- At every position in space, you have the distance from it



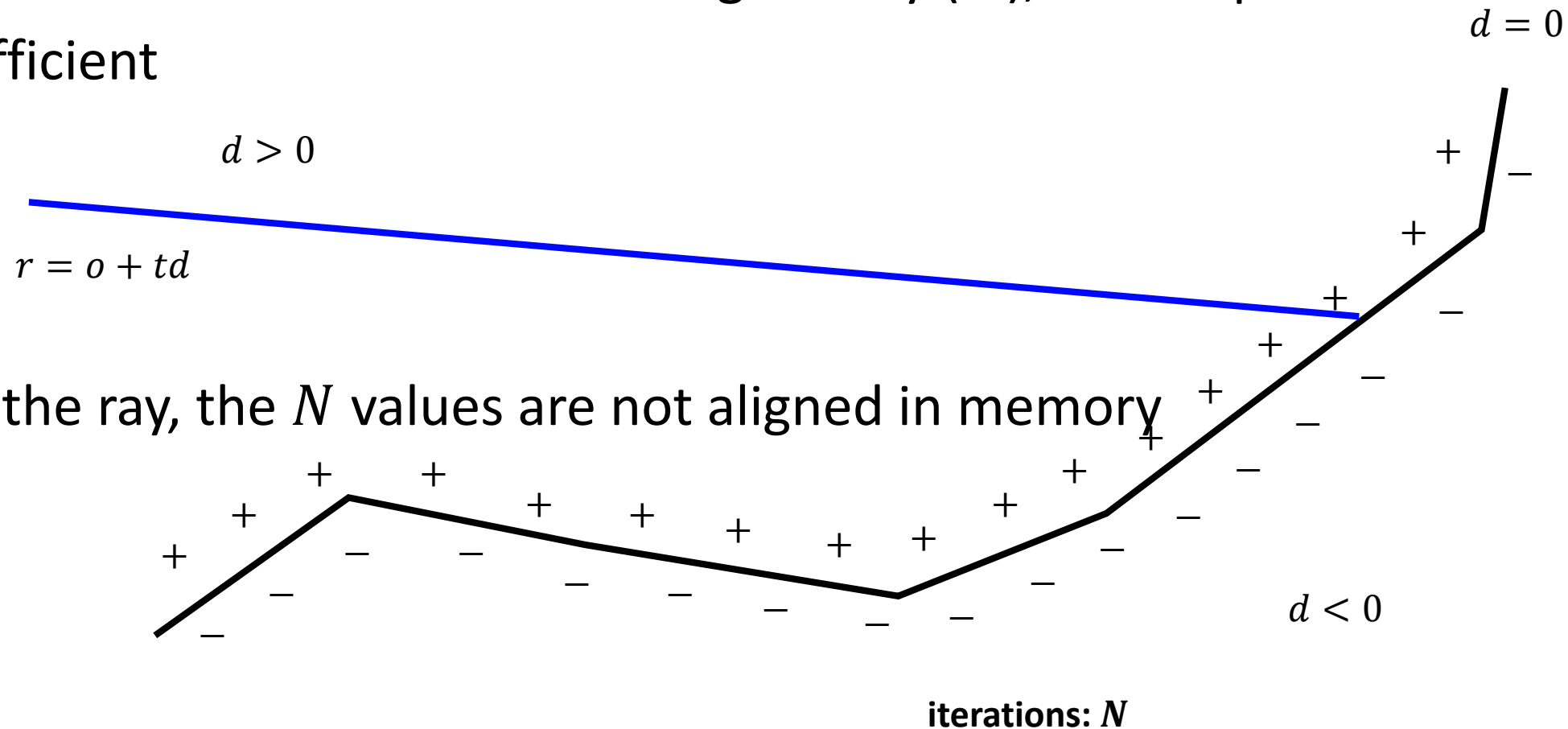
Raymarching

- You want to send a ray from the screen to retrieve the function
- But, how to find the intersection with the surface ?
- Suppose there are N steps along the ray



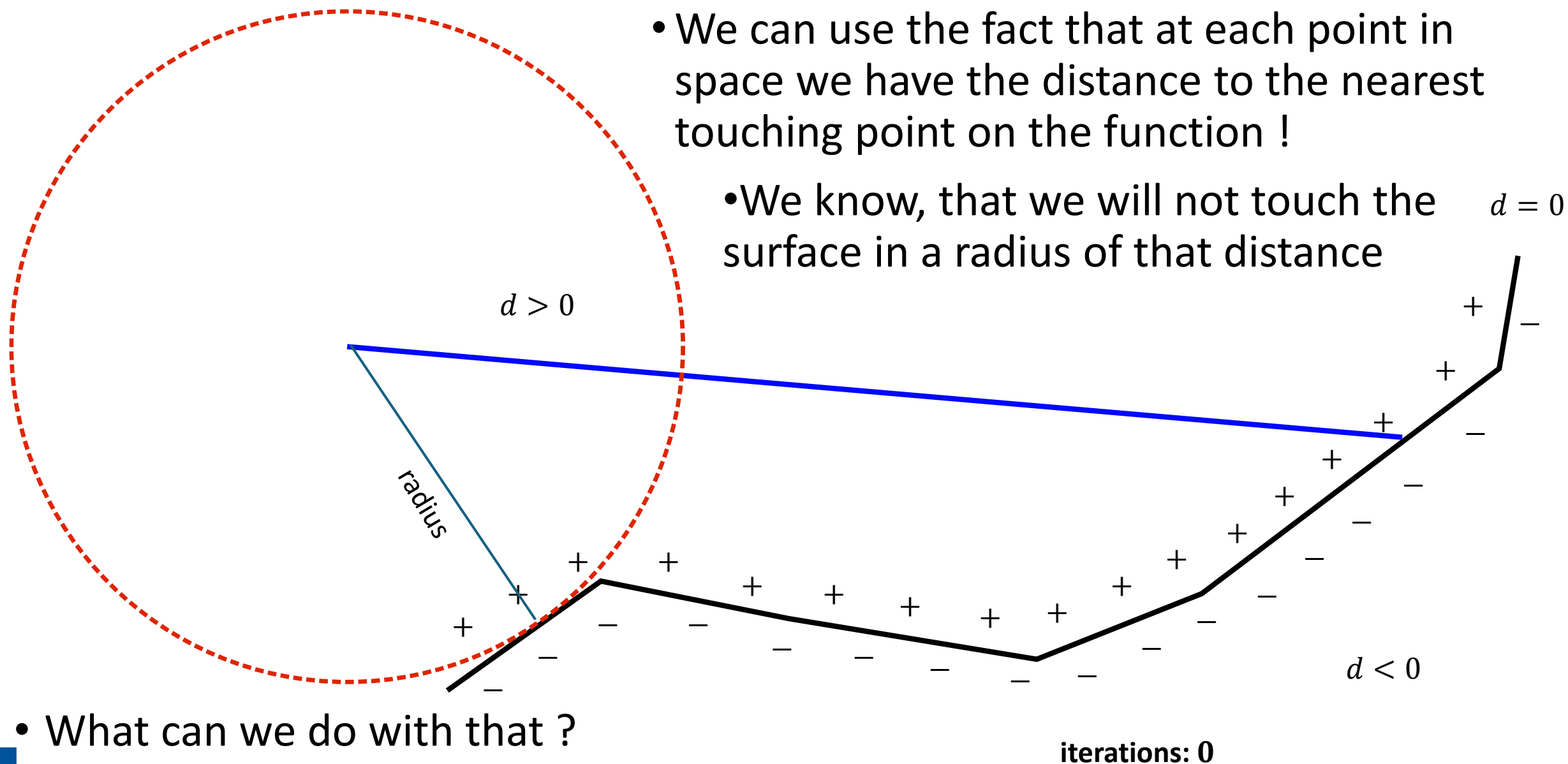
Raymarching

- You want to send a ray from the screen to retrieve the function
- But, how to find the intersection with the surface ?
- Yes, we could evaluate all the elements along the ray (N), and stop when 0
- But not very efficient



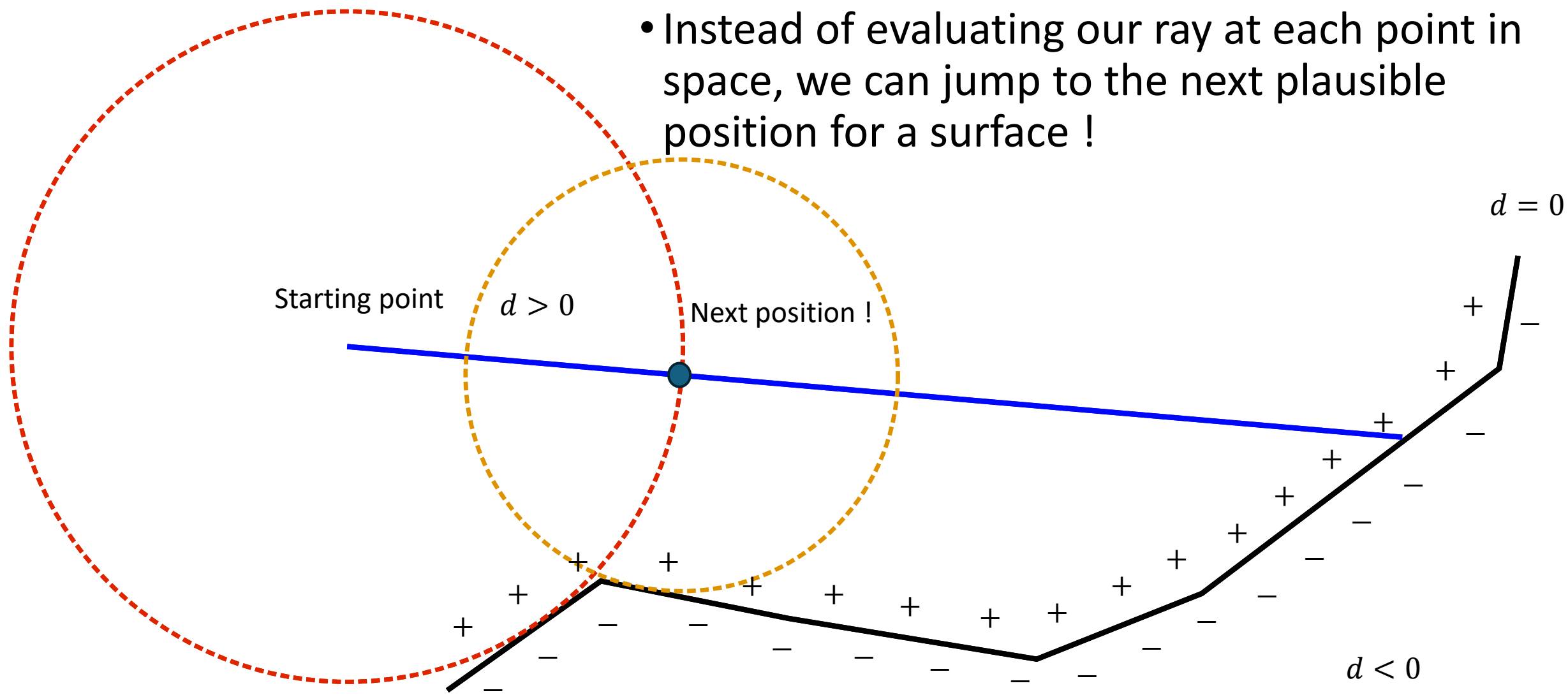
- What else ?

Raymarching



Raymarching

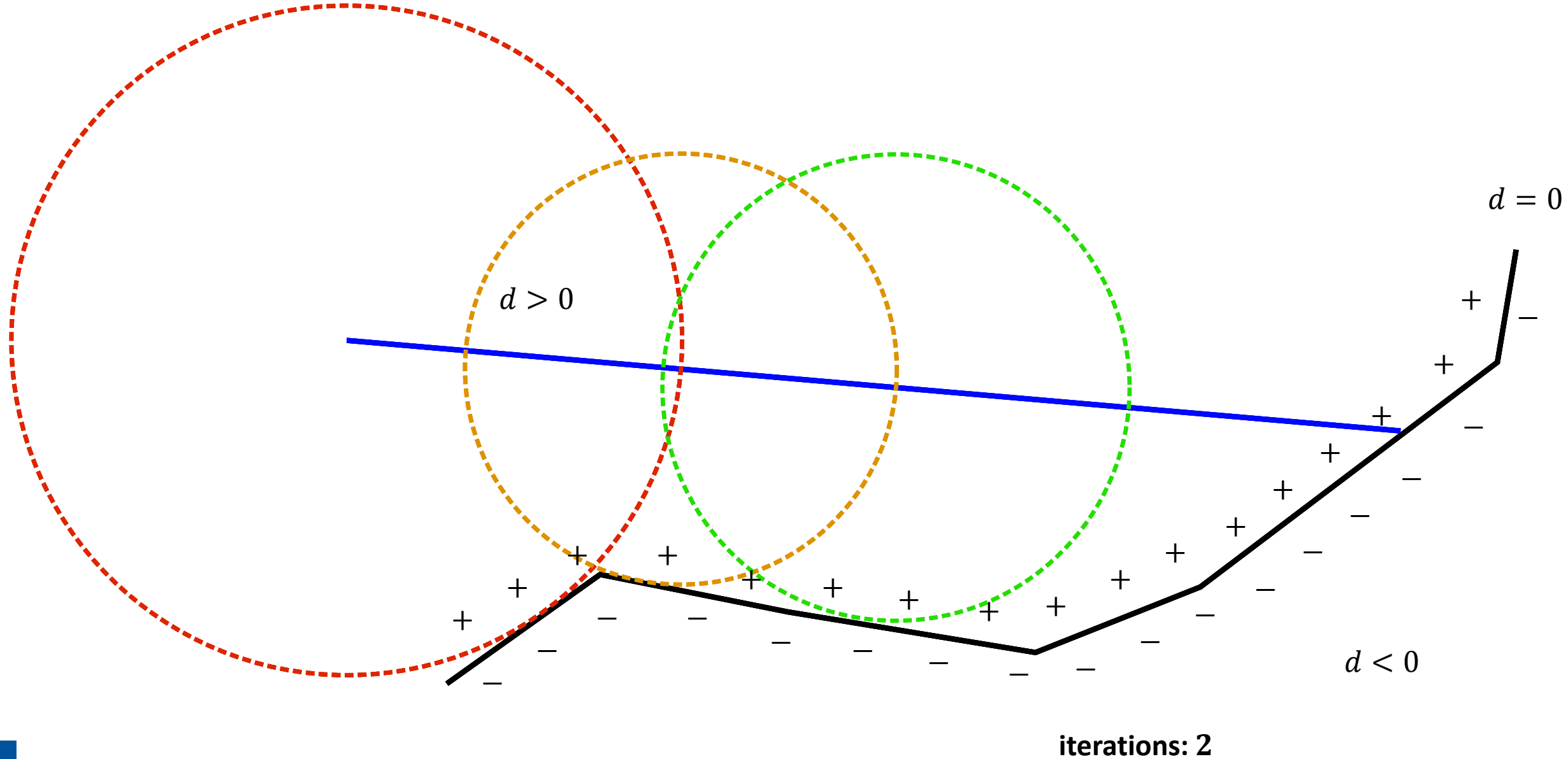
- Instead of evaluating our ray at each point in space, we can jump to the next plausible position for a surface !



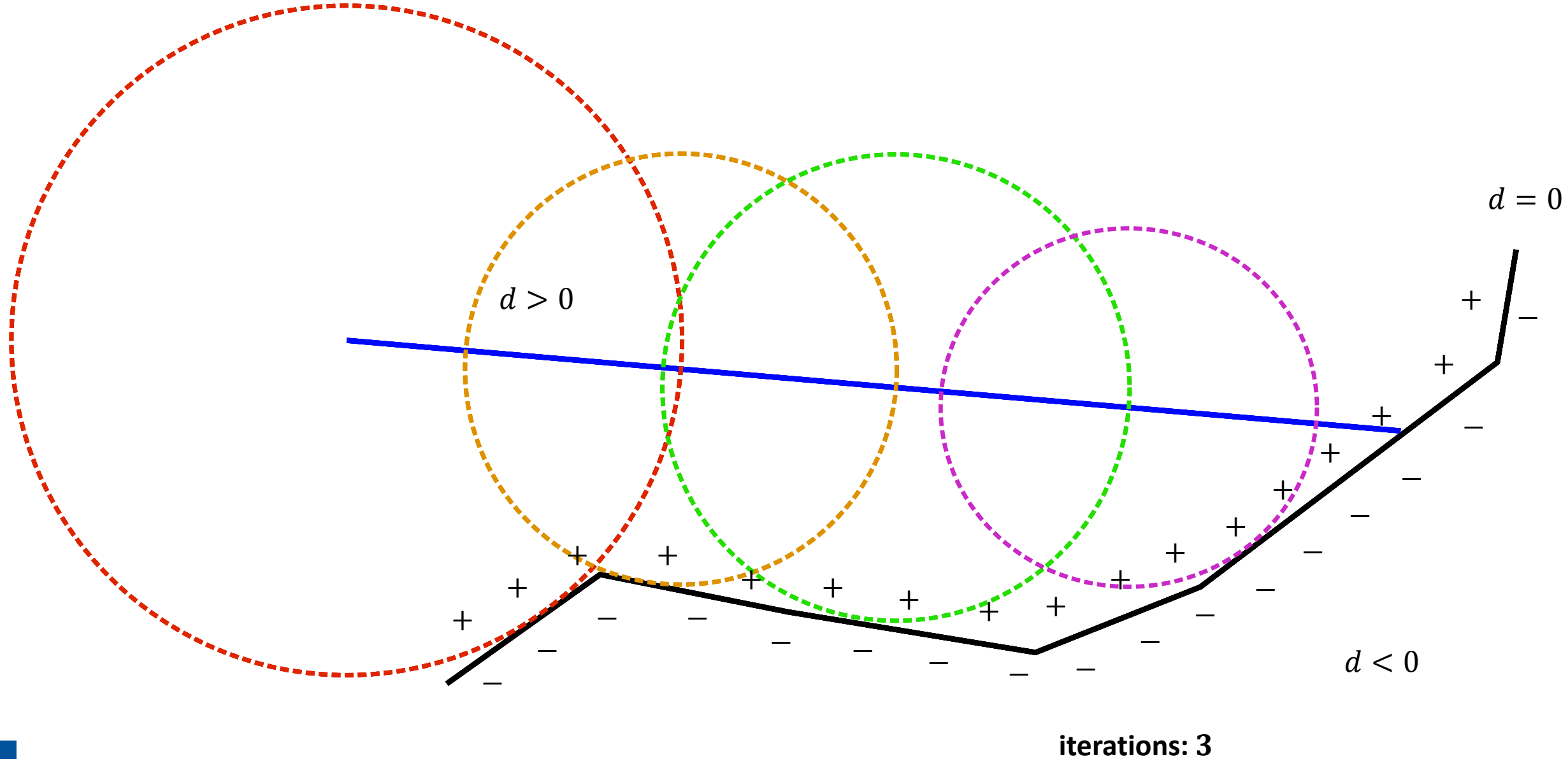
- Then again, we can iterate in the same way

iterations: 1

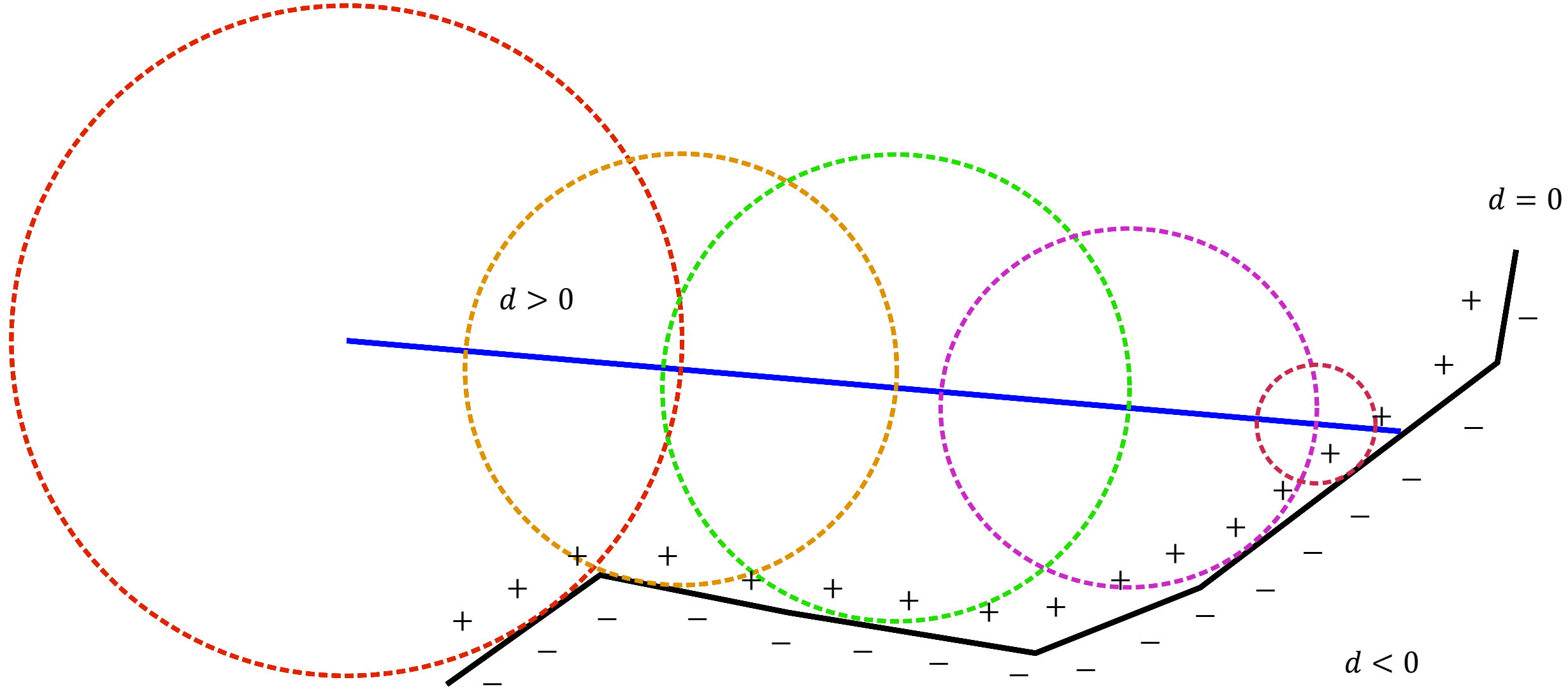
Raymarching



Raymarching

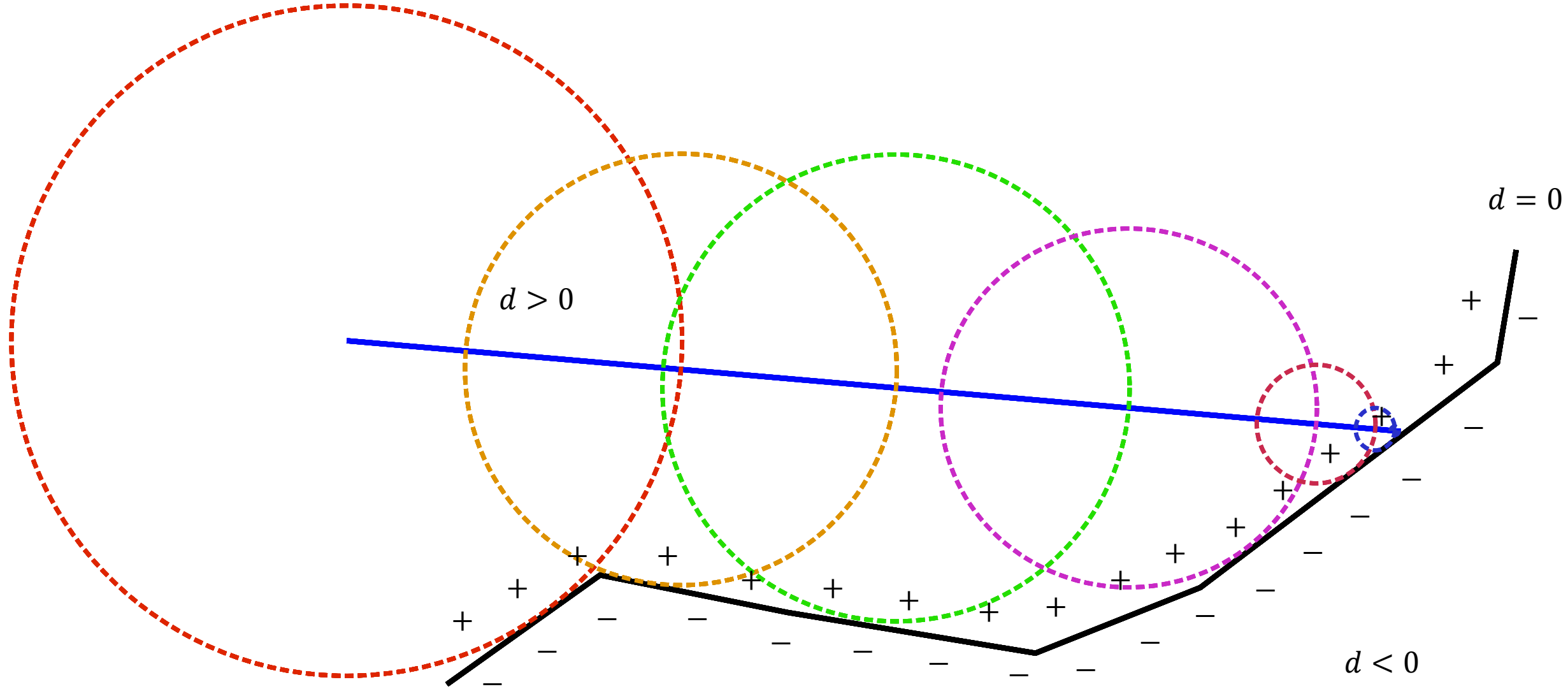


Raymarching



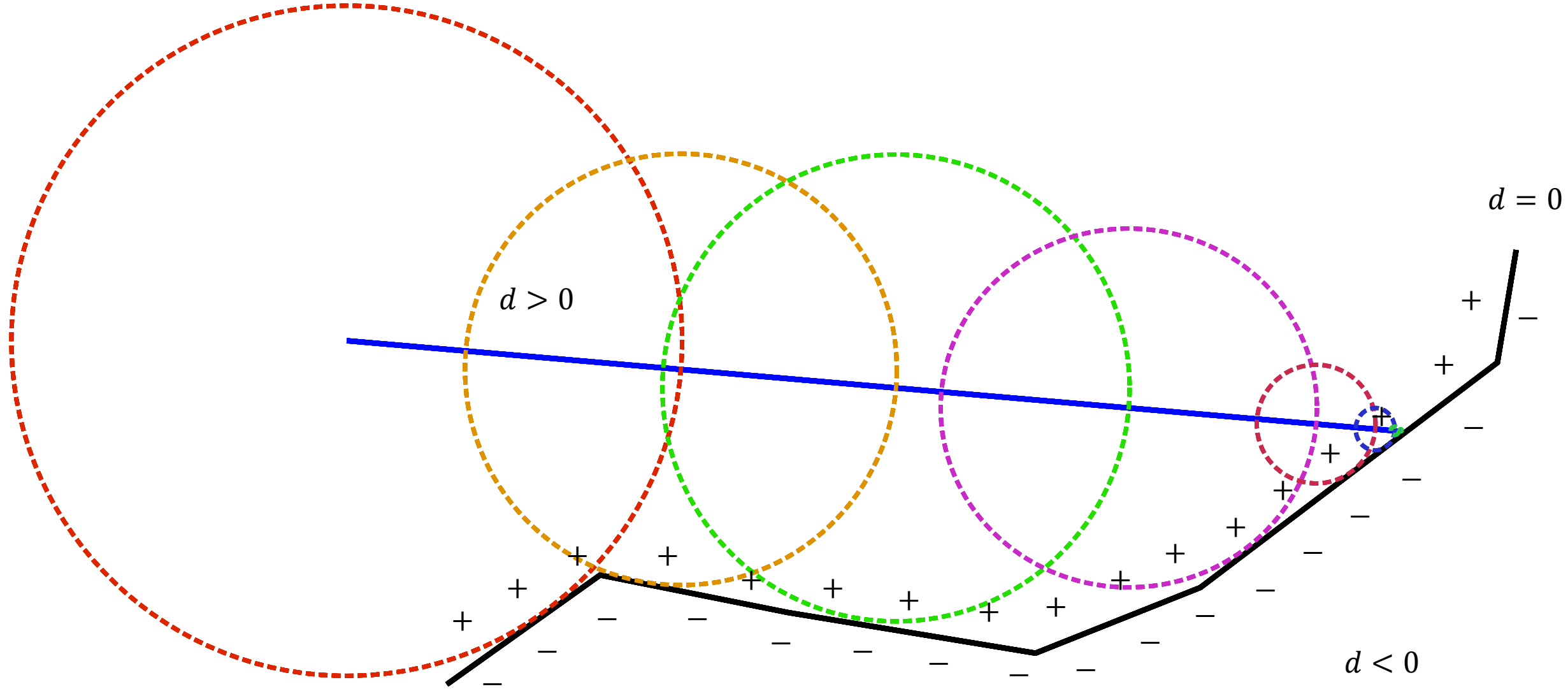
iterations: 4

Raymarching



iterations: 5

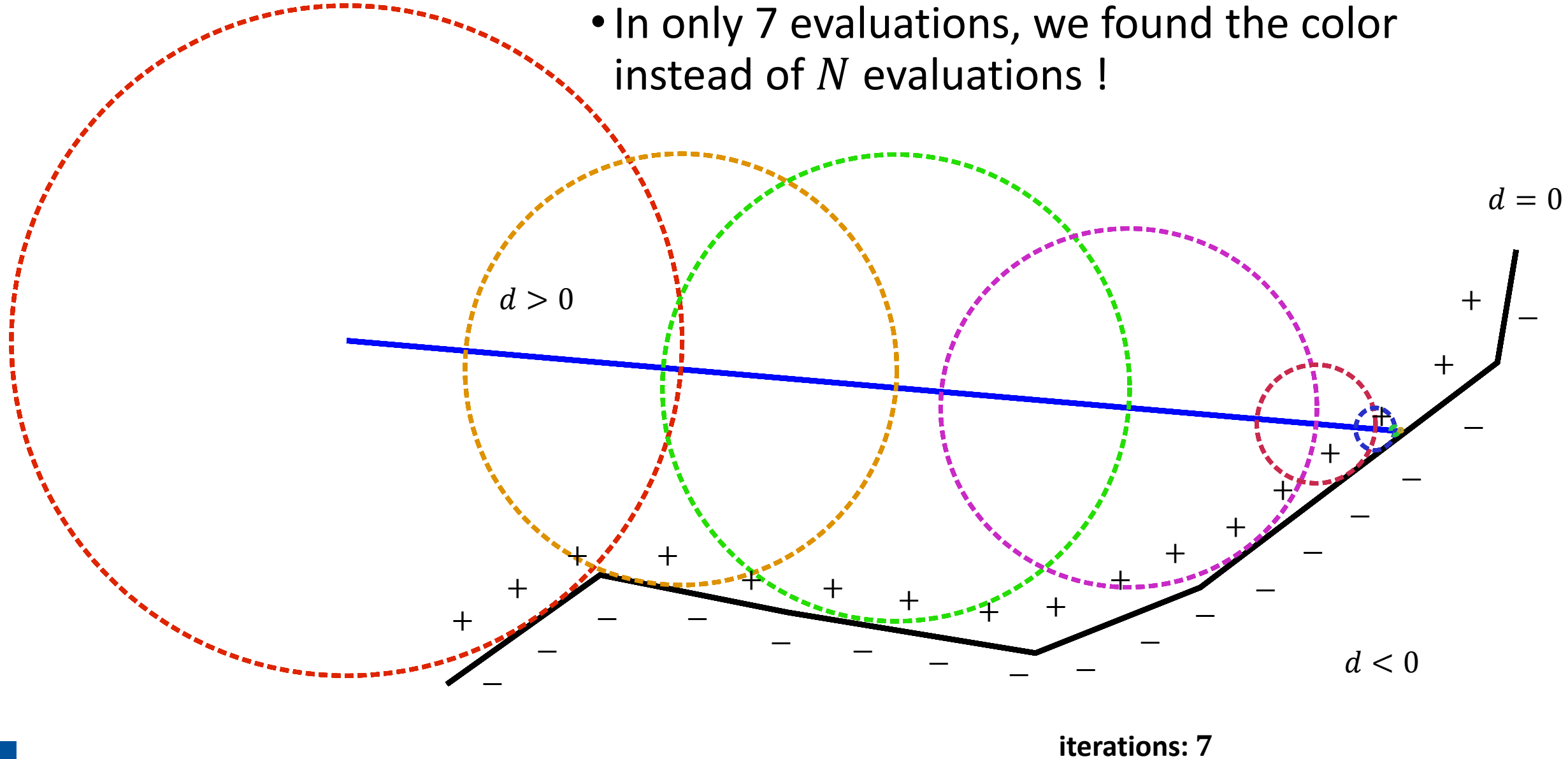
Raymarching



iterations: 6

Raymarching

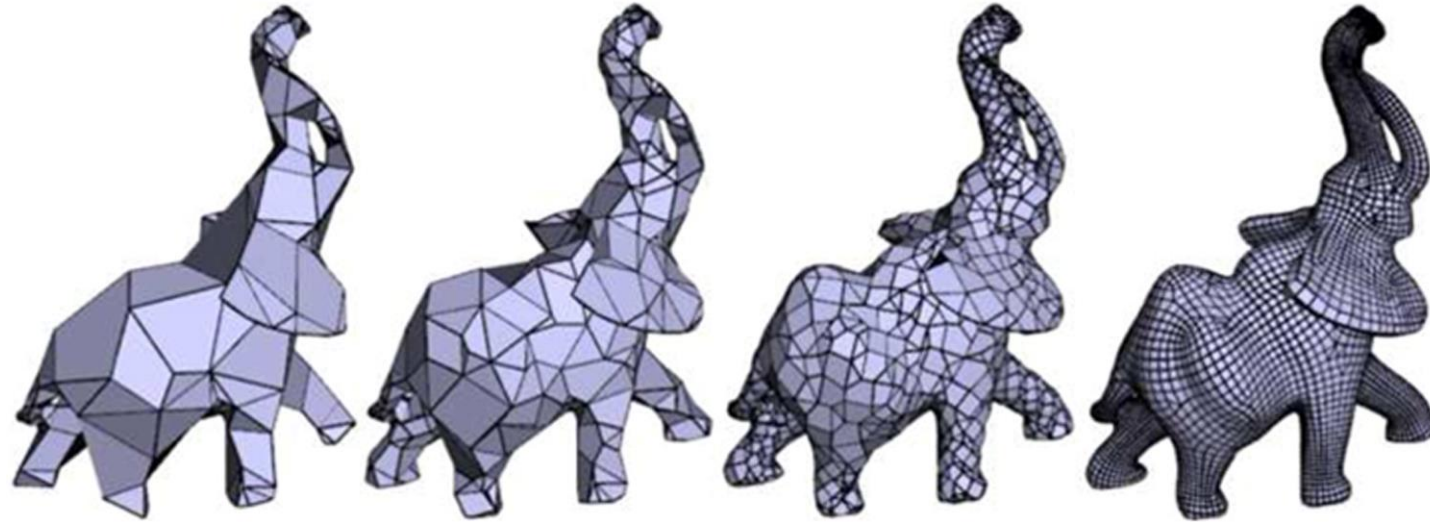
- In only 7 evaluations, we found the color instead of N evaluations !



iterations: 7

Curves in Computer Graphics

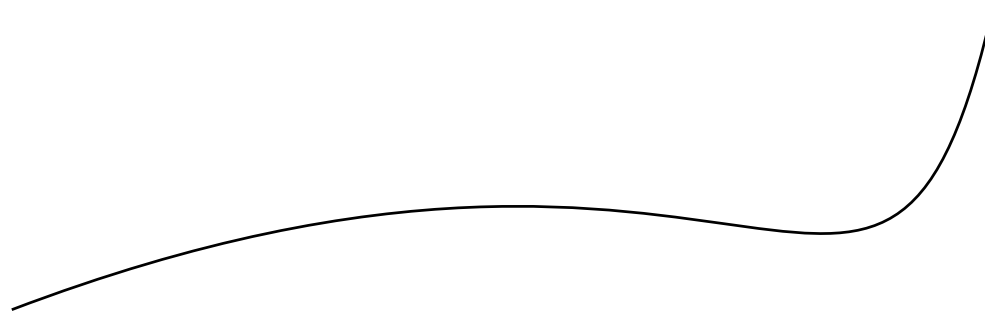
- Often polygonal meshes are not rendered nicely, we want smooth surfaces



- A nice way to achieve that is to use curves.
- In the following, we explore several kind of curves.

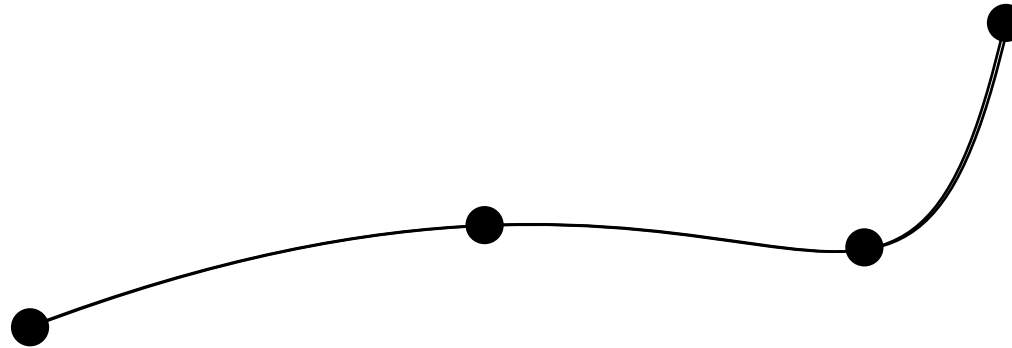
Drawing A Curve

- How can we draw this exact curve on a computer ?
- Without using an equation ! (user friendly)



Drawing A Curve

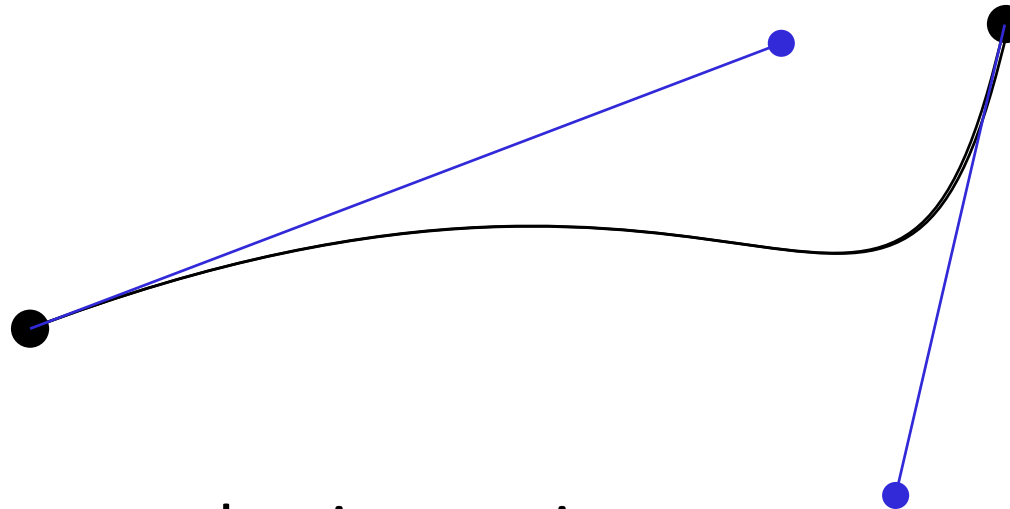
- How can we draw this exact curve on a computer ?
- Without using an equation ! (user friendly)



- We can define some control points on it
- But, which ones ?

Drawing A Curve

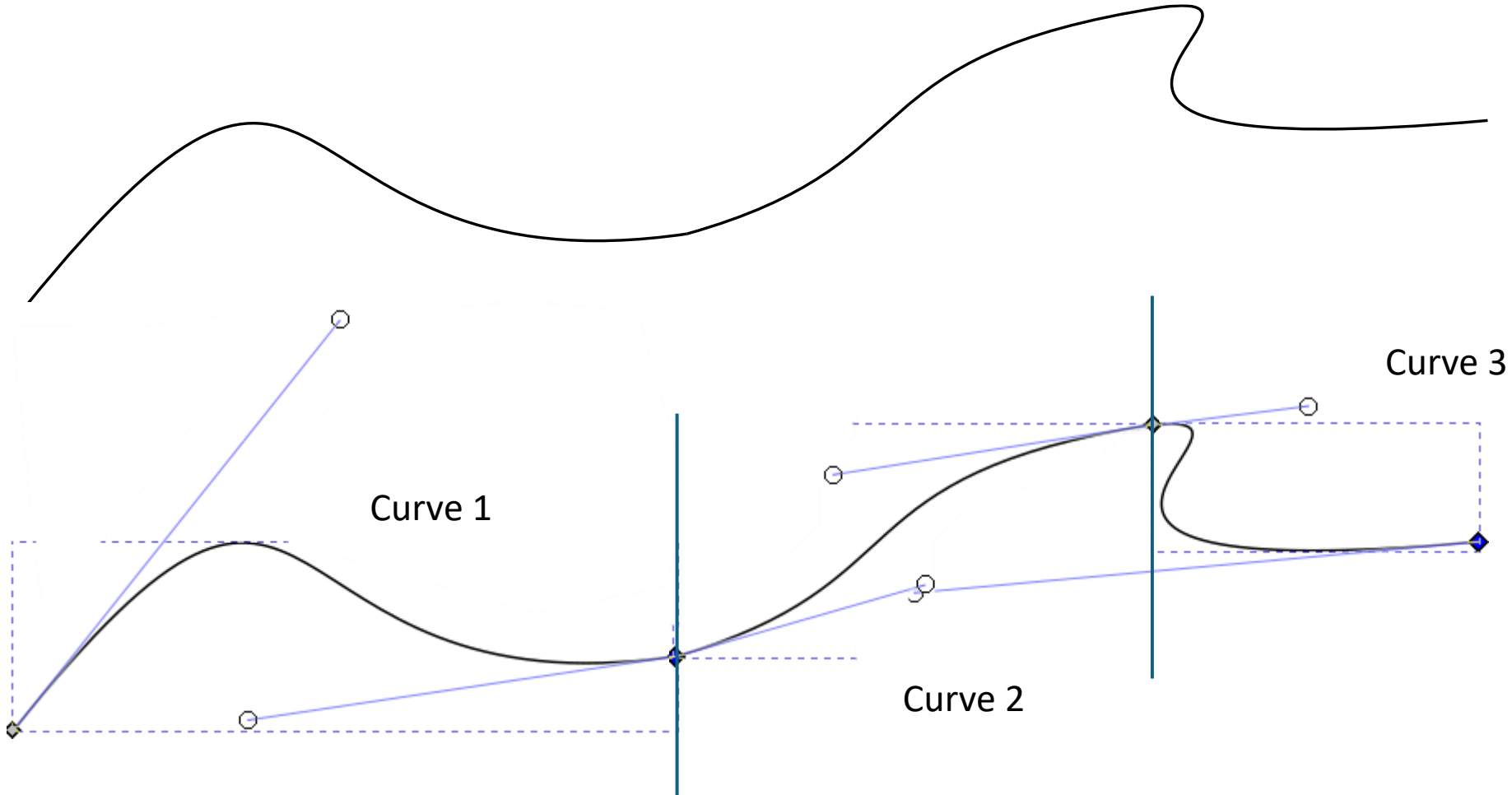
- How can we draw this exact curve on a computer ?
- Without using an equation ! (user friendly)



- We can define some control points on it
- But, which ones ?
- Nice ones are initial ($t = 0$) and final ($t = 1$) points and their tangents !
 - = Hermite curve

Drawing A Curve

- A long curving path is usually defined by having several smaller curves with end points and tangents corresponding.



Cubic curves

- Parametric representation: $P(t) = a + bt + ct^2 + dt^3$
- Where $a, b, c, d \in \mathbb{R}^3, t \in \mathbb{R}$
- But this is just a point, it would be nice to have a 3D representation

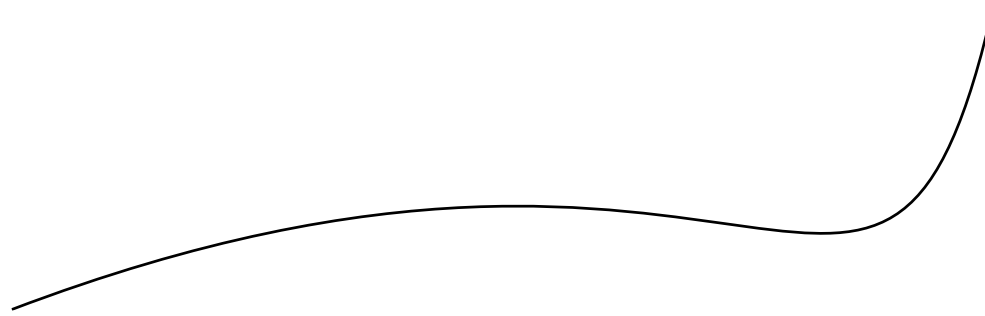
$$Q(t) = \begin{pmatrix} P_x(t) \\ P_y(t) \\ P_z(t) \end{pmatrix} = \begin{pmatrix} a_x + b_x t + c_x t^2 + d_x t^3 \\ a_y + b_y t + c_y t^2 + d_y t^3 \\ a_z + b_z t + c_z t^2 + d_z t^3 \end{pmatrix} = \begin{pmatrix} a_x & b_x & c_x & d_x \\ a_y & b_y & c_y & d_y \\ a_z & b_z & c_z & d_z \end{pmatrix} \begin{pmatrix} 1 \\ t \\ t^2 \\ t^3 \end{pmatrix}$$

$$Q(t) = CT(t)$$

Cubic curves

$$Q(t) = CT(t)$$

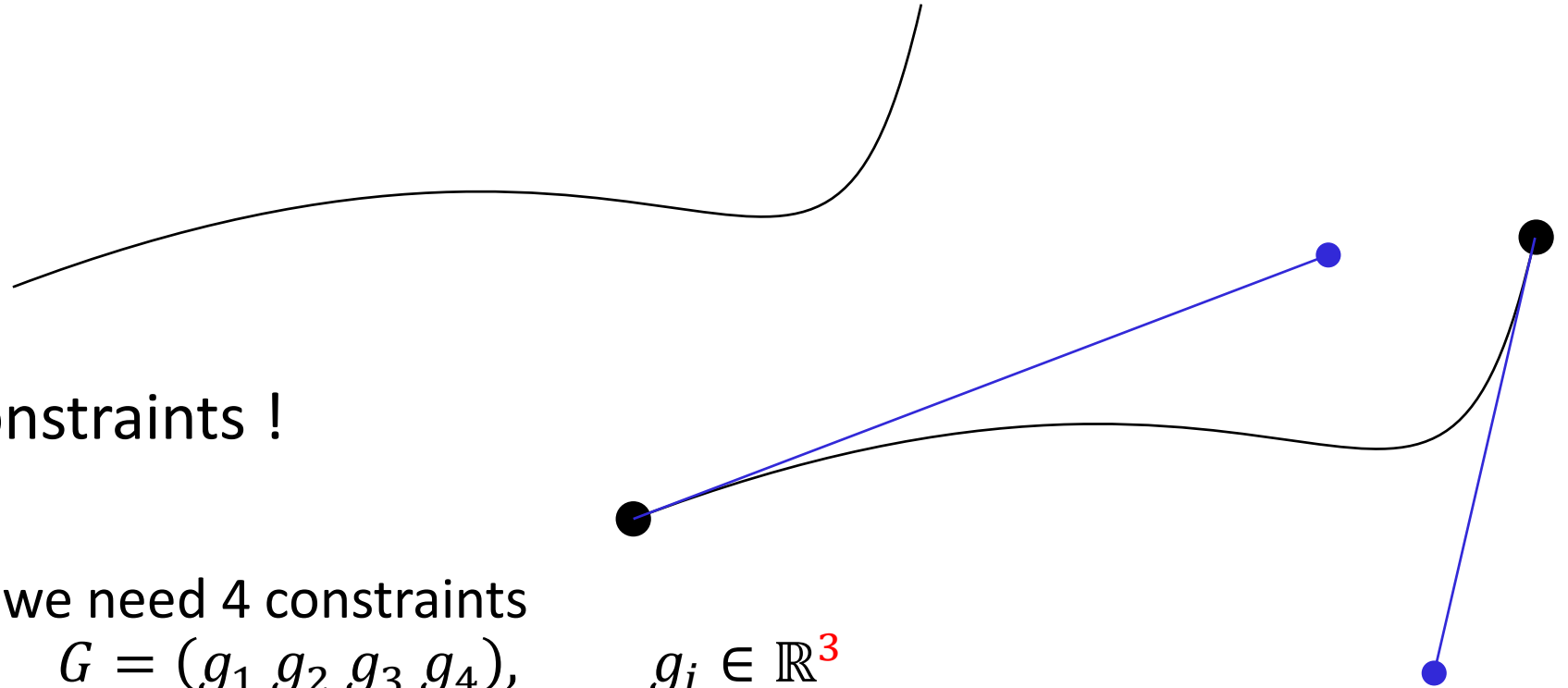
- We still don't have any ways to find the coefficients in C to make our curve



Cubic curves

$$Q(t) = CT(t)$$

- We still don't have any ways to find the coefficients in C to make our curve

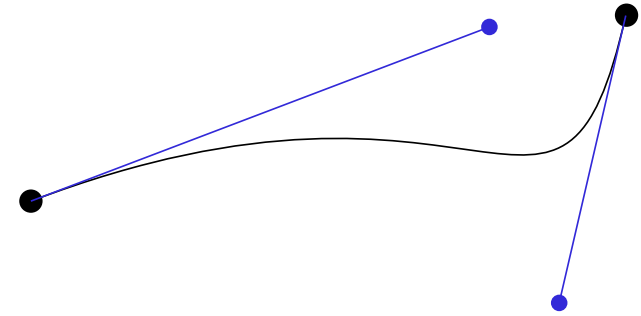


- But we can create constraints !
- Let's encode them:
 - We have $C \in \mathbb{R}^{3 \times 4}$ so we need 4 constraints
$$G = (g_1 \ g_2 \ g_3 \ g_4), \quad g_i \in \mathbb{R}^3$$
- In this case the points at $Q(0)$, $Q(1)$ and their tangents $Q'(0)$, $Q'(1)$
- However, how to encode the constraints in our equation ?

Cubic curves

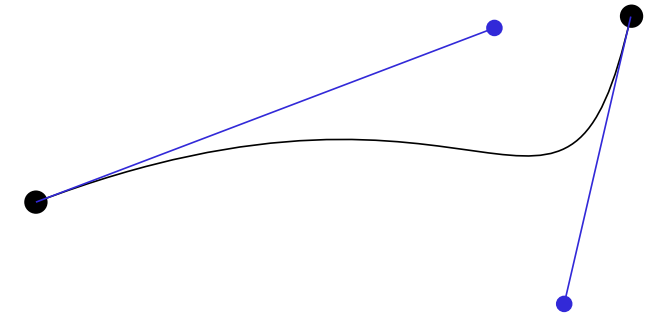
$$Q(t) = CT(t)$$
$$G = (g_1 \ g_2 \ g_3 \ g_4), g_i \in \mathbb{R}^3$$

- However, how to encode the constraints in our equation ?



Cubic curves

$$Q(t) = CT(t)$$
$$G = (g_1 \ g_2 \ g_3 \ g_4), g_i \in \mathbb{R}^3$$



- However, how to encode the constraints in our equation ?
- A trick is to decompose C in a new matrix which takes them into account !

$$C = \begin{pmatrix} a_x & b_x & c_x & d_x \\ a_y & b_y & c_y & d_y \\ a_z & b_z & c_z & d_z \end{pmatrix} = GM$$

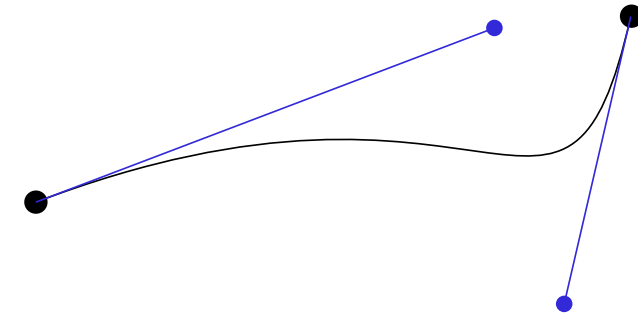
- We know that C is in $\mathbb{R}^{3 \times 4}$ and G in $\mathbb{R}^{3 \times 4}$
- So M must be in $\mathbb{R}^{4 \times 4}$, so that $\mathbb{R}^{3 \times 4} = \mathbb{R}^{3 \times 4} \times \mathbb{R}^{4 \times 4}$

$$\begin{pmatrix} a_x & b_x & c_x & d_x \\ a_y & b_y & c_y & d_y \\ a_z & b_z & c_z & d_z \end{pmatrix} = (g_1 \ g_2 \ g_3 \ g_4) \begin{pmatrix} a_1 & b_1 & c_1 & d_1 \\ a_2 & b_2 & c_2 & d_2 \\ a_3 & b_3 & c_3 & d_3 \\ a_4 & b_4 & c_4 & d_4 \end{pmatrix}$$

Cubic curves

$$Q(t) = CT(t)$$
$$G = (g_1 \ g_2 \ g_3 \ g_4), g_i \in \mathbb{R}^3$$

$$Q(t) = GMT(t)$$



- If we find M we can find C by multiplying it with G
$$C = GM$$
- The matrix G is called the **Geometry matrix**
- The matrix M is called the **Basis matrix**
 - The basis matrix will be fixed and depend only on the constraints we want to use !
- Now that we have our constraints G let's find M

Hermite Curves ($H(t) \equiv G(t)$ for hermite)

- Cubic curve defined by two end points P_1, P_2 and tangent directions T_1, T_2
- Let $G = (P_1 \ P_2 \ T_1 \ T_2) \in \mathbb{R}^{3 \times 4}$
- We solve the equations corresponding to the constraints:
- $H(0) = G_H M_H T(0) = P_1$
- When $t = 0$, we should obtain the point P_1

Hermite Curves ($H(t) \equiv G(t)$ for hermite)

- Cubic curve defined by two end points P_1, P_2 and tangent directions T_1, T_2
- Let $G = (P_1 \ P_2 \ T_1 \ T_2) \in \mathbb{R}^{3 \times 4}$
- We solve the equations corresponding to the constraints:
- $H(0) = G_H M_H T(0) = (P_1 \ P_2 \ T_1 \ T_2) M_H (1 \ 0 \ 0 \ 0)^T = P_1$

$$T(t) = (1 \ t \ t^2 \ t^3)^T$$

Hermite Curves ($H(t) \equiv G(t)$ for hermite)

- Cubic curve defined by two end points P_1, P_2 and tangent directions T_1, T_2
- Let $G = (P_1 \ P_2 \ T_1 \ T_2) \in \mathbb{R}^{3 \times 4}$
- We solve the equations corresponding to the constraints:
- $H(0) = G_H M_H T(0) = (P_1 \ P_2 \ T_1 \ T_2) M_H (1 \ 0 \ 0 \ 0)^T = P_1$
- $H(1) = G_H M_H T(1) = (P_1 \ P_2 \ T_1 \ T_2) M_H (1 \ 1 \ 1 \ 1)^T = P_2$

$$T(t) = (1 \ t \ t^2 \ t^3)^T$$

Hermite Curves ($H(t) \equiv G(t)$ for hermite)

- Cubic curve defined by two end points P_1, P_2 and tangent directions T_1, T_2
- Let $G = (P_1 \ P_2 \ T_1 \ T_2) \in \mathbb{R}^{3 \times 4}$
- We solve the equations corresponding to the constraints:

- $H(0) = G_H M_H T(0) = (P_1 \ P_2 \ T_1 \ T_2) M_H (1 \ 0 \ 0 \ 0)^T = P_1$

- $H(1) = G_H M_H T(1) = (P_1 \ P_2 \ T_1 \ T_2) M_H (1 \ 1 \ 1 \ 1)^T = P_2$

- $H'(\mathbf{0}) = G_H M_H T'(0) = (P_1 \ P_2 \ \mathbf{T_1} \ T_2) M_H (0 \ 1 \ 0 \ 0)^T = \mathbf{T_1}$

$$T(t) = (1 \ t \ t^2 \ t^3)^T$$

$$\frac{d}{dt} T(t) = (0 \ 1 \ 2t \ 3t^2)^T$$

Hermite Curves ($H(t) \equiv G(t)$ for hermite)

- Cubic curve defined by two end points P_1, P_2 and tangent directions T_1, T_2
- Let $G = (P_1 \ P_2 \ T_1 \ T_2) \in \mathbb{R}^{3 \times 4}$
- We solve the equations corresponding to the constraints:

$$\bullet H(0) = G_H M_H T(0) = (P_1 \ P_2 \ T_1 \ T_2) M_H (1 \ 0 \ 0 \ 0)^T = P_1$$

$$\bullet H(1) = G_H M_H T(1) = (P_1 \ P_2 \ T_1 \ T_2) M_H (1 \ 1 \ 1 \ 1)^T = P_2$$

$$\bullet H'(0) = G_H M_H T'(0) = (P_1 \ P_2 \ T_1 \ T_2) M_H (0 \ 1 \ 0 \ 0)^T = T_1$$

$$\bullet H'(\mathbf{1}) = G_H M_H T'(1) = (P_1 \ P_2 \ T_1 \ T_2) M_H (0 \ 1 \ 2 \ 3)^T = T_2$$

$$T(t) = (1 \ t \ t^2 \ t^3)^T$$

$$\frac{d}{dt} T(t) = (0 \ 1 \ 2t \ 3t^2)^T$$

$$\bullet \text{ As one equation: } (P_1 \ P_2 \ T_1 \ T_2) M_H \begin{matrix} G_H \\ \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 1 & 0 & 3 \end{pmatrix} \end{matrix} = (P_1 \ P_2 \ T_1 \ T_2)$$

$(T(0) \ T(1) \ T'(0) \ T'(1))$

Hermite Curves ($H(t) \equiv G(t)$ for hermite)

- Cubic curve defined by two end points P_1, P_2 and tangent directions T_1, T_2

- Let $G = (P_1 \ P_2 \ T_1 \ T_2) \in \mathbb{R}^{3 \times 4}$

• As one equation: $(P_1 \ P_2 \ T_1 \ T_2) M_H = (P_1 \ P_2 \ T_1 \ T_2)$

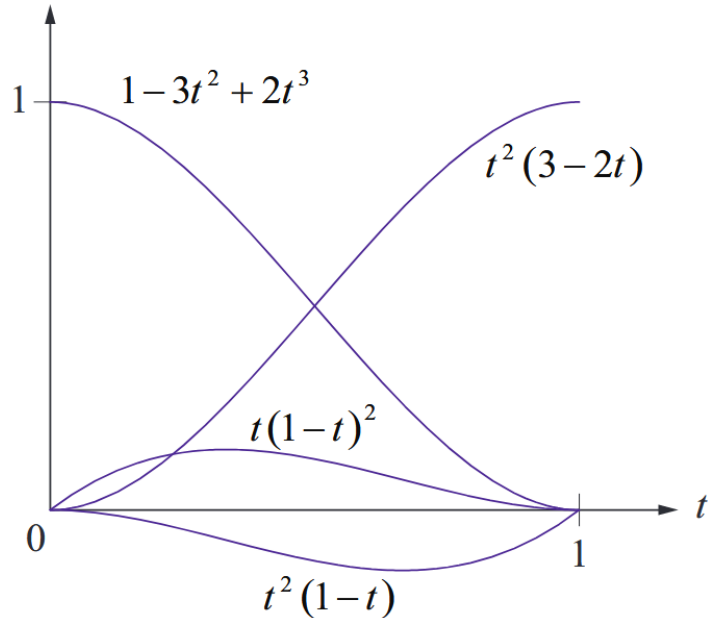
$$G_H \begin{pmatrix} T(0) & T(1) & T'(0) & T'(1) \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 1 & 0 & 3 \end{pmatrix} = (P_1 \ P_2 \ T_1 \ T_2)$$

• Therefore, $M_H = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 1 & 0 & 3 \end{pmatrix}^{-1} = \begin{pmatrix} 1 & 0 & -3 & 2 \\ 0 & 0 & 3 & -2 \\ 0 & 1 & -2 & 1 \\ 0 & 0 & -1 & 1 \end{pmatrix}$

- This is the basis matrix for Hermite curves. By having any 2 points and their tangents, we can find the coefficients C , and therefore, we obtain our curve.

What is exactly the basis matrix ?

- The basis matrix M describes blending functions of the constraints.
- With our Hermite curve $H(t)$, if we develop the matrix, we find:
- $H(t) = (1 - 3t^2 + 2t^3)P_1 + t^2(3 - 2t)P_2 + t(t - 1)^2T_1 + t^2(t - 1)T_2$
- In other words, $H(t) = f_1(t)P_1 + f_2(t)P_2 + f_3(t)T_1 + f_4(t)T_2$
- Where the functions $f_i(t)$ are blending functions of the constraints.



Convince yourself by plugging different values of t

Note that basis functions sum to 1 !
They weight the constraints

Bézier Curves

- Bézier curves are another variety of curves, defined by only control points P_i
- Bézier curves can be defined for any polynomial of degree n , by providing $n + 1$ control points.
- $B(t) = \sum_{k=0}^n B_{n,k}(t)P_k$
- Where the blending functions are the Bernstein polynomials:
 - $B_{n,k} = \binom{n}{k} t^k (1 - t)^{n-k}$

Cubic Bézier Curves

- Let's focus on cubic Bézier curves

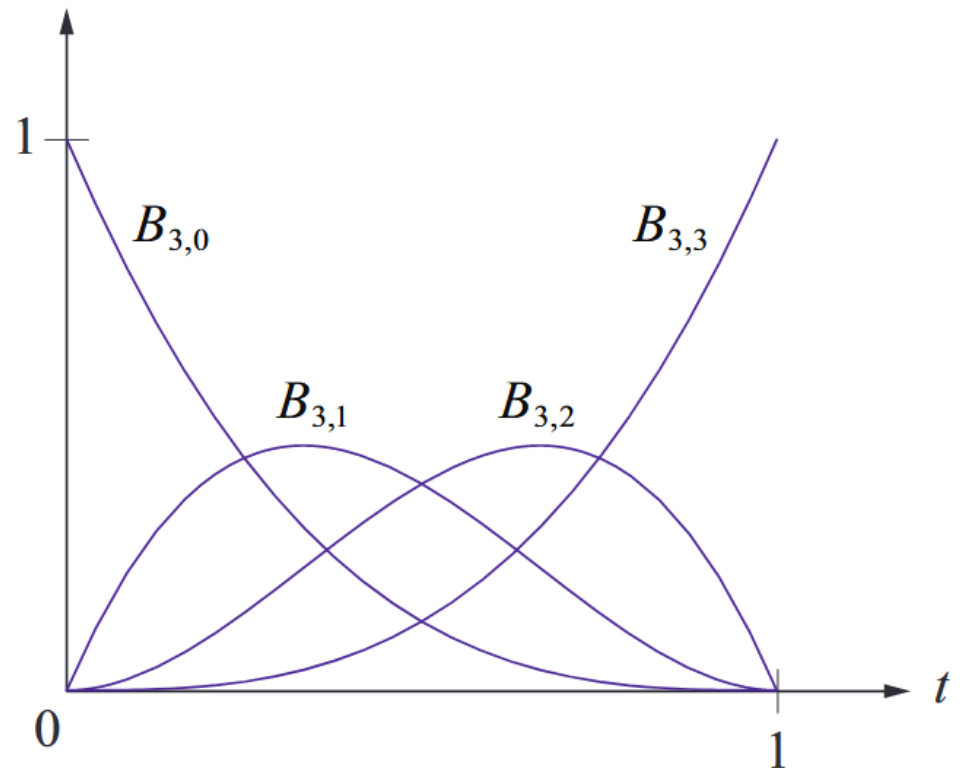
- $B(t) = \sum_{k=0}^3 B_{3,k}(t)P_k = \sum_{k=0}^3 \binom{3}{k} t^k (1-t)^{3-k} P_k = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t) P_2 + t^3 P_3$

- The geometry matrix is $G = (P_1 \ P_2 \ P_3 \ P_4)$

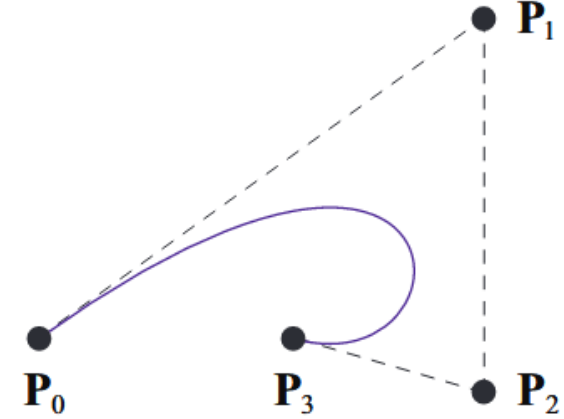
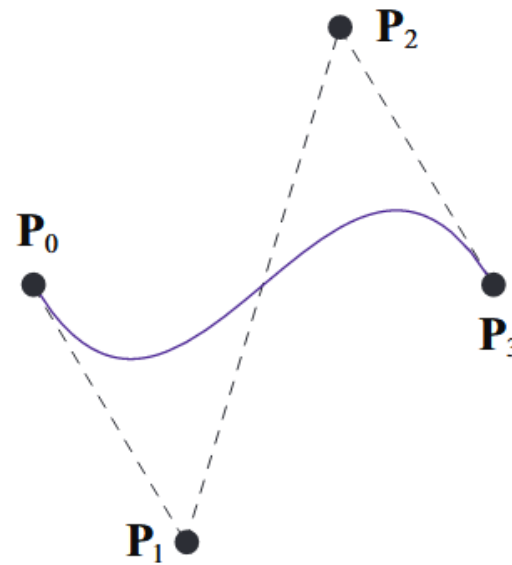
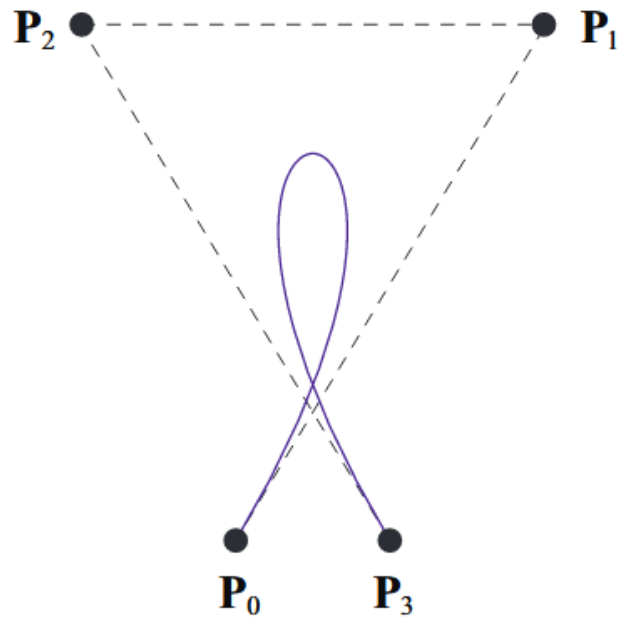
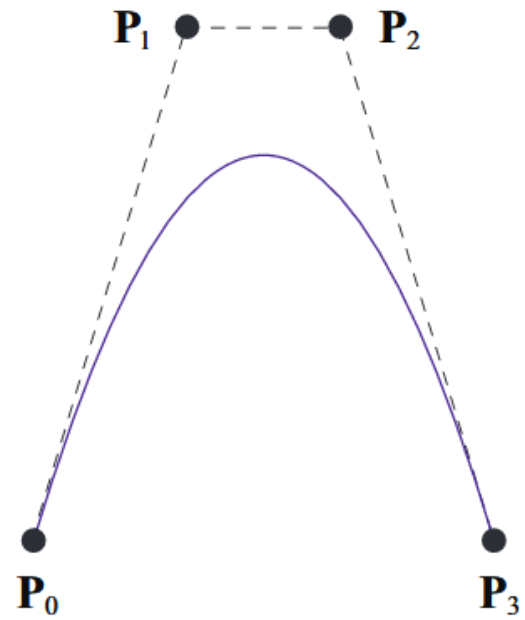
- Similarly to previously, we can derive the basis matrix M_B

- $M_B = \begin{pmatrix} 1 & -3 & 3 & -1 \\ 0 & 3 & -6 & 3 \\ 0 & 0 & -3 & 3 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ Derive yourself!

- So: $B(t) = (P_1 \ P_2 \ P_3 \ P_4) M_B (1 \ t \ t^2 \ t^3)^T$



Cubic Bézier Curves – Examples

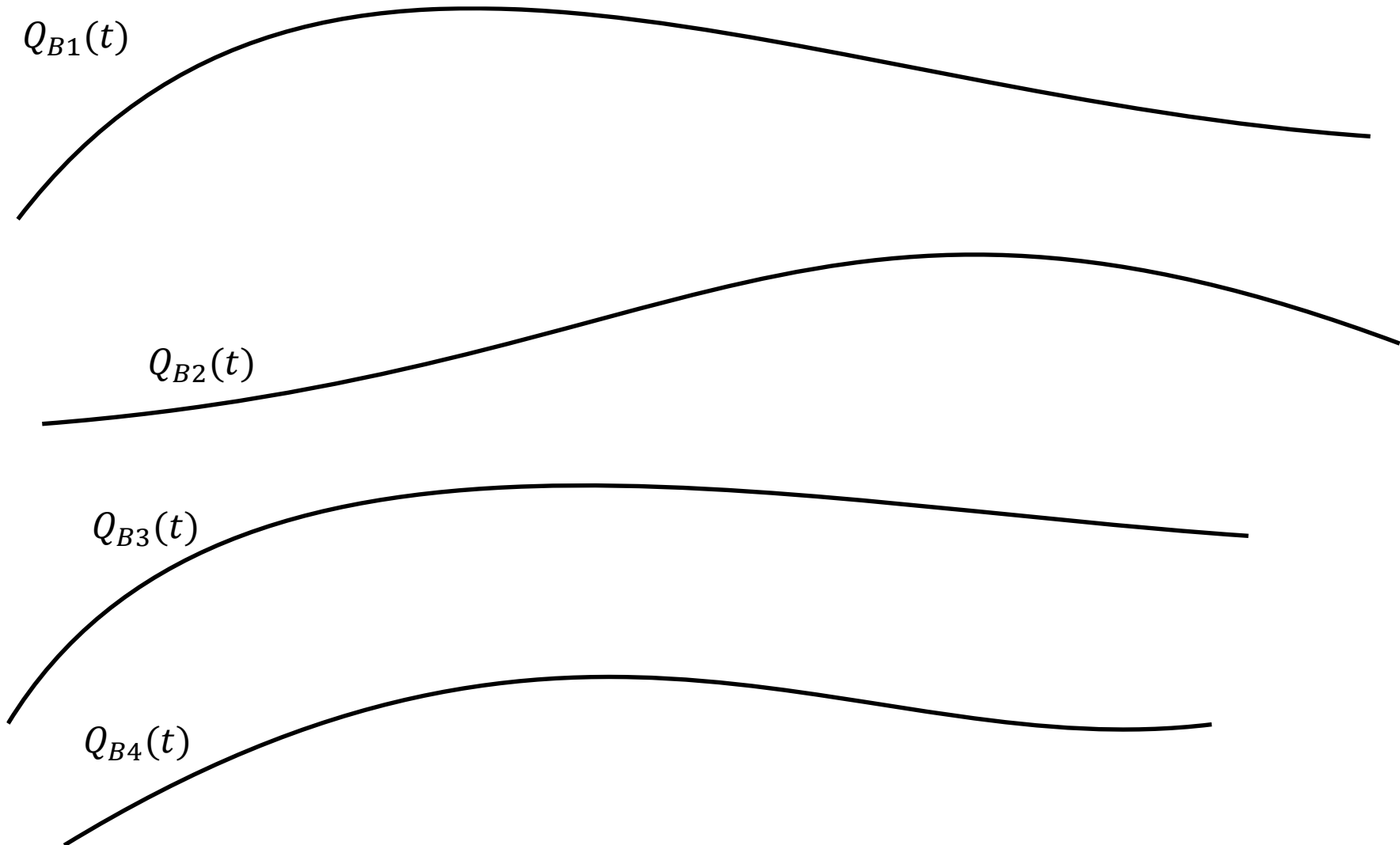


Bicubic Surfaces

- We want to extend our knowledge to 3D (smooth) surfaces.
- Let's construct this using Bézier curves !

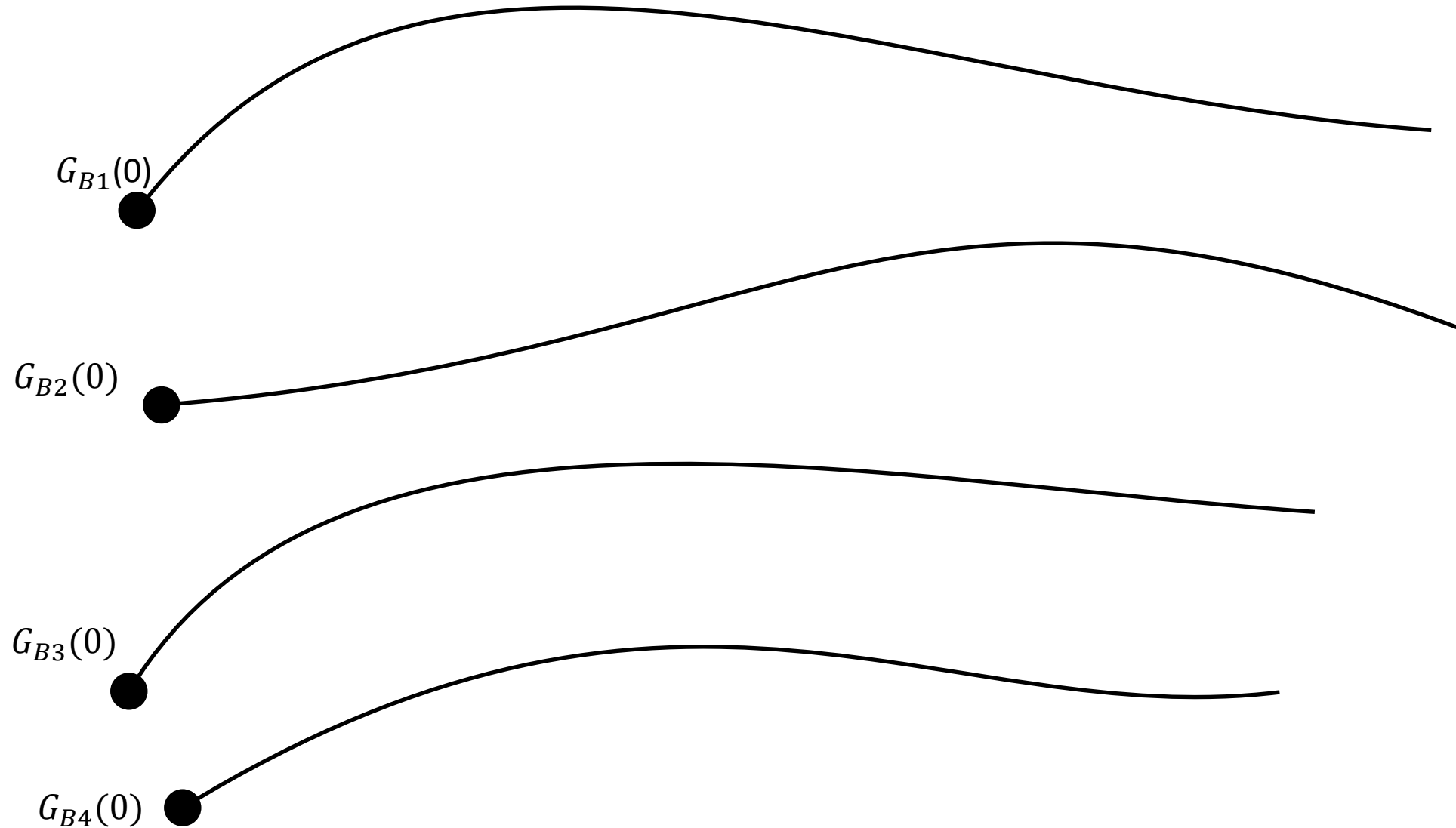
Bicubic Surfaces

- Imagine we have 4 Bézier curves, parametrized by the same t



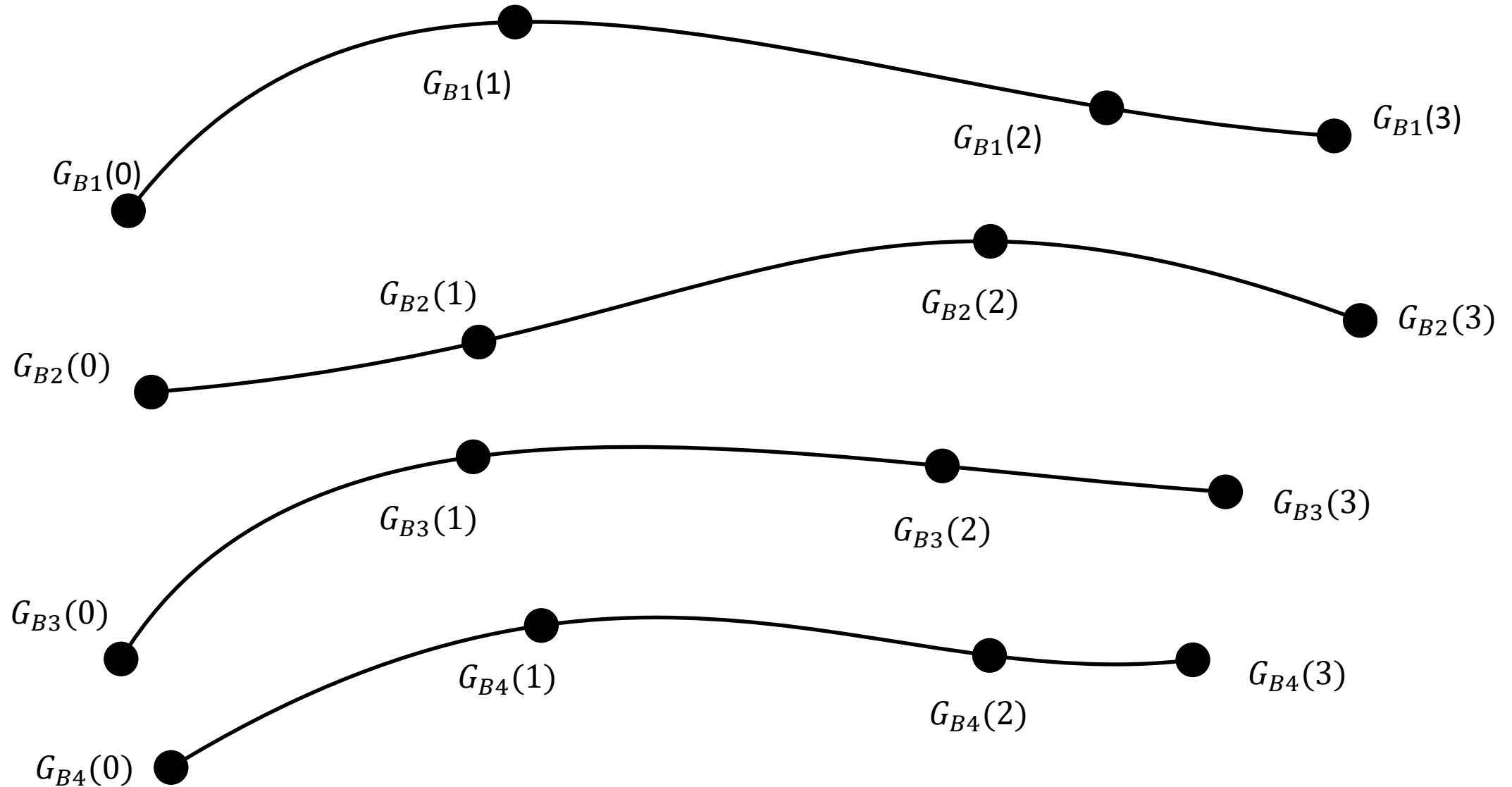
Bicubic Surfaces

- Each have a control point at $t = 0$



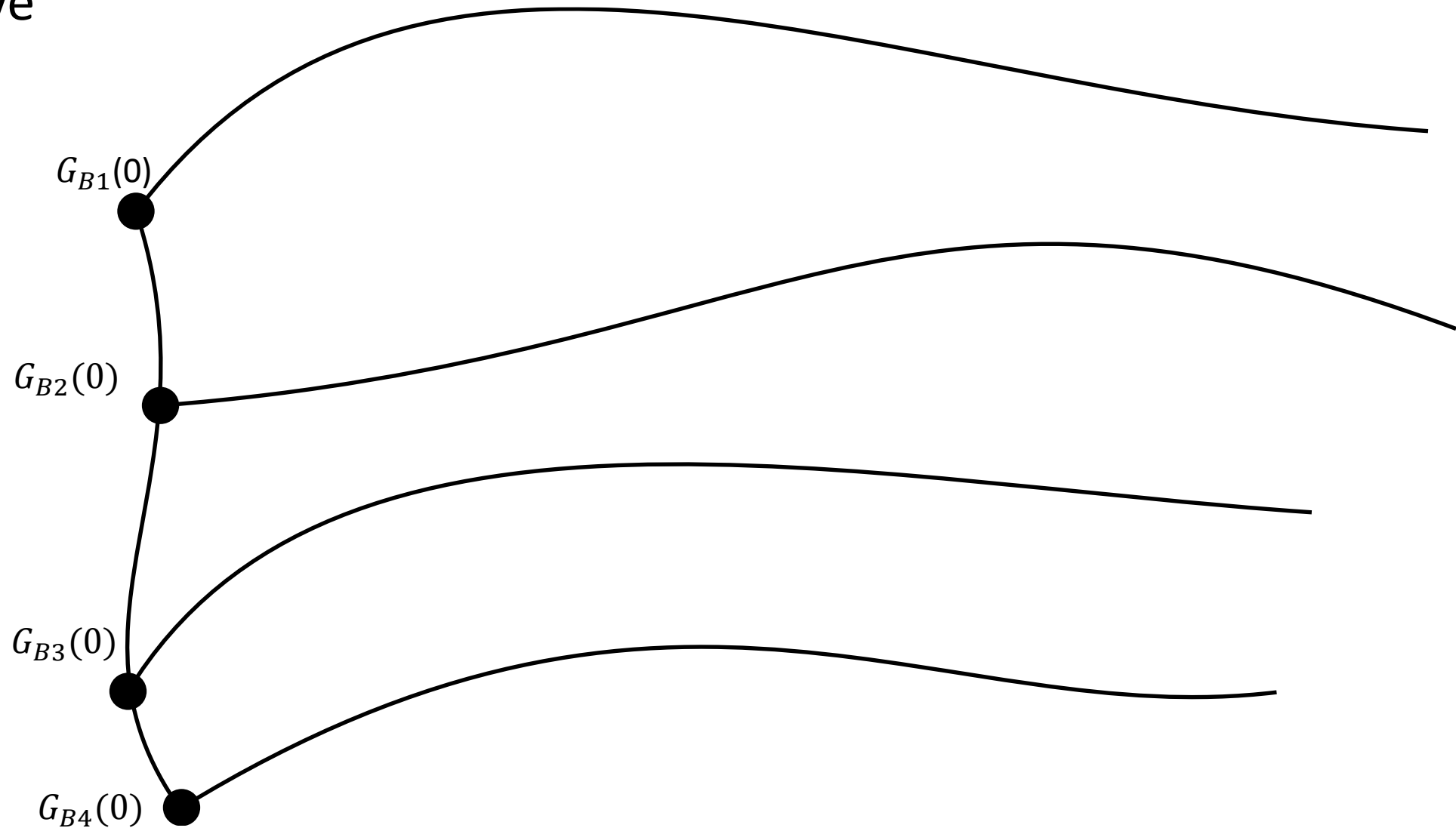
Bicubic Surfaces

- And 3 others each along the curves



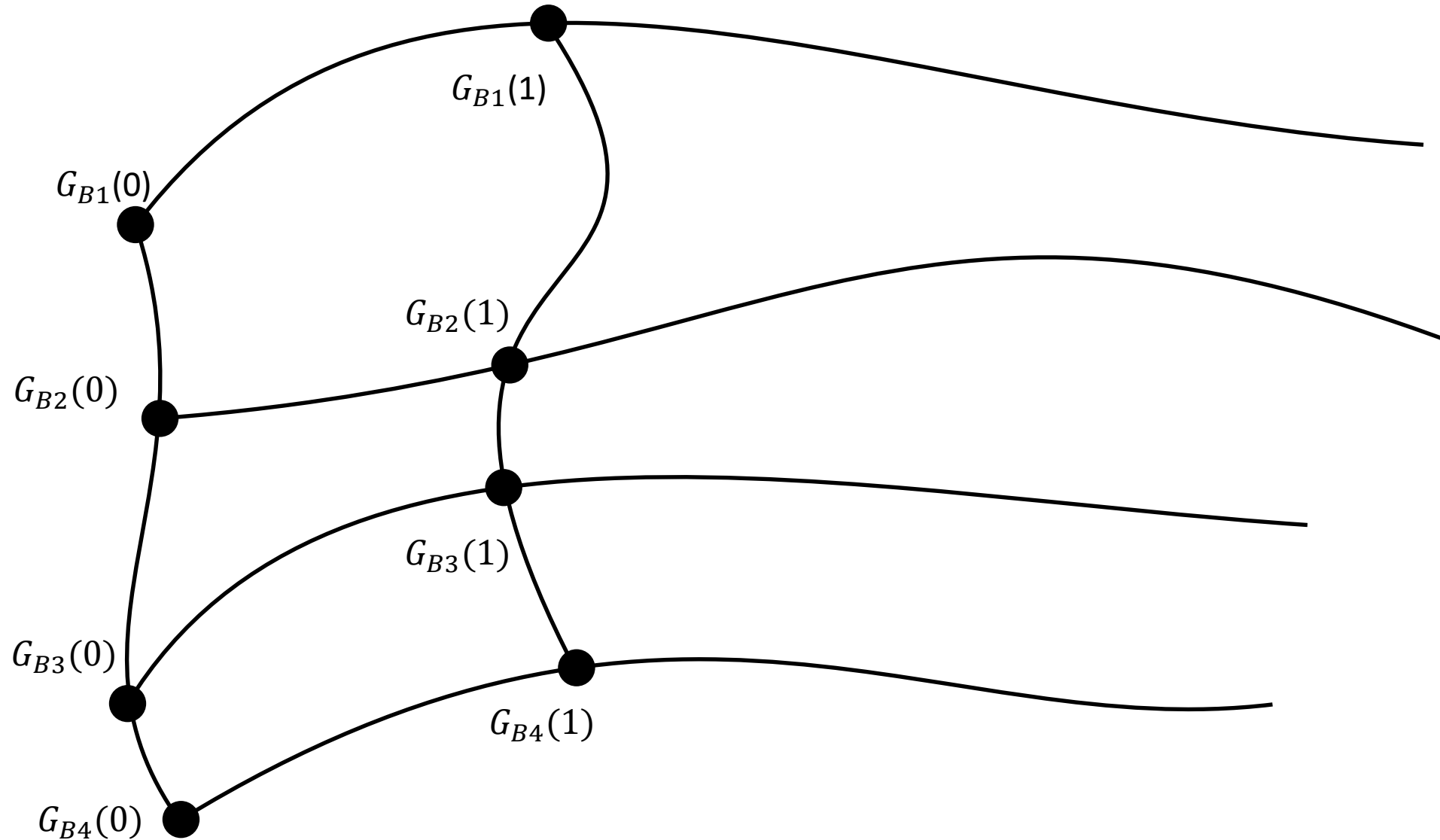
Bicubic Surfaces

- Let's use the each Bézier curve first point as a control point for a new Bézier curve



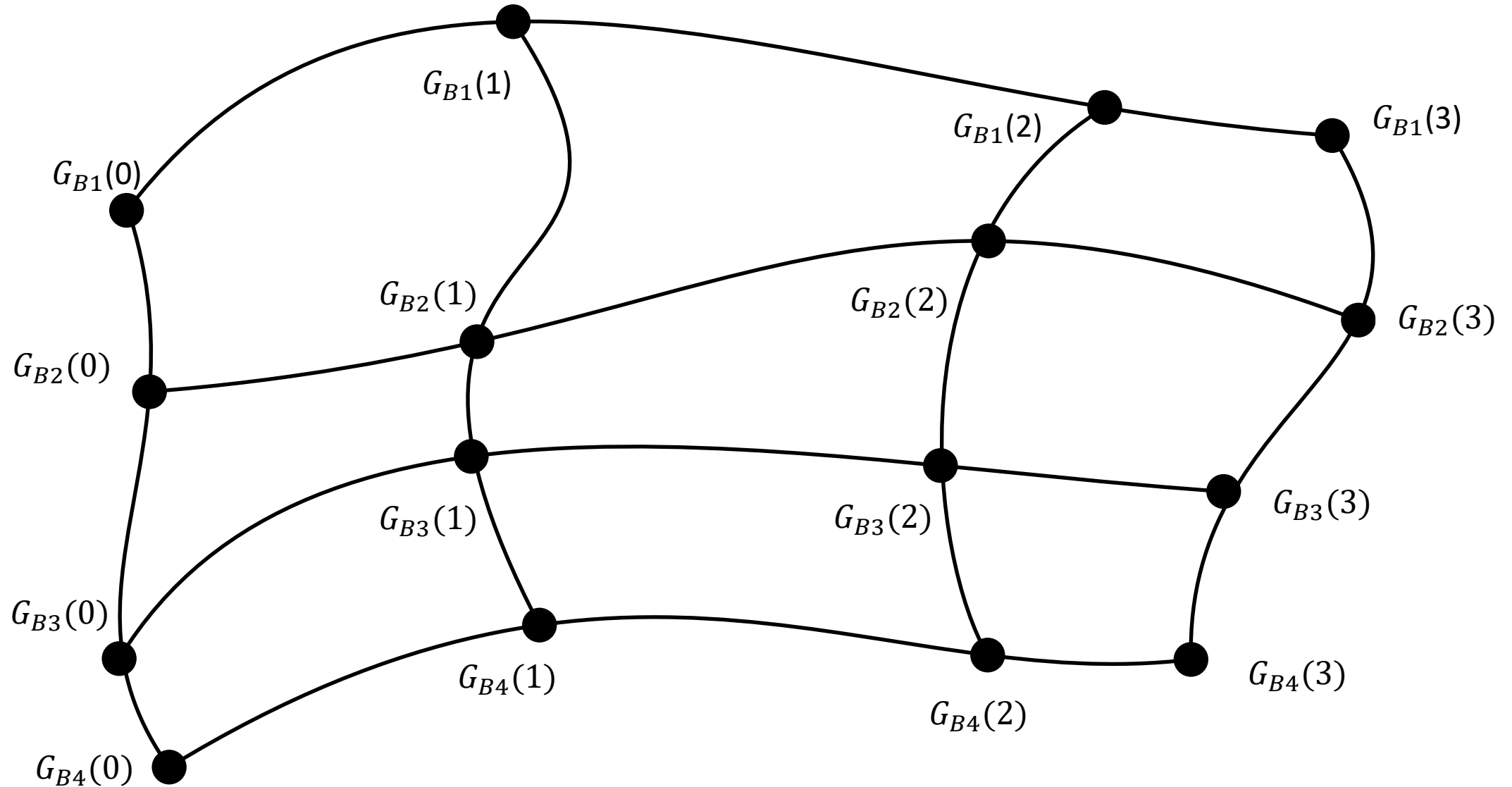
Bicubic Surfaces

- We can do the same for the second control points



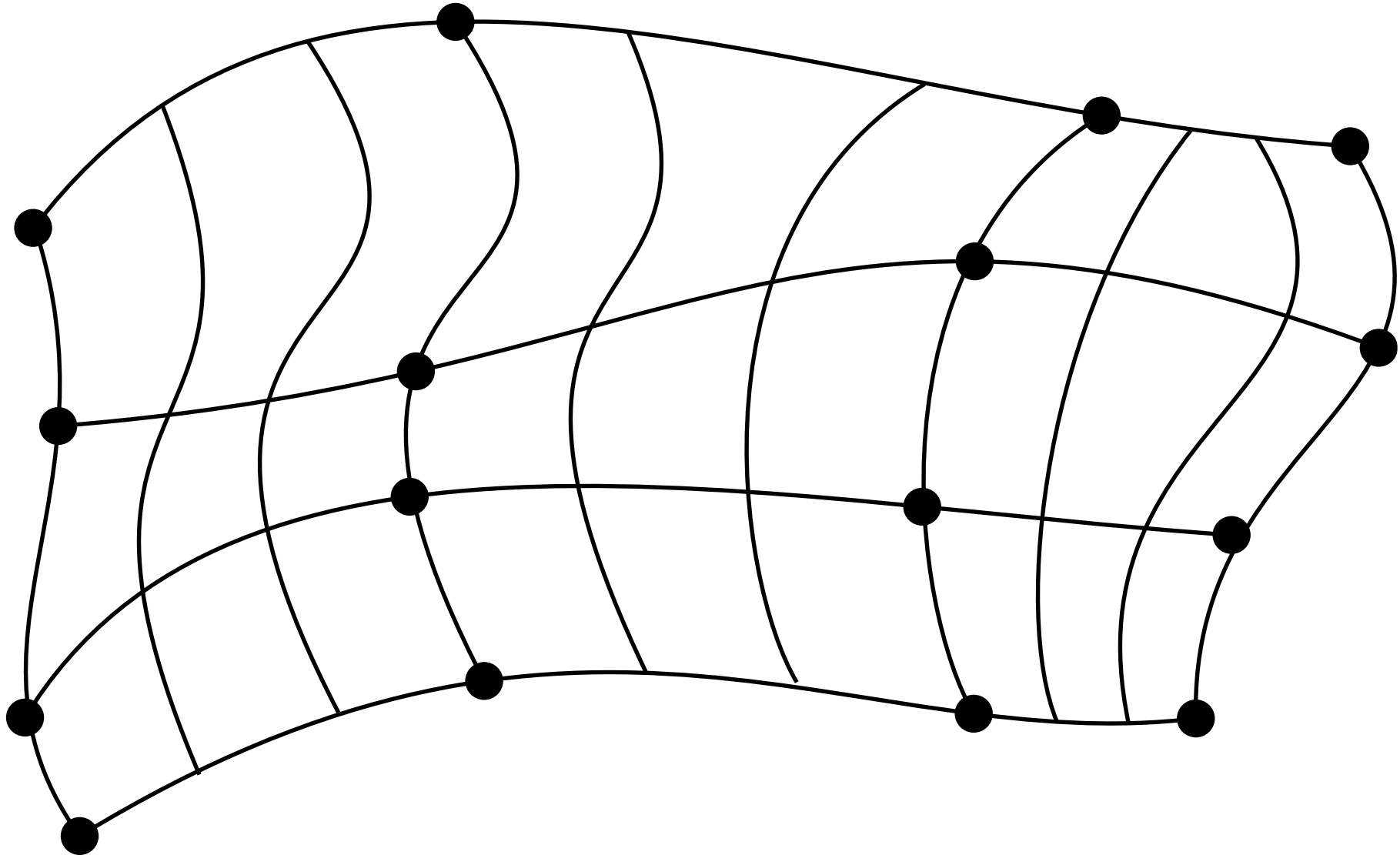
Bicubic Surfaces

- And for all control points..



Bicubic Surfaces

- Actually, we can do the same for any t , creating a surface



Bicubic Surfaces

- We observe that we need only $4 \times 4 = 16$ control points to make a surface !
 - 4 points/Bézier Curve, and 4 Bézier curves.
- The surface $Q_{BS}(s, t)$ (called a patch) depends on 2 parameters s and t
 - One parameter t for the 4 initial curves.
 - One parameter s for the constructed curves.
- The general parametric representation of a surface patch:
 - $Q_{ij}^{BS}(s, t) = \sum_{k=0}^3 \sum_{l=0}^3 B_k(s) B_l(t) G_{i+k, j+l}$
 - $i, j \in \{x, y, z\}$
 - Or $Q_{BS}(s, t) = S(t) M_B G_B T(t)$ where $S(t) = [1 \ s \ s^2 \ s^3]$, $T(t) = [1 \ t \ t^2 \ t^3]$
 - $G_B \in \mathbb{R}^{4 \times 3 \times 4}$ represent our 16 constraint points
 - M_B stays the same as for one Bézier curve, several stacked together to match the dimensions !

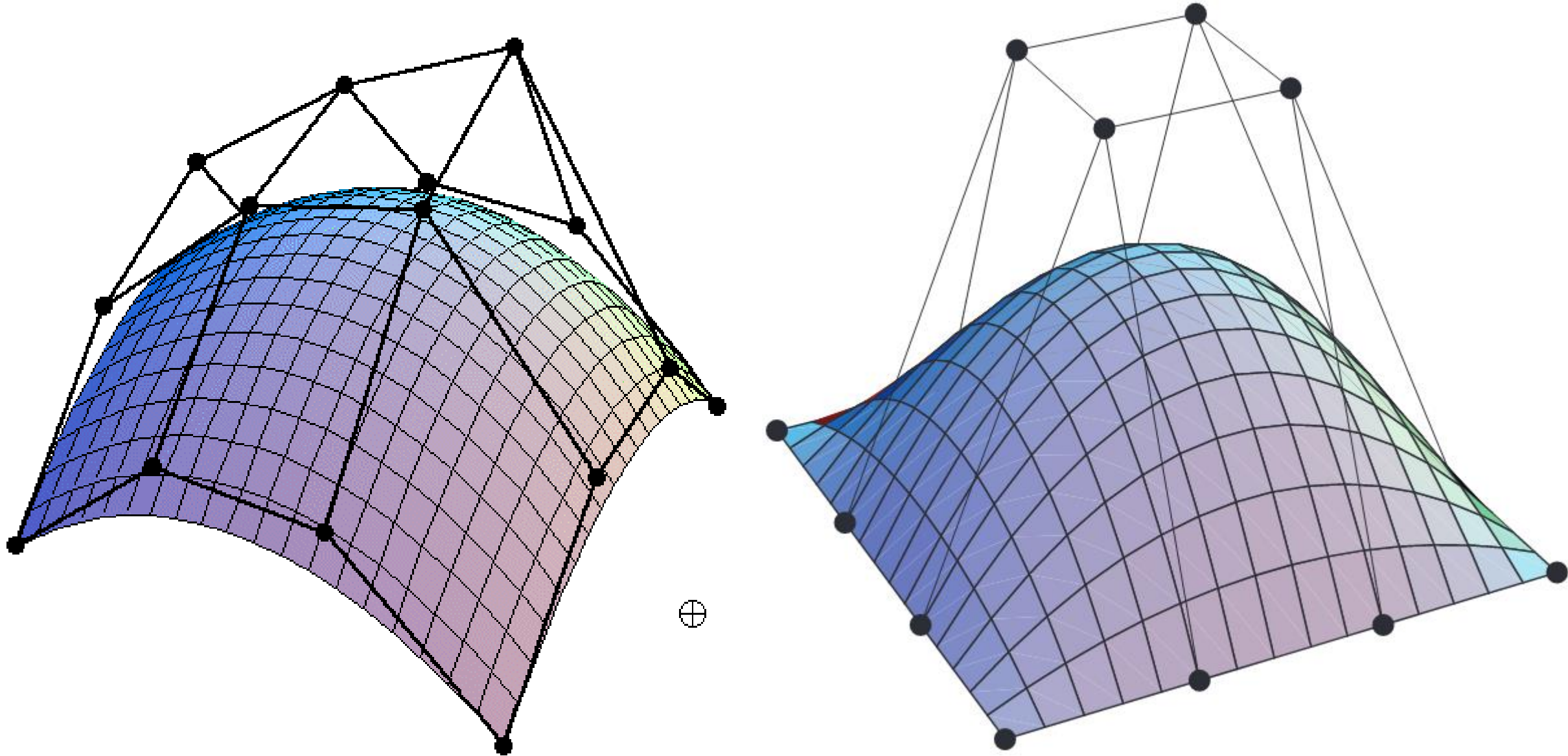
Bicubic Surfaces

- If we develop the formula we get

$$\begin{aligned}
 Q_{BS}(s, t) = & \underbrace{(-s^3 + 3s^2 - 3s + 1)}_{\text{orange}} \left(\underbrace{(-t^3 + 3t^2 - 3t + 1)}_{\text{blue}} \underbrace{x_{1,1}}_{\text{green}} + \underbrace{(3t^3 - 6t^2 + 3t)}_{\text{blue}} \underbrace{x_{1,2}}_{\text{green}} + \underbrace{(-3t^3 + 3t^2)}_{\text{blue}} \underbrace{x_{1,3}}_{\text{green}} + \underbrace{t^3}_{\text{blue}} \underbrace{x_{1,4}}_{\text{green}} \right) \\
 & + \underbrace{(3s^3 - 6s^2 + 3s)}_{\text{orange}} \left(\underbrace{(-t^3 + 3t^2 - 3t + 1)}_{\text{blue}} \underbrace{x_{2,1}}_{\text{green}} + \underbrace{(3t^3 - 6t^2 + 3t)}_{\text{blue}} \underbrace{x_{2,2}}_{\text{green}} + \underbrace{(-3t^3 + 3t^2)}_{\text{blue}} \underbrace{x_{2,3}}_{\text{green}} + \underbrace{t^3}_{\text{blue}} \underbrace{x_{2,4}}_{\text{green}} \right) \\
 & + \underbrace{(-3s^3 + 3s^2)}_{\text{orange}} \left(\underbrace{(-t^3 + 3t^2 - 3t + 1)}_{\text{blue}} \underbrace{x_{3,1}}_{\text{green}} + \underbrace{(3t^3 - 6t^2 + 3t)}_{\text{blue}} \underbrace{x_{3,2}}_{\text{green}} + \underbrace{(-3t^3 + 3t^2)}_{\text{blue}} \underbrace{x_{3,3}}_{\text{green}} + \underbrace{t^3}_{\text{blue}} \underbrace{x_{3,4}}_{\text{green}} \right) \\
 & + \underbrace{s^3}_{\text{orange}} \left(\underbrace{(-t^3 + 3t^2 - 3t + 1)}_{\text{blue}} \underbrace{x_{4,1}}_{\text{green}} + \underbrace{(3t^3 - 6t^2 + 3t)}_{\text{blue}} \underbrace{x_{4,2}}_{\text{green}} + \underbrace{(-3t^3 + 3t^2)}_{\text{blue}} \underbrace{x_{4,3}}_{\text{green}} + \underbrace{t^3}_{\text{blue}} \underbrace{x_{4,4}}_{\text{green}} \right)
 \end{aligned}$$

- We observe in the equation:
 - The Bézier curve in s (orange)
 - The Bézier curves in t (blue)
 - The control points $x_{i,j}$ (green)

Bicubic Surface – Illustrations



New Perspectives On Geometry

- We saw several ways of encoding geometry,
 - But, ultimately, what we want to represent is the light
 - Every representation is some kind of simplification of how the light interact with the material
-
- Let's focus a bit more on the light equations to have a general framework.

Plenoptic function from BRDF

- Previously, we defined the BRDF

$$f_r(x, \omega_i, \omega_o, \lambda, t)$$

- It describes the material properties of the surface and how it scatters light.
- To use it, we described the rendering function

$$L_o(\vec{x}, \omega_o, \lambda, t) = L_e(\vec{x}, \omega_o, \lambda, t) + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t) (\omega_i \cdot \vec{n}) d\omega_i$$

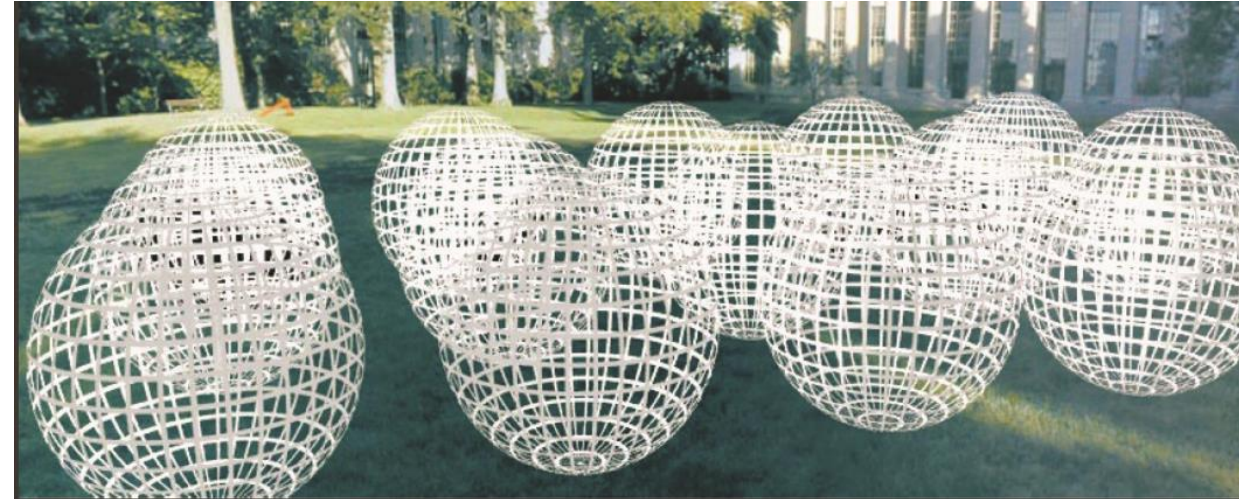
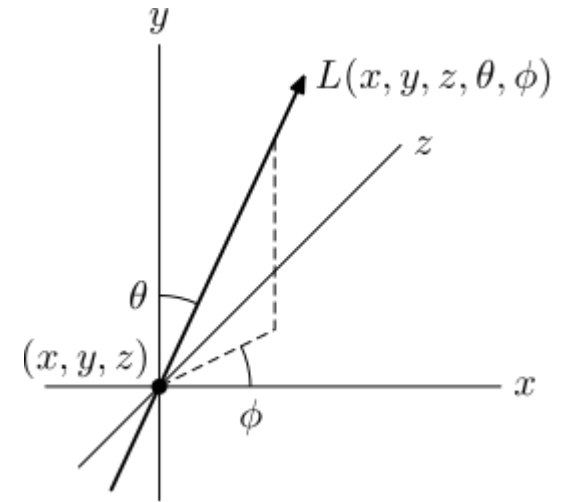
- It computes light transport for surface points, determining how light interacts with materials to produce the observed appearance.
- The light equation is part of a bigger function, the **Plenoptic function**
$$L(x, y, z, \lambda, \theta, \varphi, t)$$
 - It is the most general representation of light in a scene, capturing every possible ray of light at every location.
 - It stores all light traveling in every direction through every point in space, a lightfield.

The Plenoptic Function

- The Plenoptic function

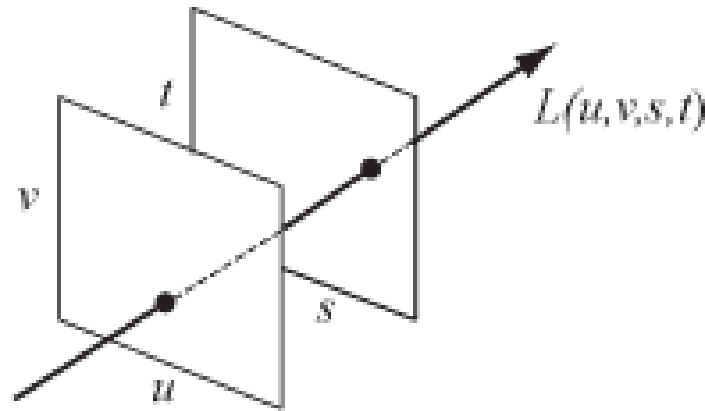
$$L(x, y, z, \theta, \varphi, \lambda, t)$$

- (x, y, z) encode the spatial position of a point
- (θ, φ) the angular information
- (λ, t) the wavelength and time
- It is like capturing a 360 video at every point in space



The Plenoptic Function – Storage

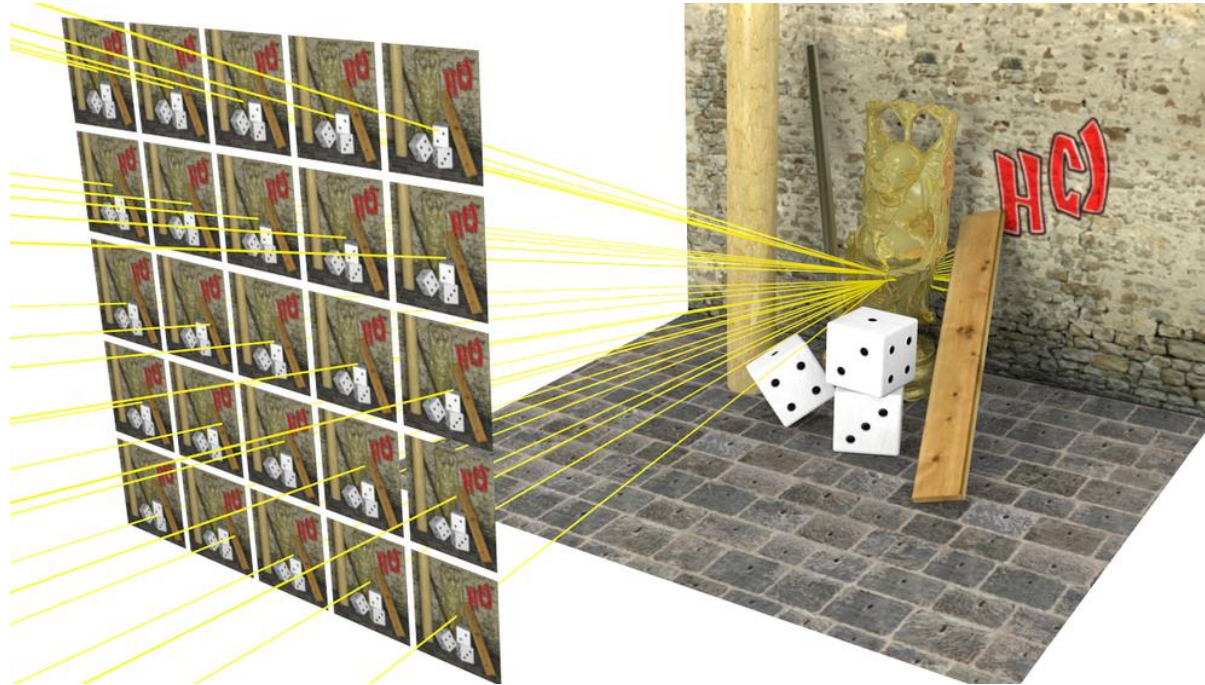
- We cannot easily store or acquire the plenoptic function
- Several methods were proposed to store simplified version of it.
- One in particular is the lightfield
 - We remove λ, t assuming they are irrelevant
 - And create an encoding for (x, y, z) and (θ, φ) simultaneously
 - The most widely used encoding is the two planes parametrization



- Do you see how we could capture and store the lightfield ?

Acquiring And Storing The Lightfield

- Do you see how we could capture and store the lightfield ?



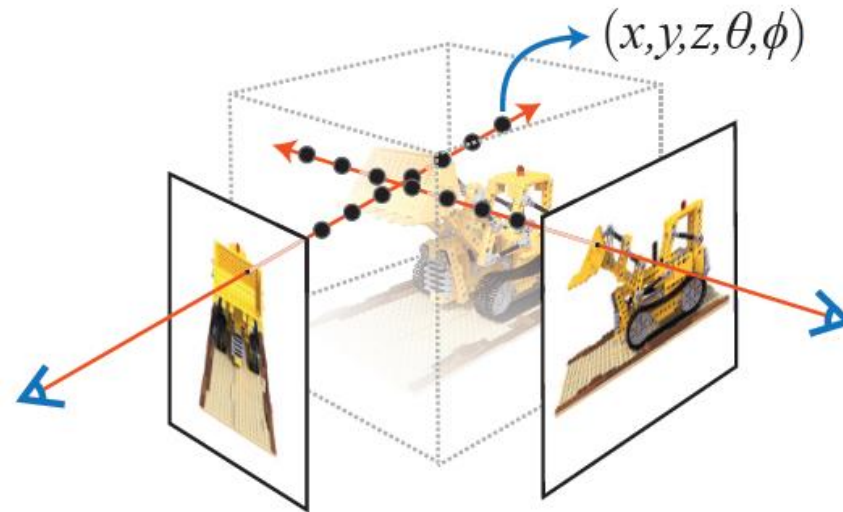
- By capturing multiples photos of a scene with a slight different position, we effectively encode the position and direction of every point in the scene

Encoding The Lightfield

- Now that we know how the light can be stored
- It is time to render it !
- Several techniques were proposed
- We will cover 3 of them:
 - Volume rendering (NeRF)
 - Point Cloud Splatting (Gaussian Splatting)
 - Depth image-based rendering (RVS)

Volume Rendering

- One of the first geometry representations !
- The idea is that every point in a volume can have a color
- The rendering happens by launching rays in that volume and accumulating the colors



Volume Rendering

- Neural Radiance Fields (NeRF) is smart way to get the values of the points in the volume starting from photos and using neural networks.

A lot of input
(100 here)

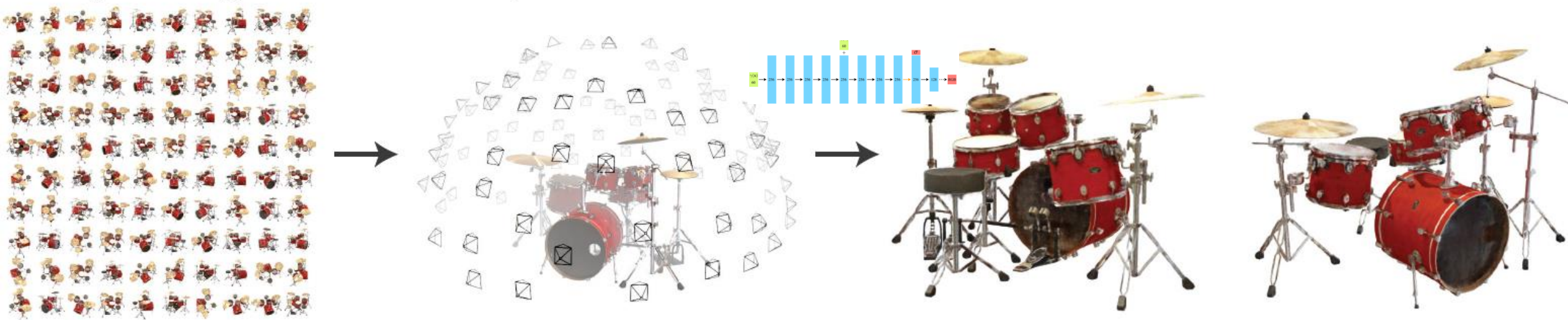
Get calibration

Volume rendering
360 degrees

Input Images

Optimize NeRF

Render new views



NeRF: Representing Scenes as
Neural Radiance Fields for View Synthesis

Ben Mildenhall^{1*} Pratul P. Srinivasan^{1*} Matthew Tancik^{1*}
Jonathan T. Barron² Ravi Ramamoorthi³ Ren Ng¹

¹UC Berkeley ²Google Research ³UC San Diego

From Point Clouds to Volumetric Representations

- NeRF Results



Volume Rendering

- NeRF gives high quality results !
- But it requires a lot of input views
- And does not store the depth of the objects
 - Not easy to manipulate.
- What could we do instead ?

A lot of input
(100 here)

Input Images



NeRF: Representing Scenes as
Neural Radiance Fields for View Synthesis

Ben Mildenhall^{1*} Pratul P. Srinivasan^{1*} Matthew Tancik^{1*}
Jonathan T. Barron² Ravi Ramamoorthi³ Ren Ng¹

¹UC Berkeley ²Google Research ³UC San Diego

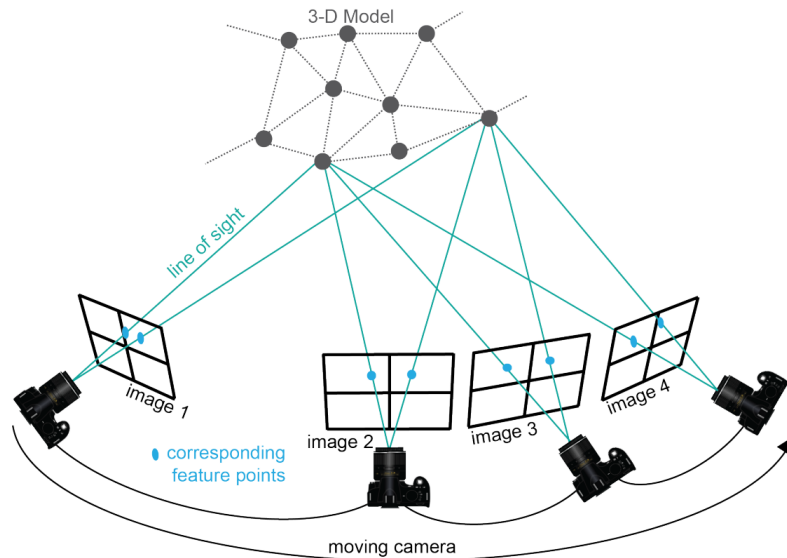
ECCV 2020

92

Acquiring Depth

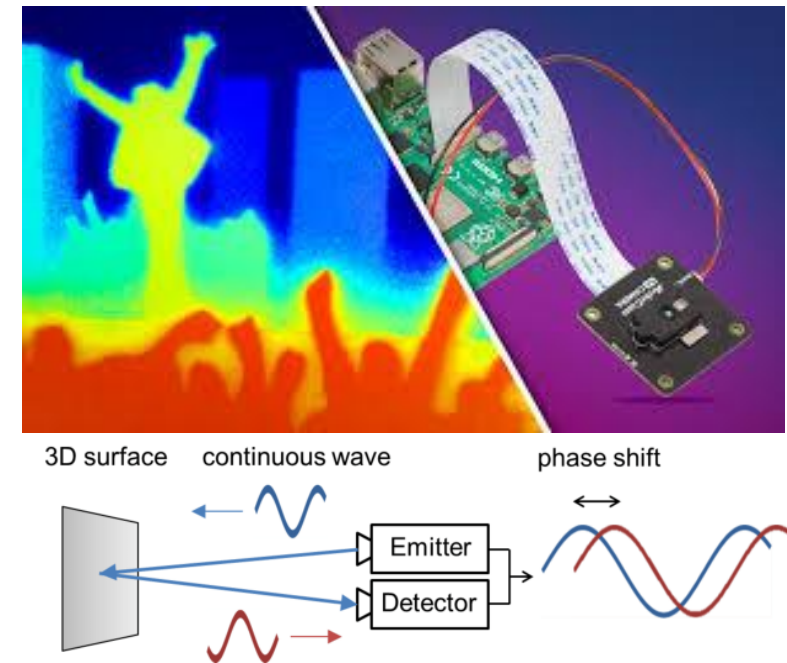
- Instead of starting with pictures of the scene
- We can acquire the depth of the scene directly using algorithms or hardware
- Two very common techniques are Structure-from-Motion (SfM) and Time-of-Flight (ToF) cameras

Structure-from-Motion



- What do they give as outputs ?

Time-of-Flight



Point Clouds

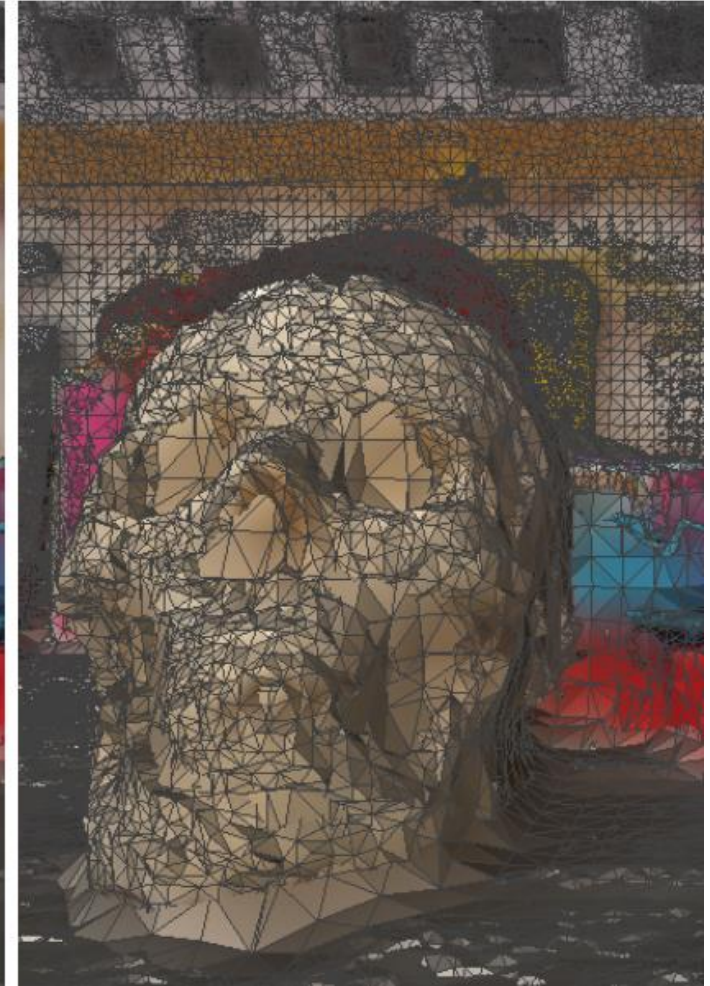
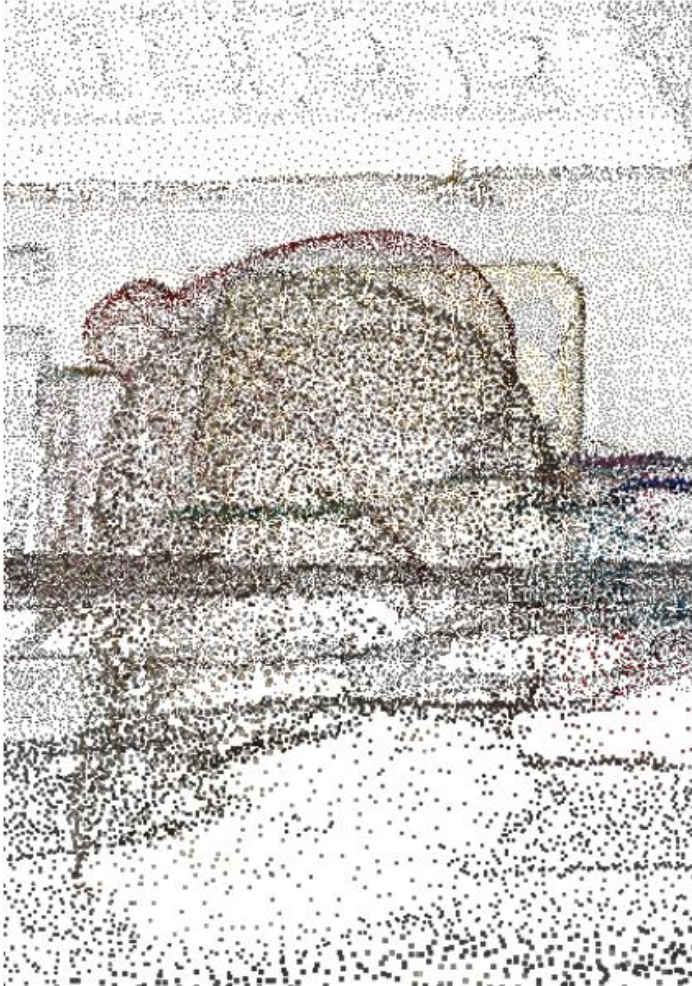
- Both SfM and ToF give point clouds as result
 - I.e., a point in 3D with an associated color



- How can we display that without all the holes ?

Point Clouds Rendering

- Yes, we could generate triangles !



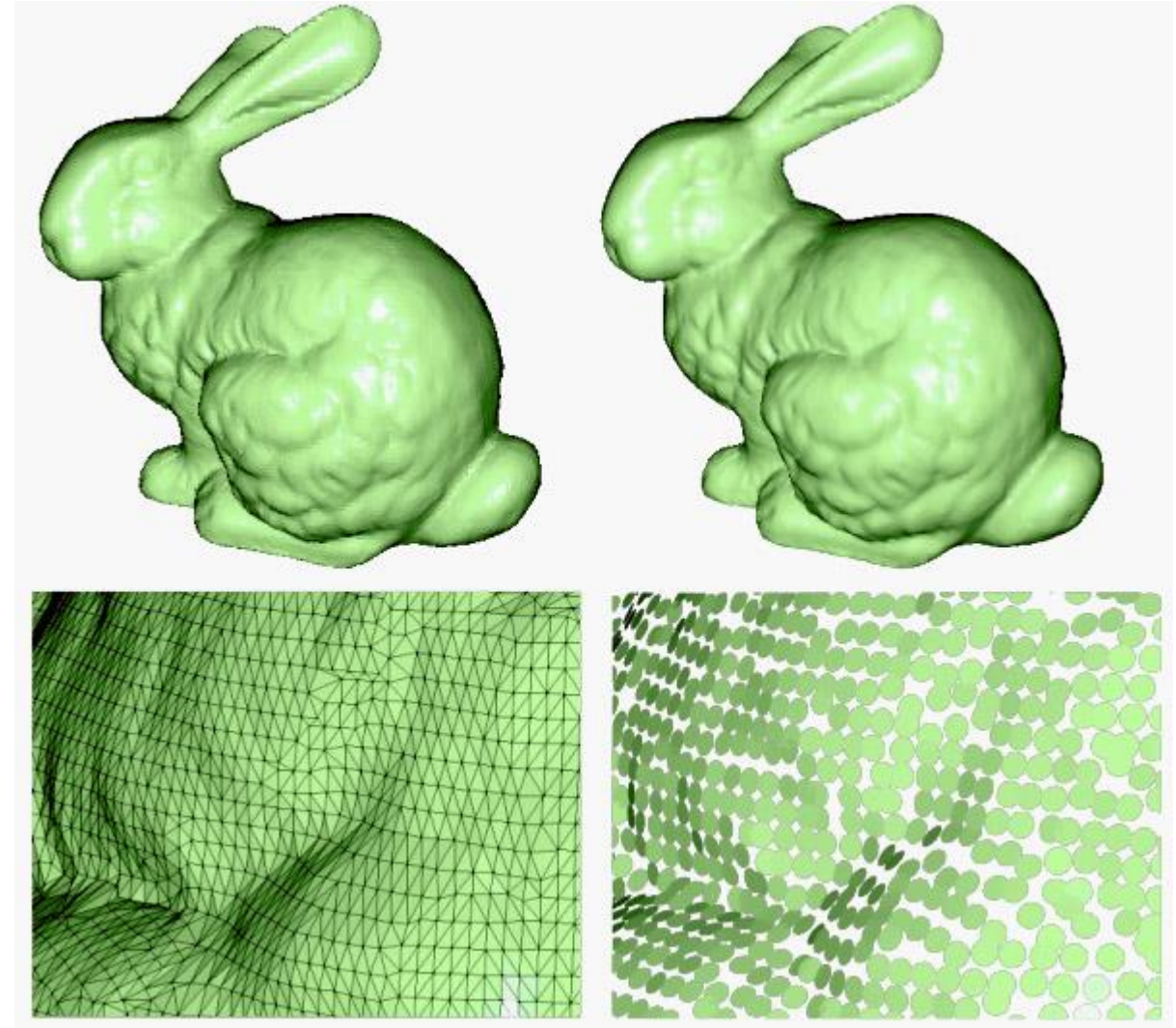
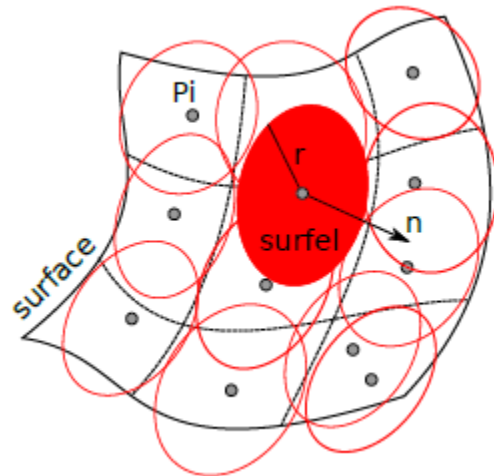
- But the results are not nice, what else ?

Point Clouds Rendering

- What if instead of generating triangles, we just assume a point represents a bigger surface



- We call it a **surfel**.
 - Surface elements around points that are
 - **Normal** oriented
 - **Colored** as points
 - **Disk** or **squares**

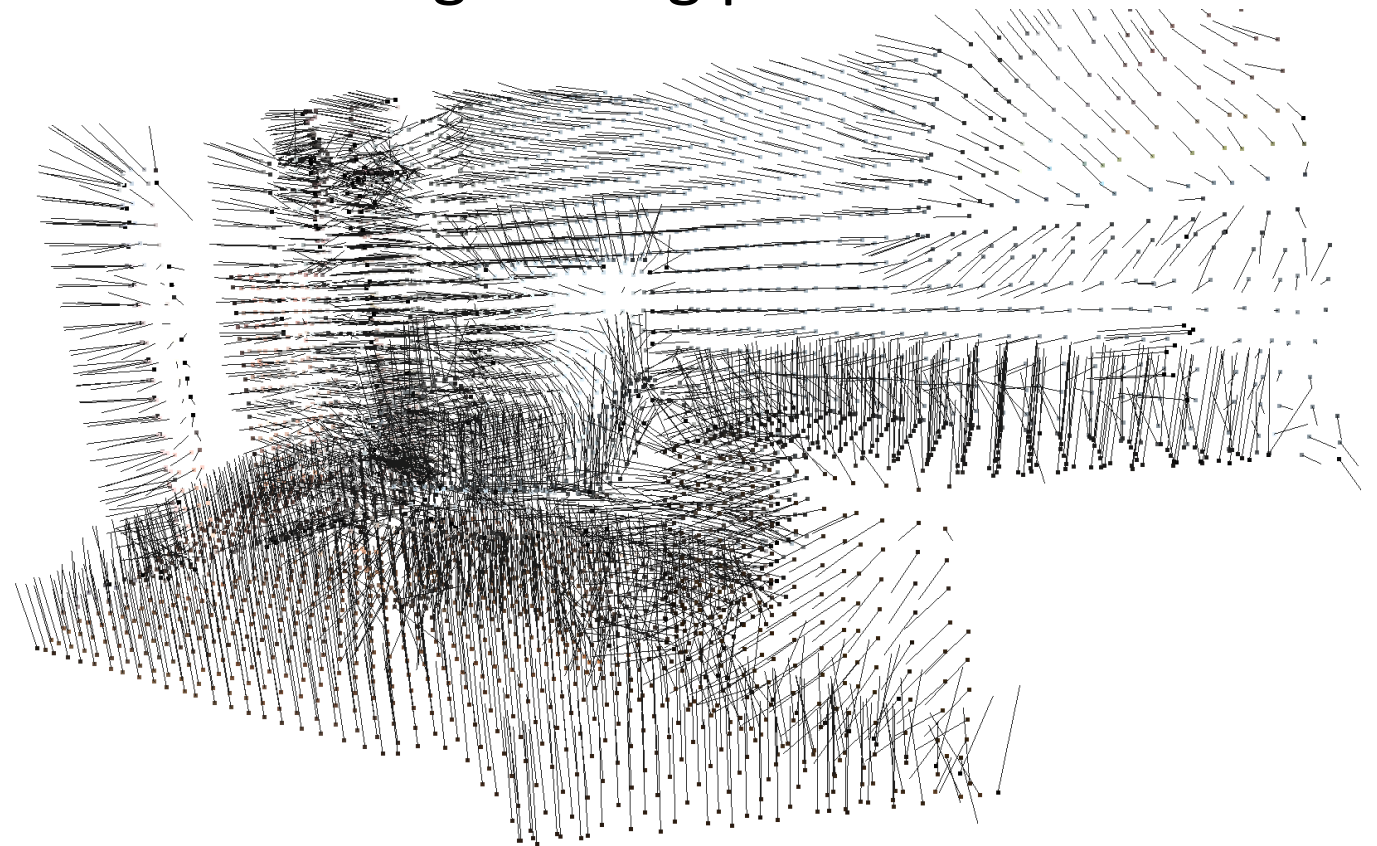
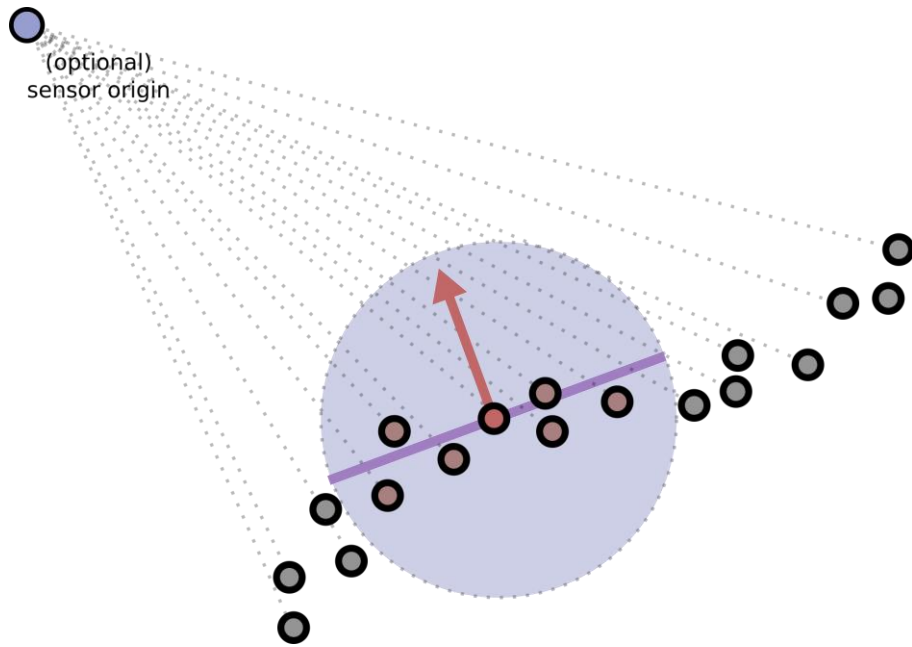


Point Clouds Rendering

- Point cloud rendering is natural for many applications.
 - We often scan a real object to have it in 3D
- Surfels have their own issues in rendering
- They are:
 - **Normal** oriented, we need to find each normal
 - **Colored** as points, the sensor is not aligned with the color camera
 - **Disk** or **squares** on points, it could impact the rendering
- Let's see the issues one by one.

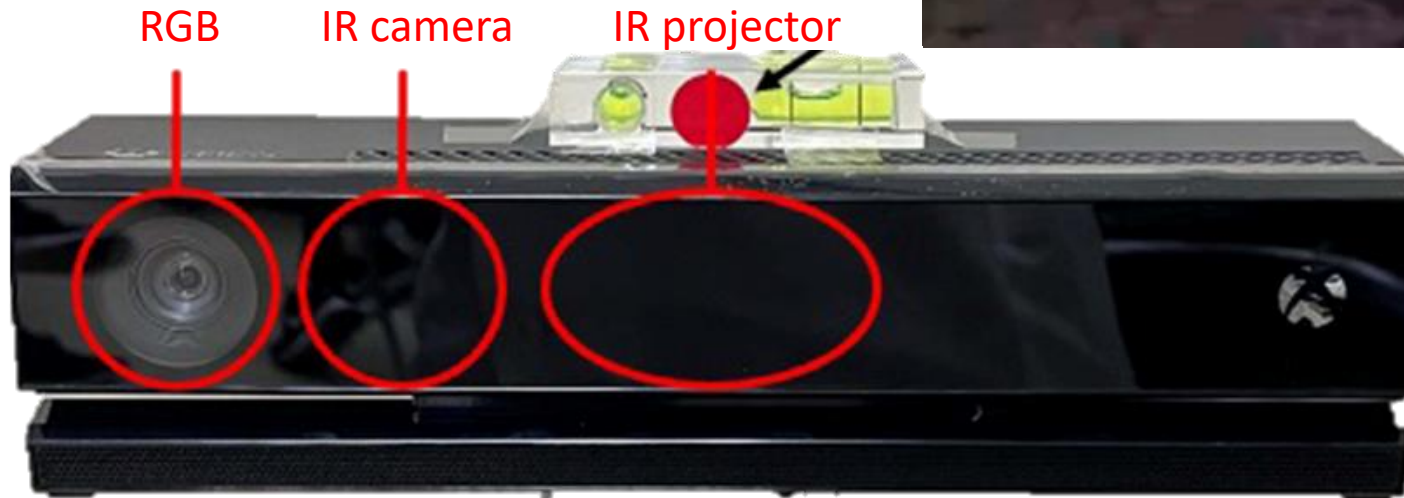
Point Clouds Rendering – Normal Estimation

- Surfels are **normal** oriented.
- We need to find the normal of each point
- We use global algorithms that consider the neighboring points to do it



Point Clouds Rendering – Colors

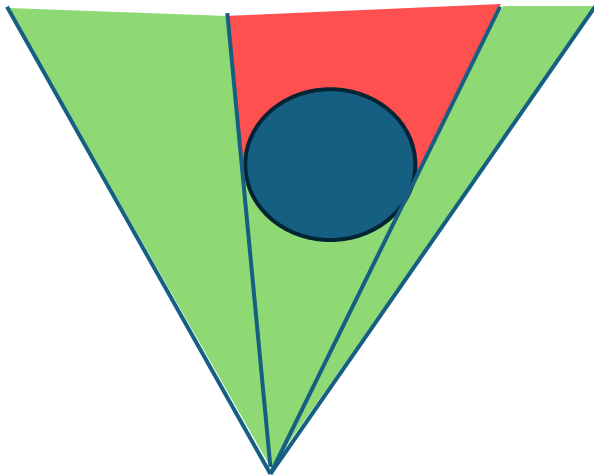
- Coloring the point cloud is not easy
- Sensors do not have the color and depth acquisition at the same place
- Generating wrong colors.



Red from the backpack

Point Clouds Rendering – Colors

- Coloring the point cloud is not easy
- The sensor acquire the scene from a unique point of view
- This creates occlusions



Person = occlusion

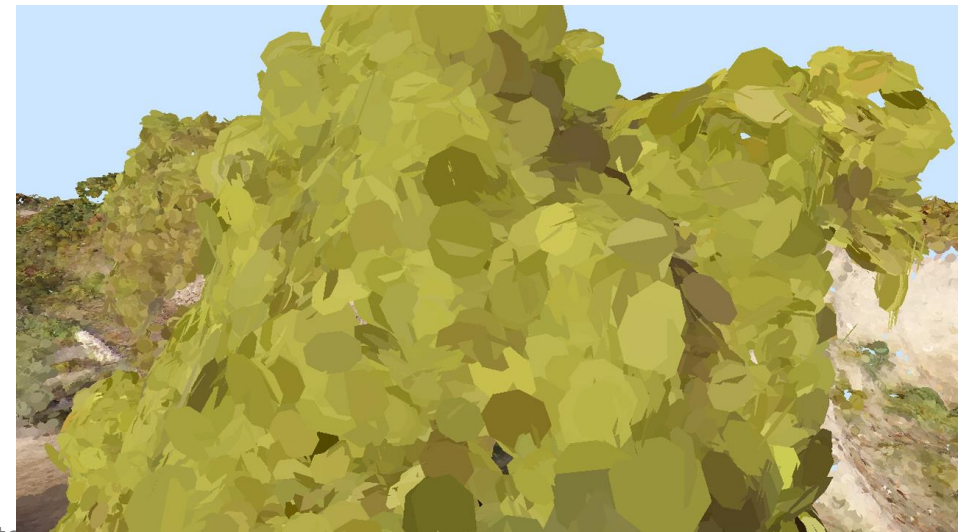
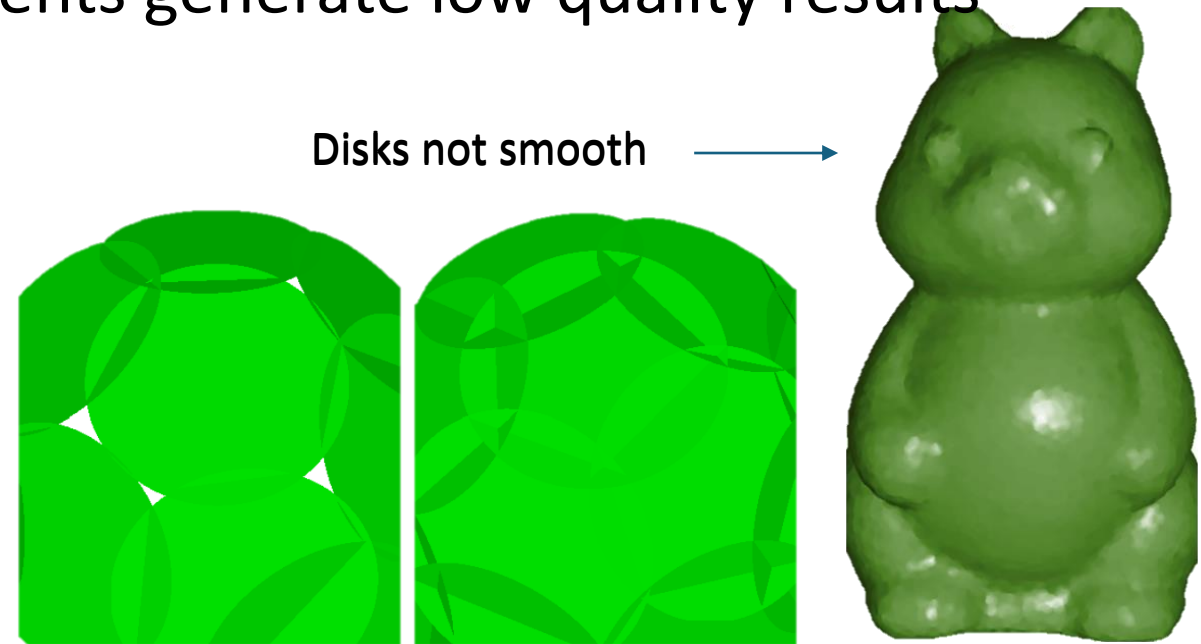
Point Clouds Rendering – Colors

- Using squares or disks as surface elements generate low quality results
- The surfaces are not smooth in the intersection between surfels



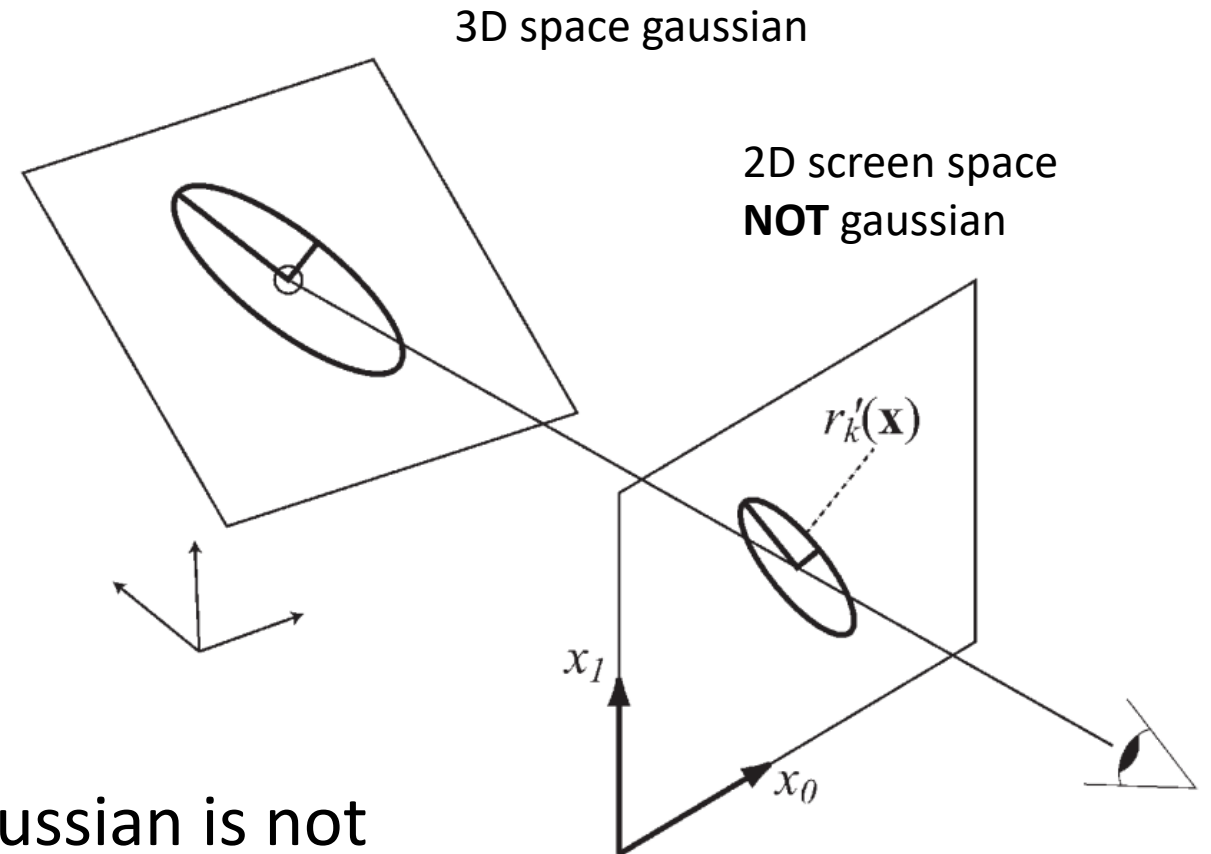
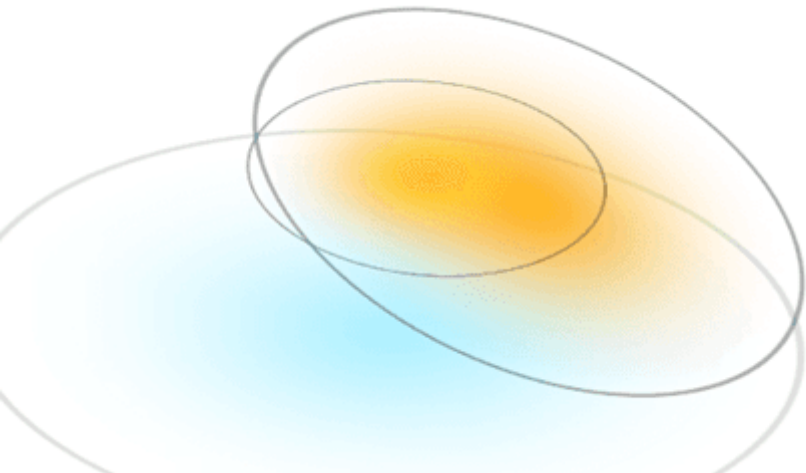
Visible squares

- What could we do to improve that ?



Point Clouds Rendering – Colors

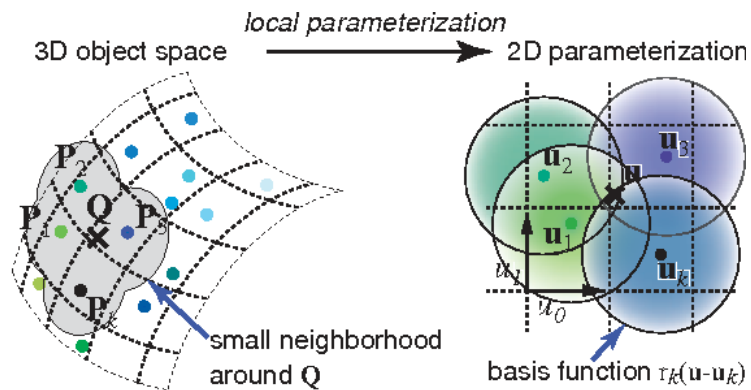
- Using squares or disks as surface elements generate low quality results
- A simple idea is to put a gaussian distribution over each surfel
- When we render them
We accumulate the contribution of several gaussians
- This gives a smooth result



- But.. A 3D gaussian is not projected to a 2D gaussian on the screen ☹️

Point Clouds Rendering – Colors

- Using squares or disks as surface elements generate low quality results
- Instead of working with gaussians in 3D space
- The Elliptical Weighted Average (**EWA**) splatting works in screen-space directly
- Allowing for accurate gaussians representation



Point Clouds Rendering – Colors

- We now have accurate 3D rendering..
- But.. No lights on the model.. What can we do ?



Point Clouds Rendering – Colors

- We now have accurate 3D rendering..
- But.. No lights on the model.. What can we do ?

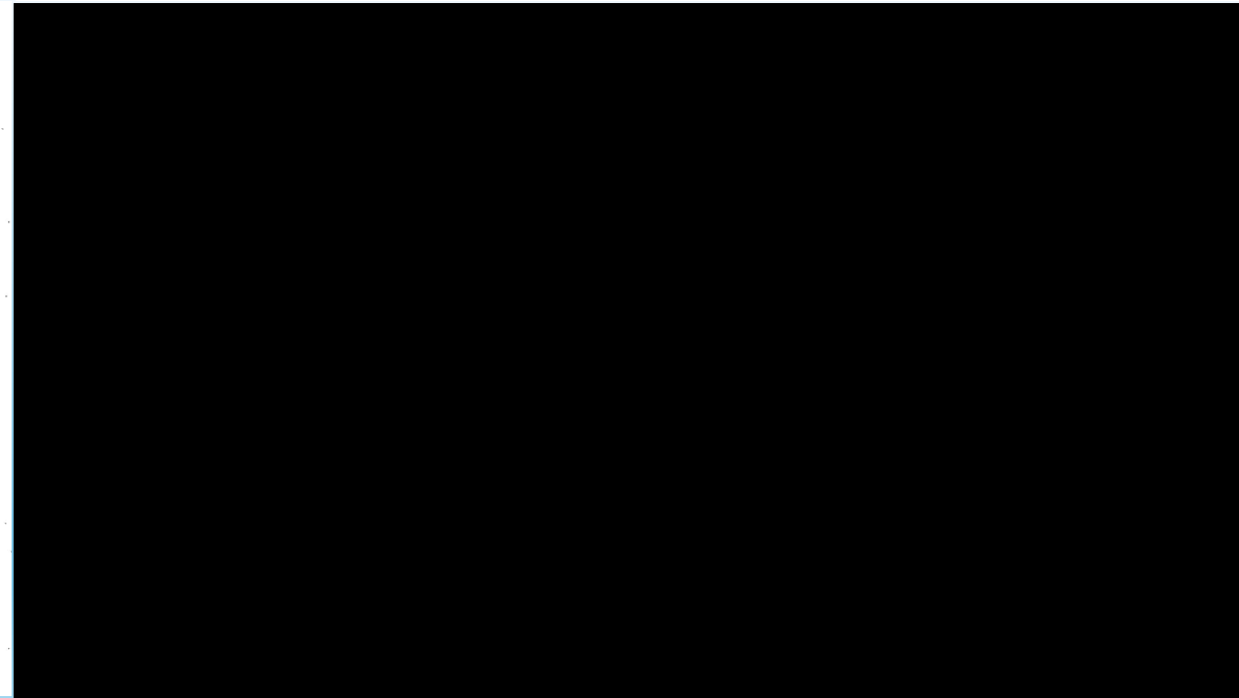


- We can also define a smooth field of normals on the surfels
- This allows us to compute light at every point of the object

Splatting – Results



Phong Splatting - Floor



Phong Splatting - LOD

Splatting – Results



Point Clouds Rendering – Rendering

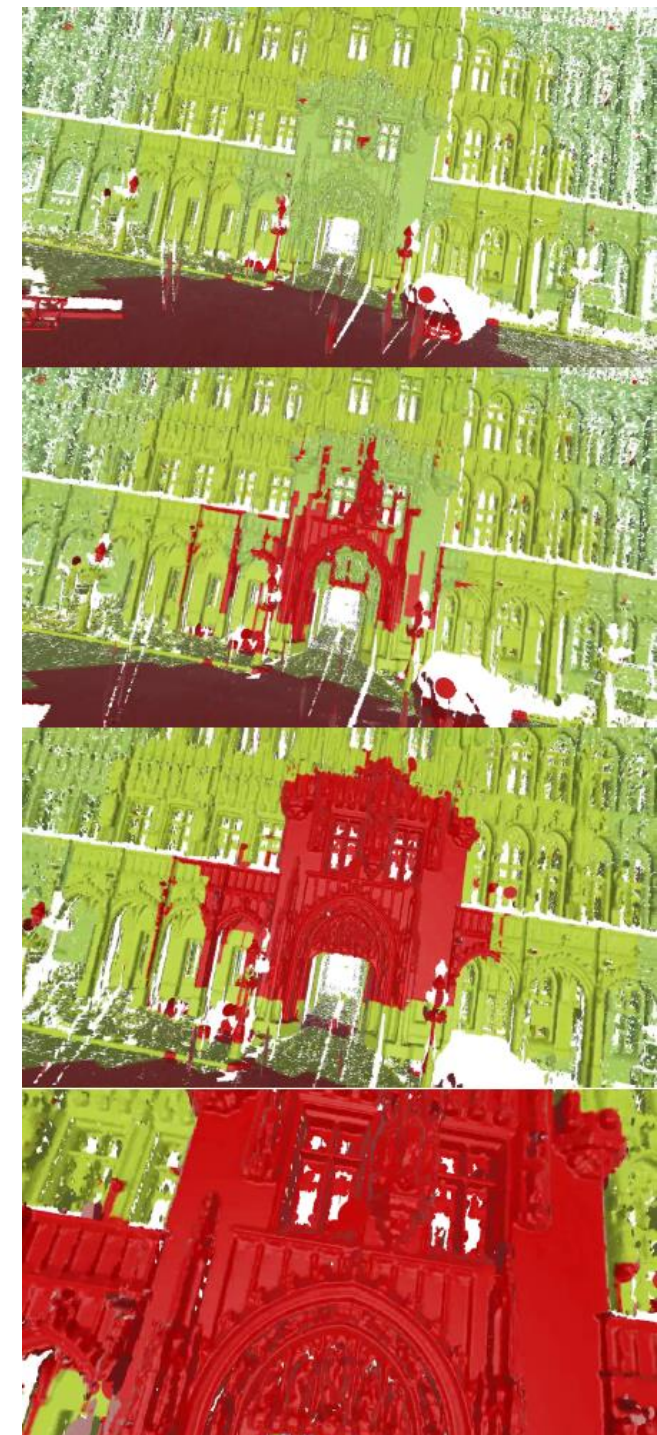
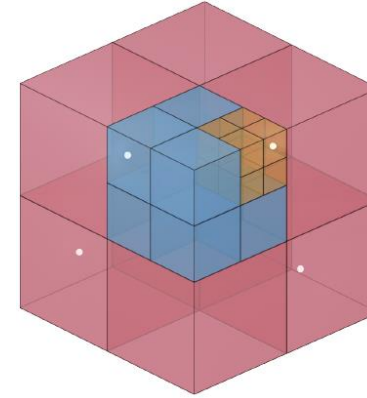
- An issue we didn't consider is the number of surfels to render
- A common scanner can generate millions of points
- Rendering them is compute intensive



- How to accelerate the computations ?

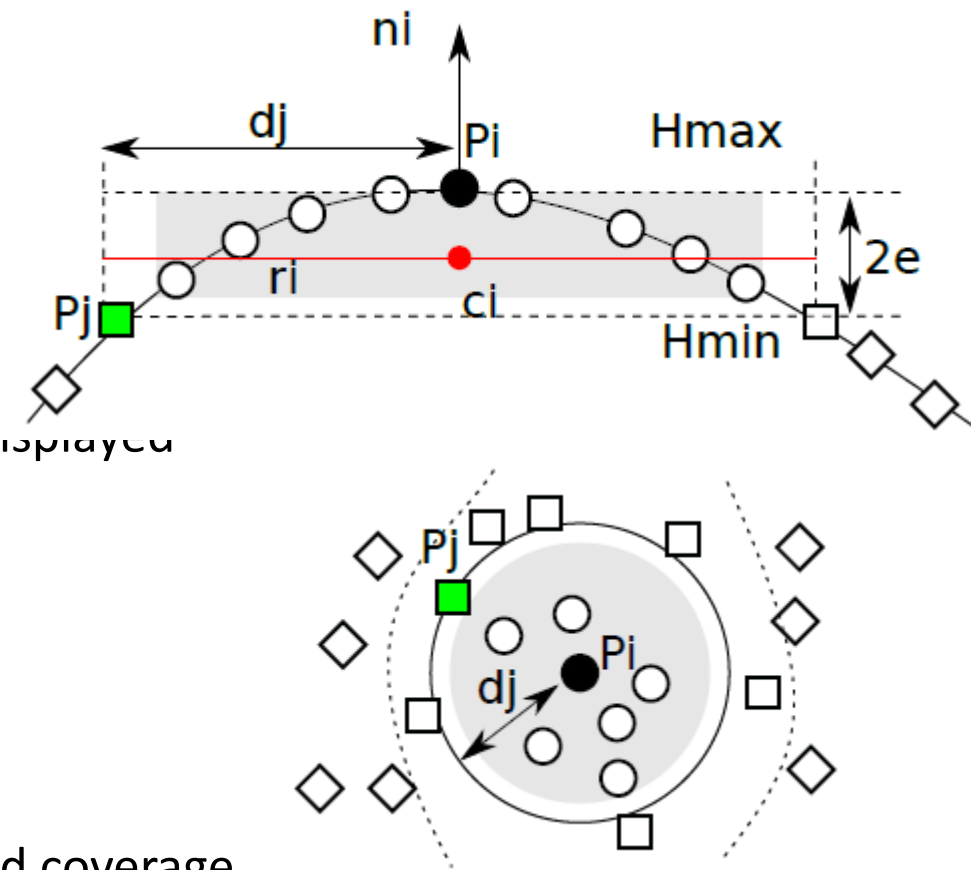
Point Clouds Rendering

- How to accelerate the computations ?
 - Spatial data structures (as with raytracing)
 - As the camera get close to the model load more points



Point Clouds Rendering

- How to accelerate the computations ?
 - Optimized Sub-sampling
 - We grow surfels around each point
 - This means that most points will be covered and never displayed
 - We can use this to select important surfels
 - To do so:
 - We use the curvature to grow the surfel
 - It stops growing when some curvature exceed a threshold
 - We remove all covered points
 - We use ellipses instead of disks for even more quality and coverage



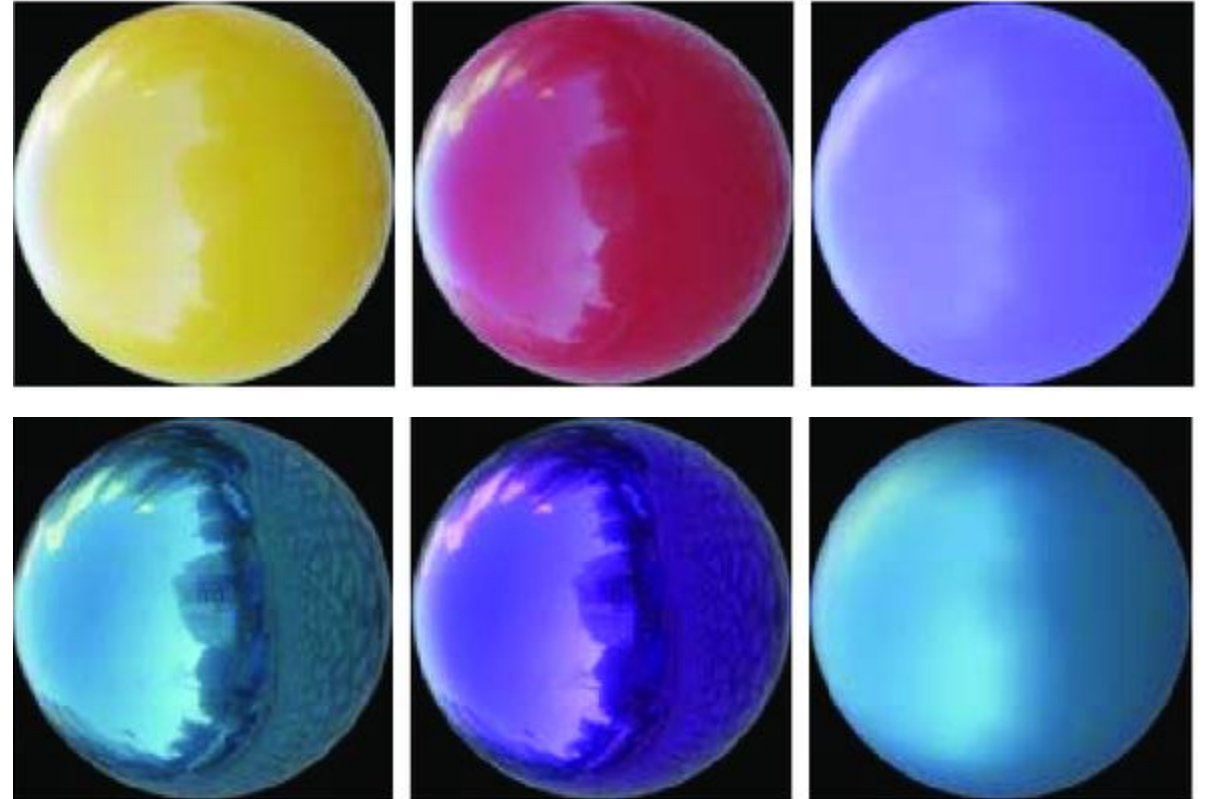
Original
171.000 points



Sub-sampled
333 points
= 0.2% of the original

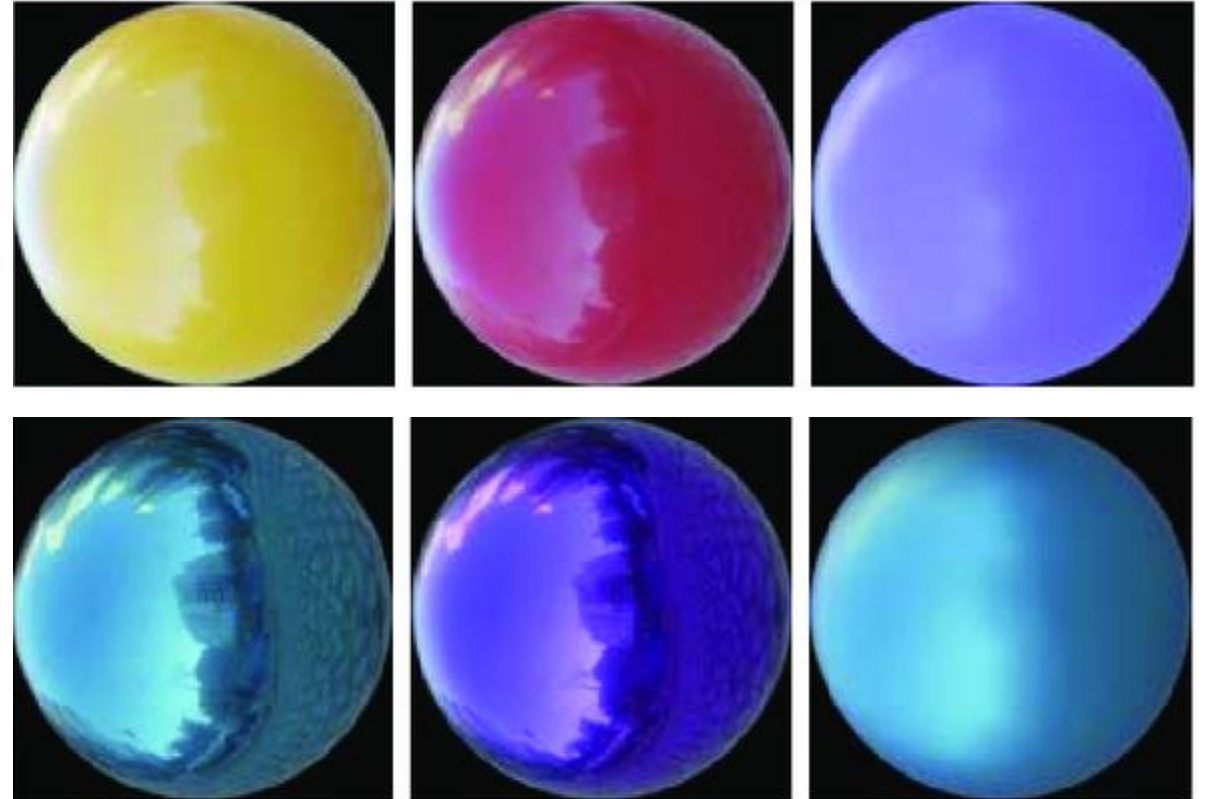
Point Clouds Rendering

- With all our simplifications..
- We lost something..
- Do you know what ?



Point Clouds Rendering

- With all our simplifications..
- We lost something..
- Do you know what ?
- We lost the specularity of our objects !
- Only one color per surfel ☹️



Point Clouds Rendering

- The first issue is that the observed light depends on the direction at which we observe the surfel
- But a surfel is a flat surface
- What to do ?

Point Clouds Rendering

- The first issue is that the observed light depends on the direction at which we observe the surfel
- But a surfel is a flat surface
- What to do ?
- We can replace the surfels by spheres
- With a 3D gaussian distribution on them
- But this does not solve the problem of the color intensity
- What to do ?

Point Clouds Rendering

- But this does not solve the problem of the color intensity
- What to do ?
- We would like to encode the color light on our 3D surfels
- It is like having a BRDF on each 3D surfel for color intensities
- How can we encode a function on a sphere ?

Point Clouds Rendering

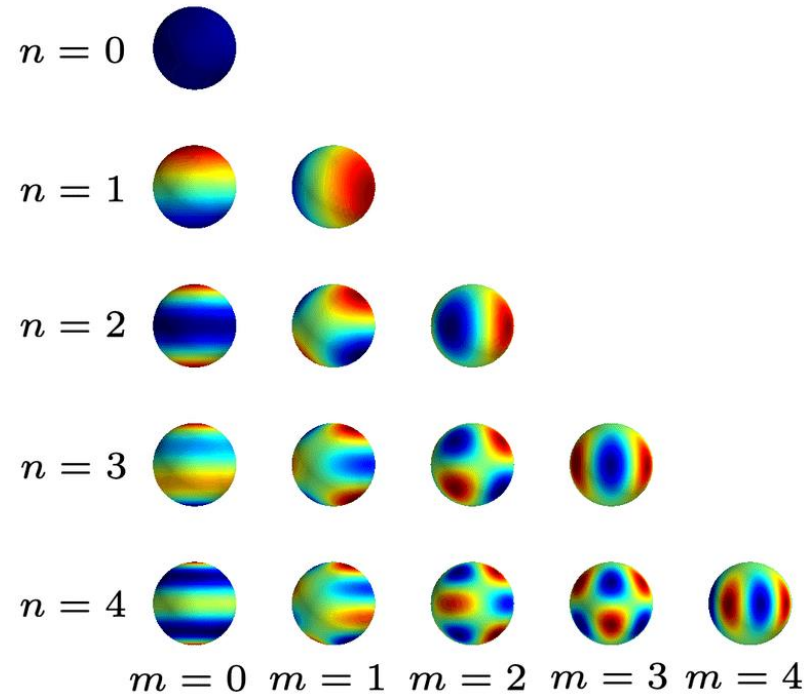
- But this does not solve the problem of the color intensity
- What to do ?
- We would like to encode the color light on our 3D surfels
- It is like having a BRDF on each 3D surfel for color intensities
- How can we encode a function on a sphere ?
- No ideas ?
- Remember your quantum physics lectures !

Point Clouds Rendering

- But this does not solve the problem of the color intensity
- What to do ?
- We would like to encode the color light on our 3D surfels
- It is like having a BRDF on each 3D surfel for color intensities
- How can we encode a function on a sphere ?
- No ideas ?
- Remember your quantum physics lectures !
- Still nothing ?

Spherical Harmonics

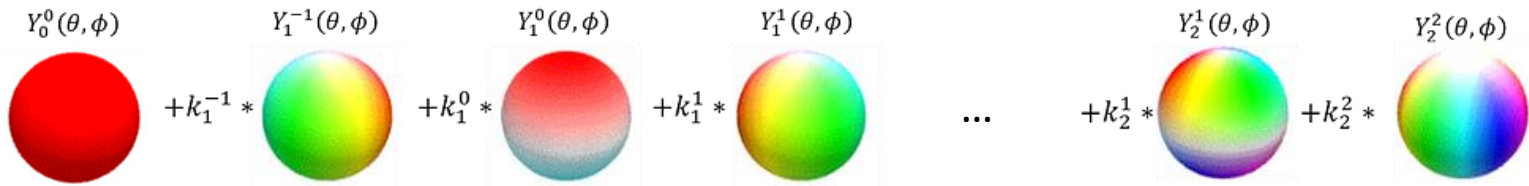
- In quantum physics, you defined a basis system over spheres !
- It is called a spherical harmonic



- Remember.. moments, spin, Legendre polynomials etc..

Spherical Harmonics

- As it is a basis over a sphere, any function can be defined by taking a linear combination of it !

$$C = \sum_{l=0}^{l_{\max}} \sum_{m=-l}^l k_l^m Y_l^m(\theta, \phi) = k_0^0 * Y_0^0(\theta, \phi) + k_1^{-1} * Y_1^{-1}(\theta, \phi) + k_1^0 * Y_1^0(\theta, \phi) + k_1^1 * Y_1^1(\theta, \phi) \dots + k_2^1 * Y_2^1(\theta, \phi) + k_2^2 * Y_2^2(\theta, \phi)$$


- We just need to store even more coefficients for our rendering 😞
- This method was popularized by INRIA in 2023.
- They use a mix between classical point cloud and NeRF
- They acquire many pictures of the scene
- And use a neural network to find the position of the surfels and the coefficients of the spherical harmonics

3D Gaussian Splatting
for Real-Time Radiance Field Rendering

SIGGRAPH 2023
(ACM Transactions on Graphics)
Bernhard Kerbl^{1,2} Georgios Kopanas^{1,2} Thomas Leimkühler³ George Drettakis^{1,2}
* Denotes equal contribution
¹Inria ²Université Côte d'Azur ³MPI Informatik

Gaussian Splatting – Results

Input



Structure-from-Motion

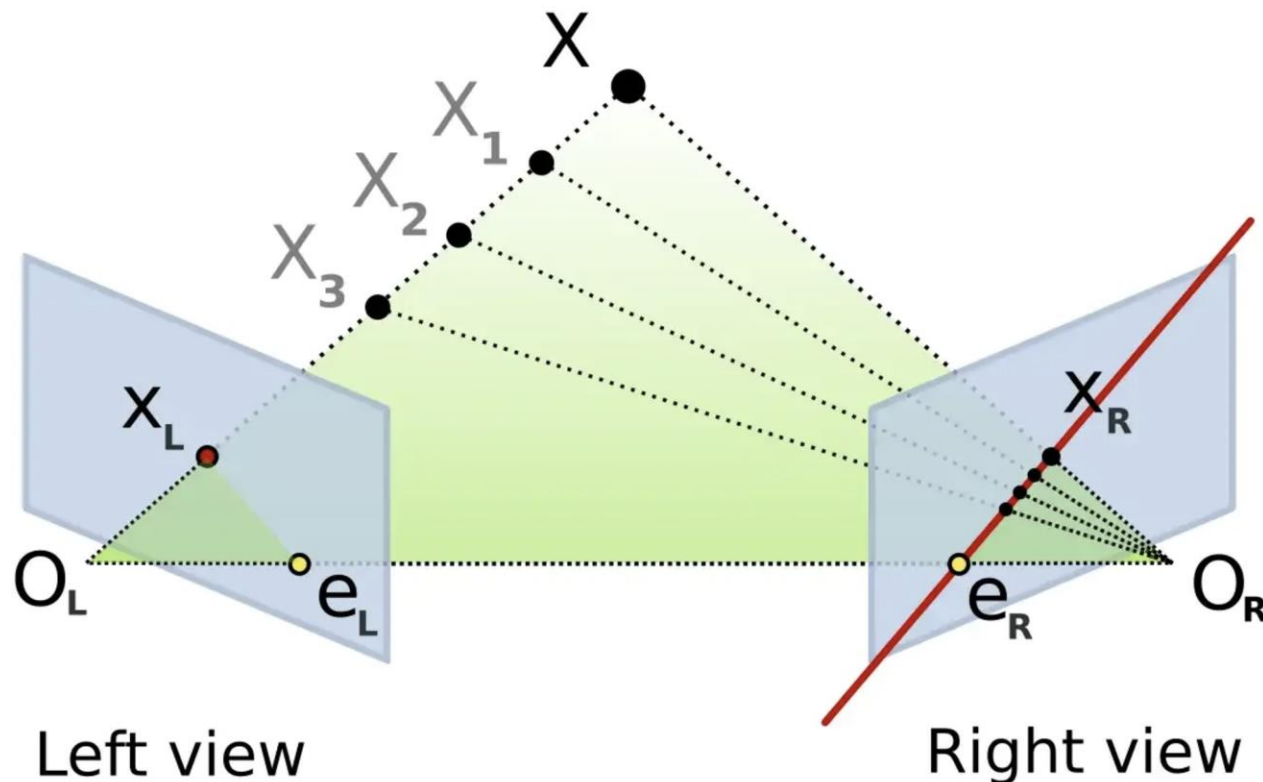


3D Gaussian Splatting

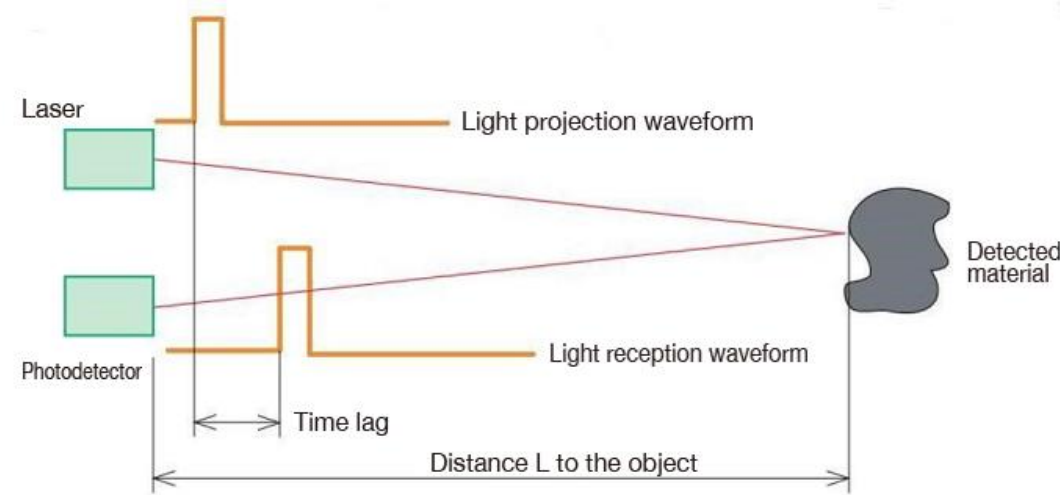


Depth Image Based Rendering

- In some applications we have access to the depth of the scene
- E.g., Setup with two cameras, we can estimate the depth by **stereo-vision**
- Or using depth sensing devices, such as a **Time-Of-Flight (ToF)** camera.



Stereo-vision



Time-Of-Flight

Depth Image Based Rendering

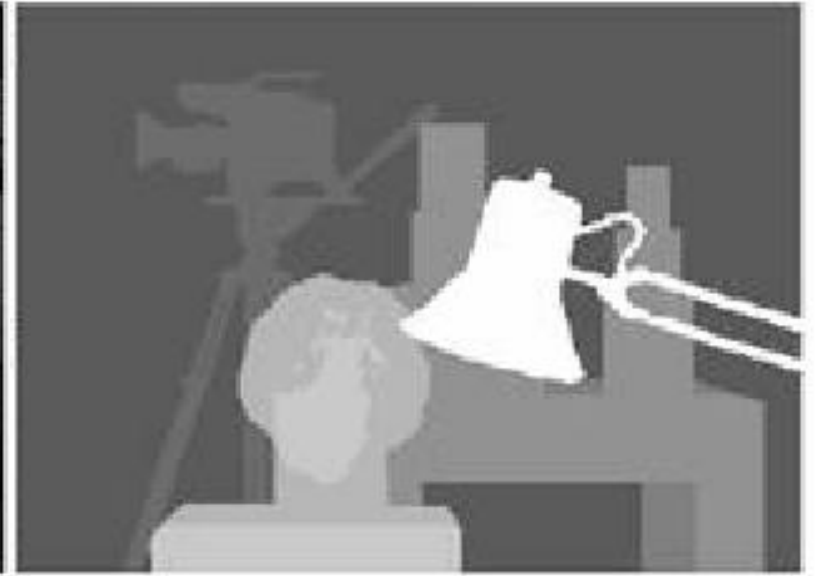
- With a left-right pair of images, we can find the depth of the objects



Left image



Right image



Depth

- It is a way to encode geometry, colors in the images, depth as geometry.
- But, how to render this from a novel point of view ?

Depth Image Based Rendering

- But, how to render this from a novel point of view ?
- By using only geometry, we can reproject the color in a novel point of view.
- This works with most of what we saw in the lectures

Depth Image Based Rendering

Developped here !

- To do that you can use the Reference View Synthesizer (RVS) software.
- It allows in real-time to render novel points of view
- It is way faster than NeRF and Gaussian Splatting with a similar quality
 - While, at the moment, not allowing directional color such as with spherical harmonics
- Used by MPEG in the standardization process for novel technologies.
- <https://gitlab.com/mpeg-i-visual/rvs>

Real-Time Depth Video-Based Rendering for 6-DoF HMD Navigation and Light Field Displays

Publisher: IEEE

Cite This



Daniele Bonatto  ; Sarah Fachada  ; Ségolène Rogge  ; Adrian Munteanu  ; Gauthier Lafruit  [All Authors](#)

Reference View Synthesizer (RVS)



RVS – One Frame Synthesis

- We can generate a movement in a scene with only one frame and one depth



Reference View Synthesizer – VR

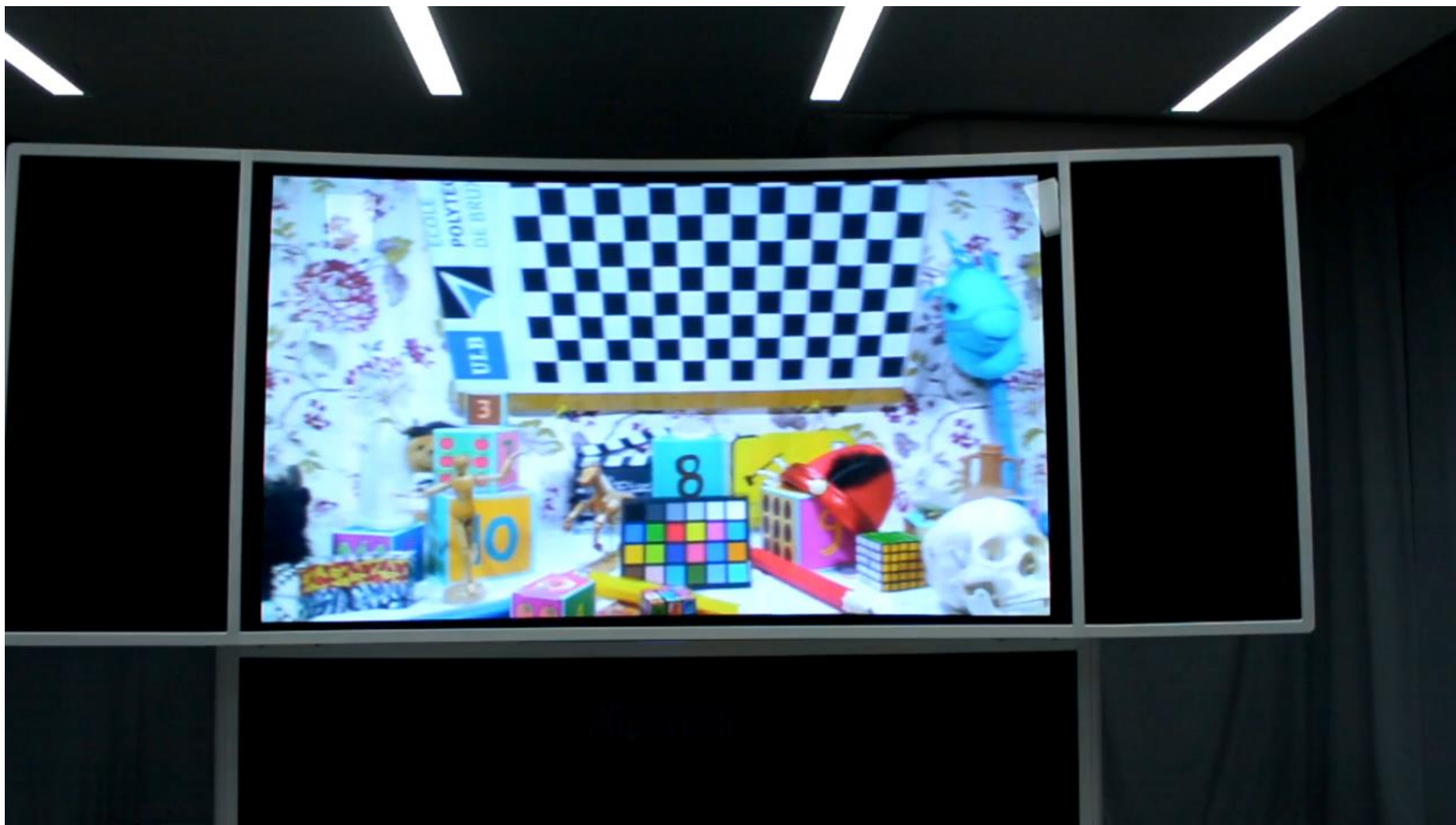
- RVS runs in real-time
- In a headset
- 2x90 Fps

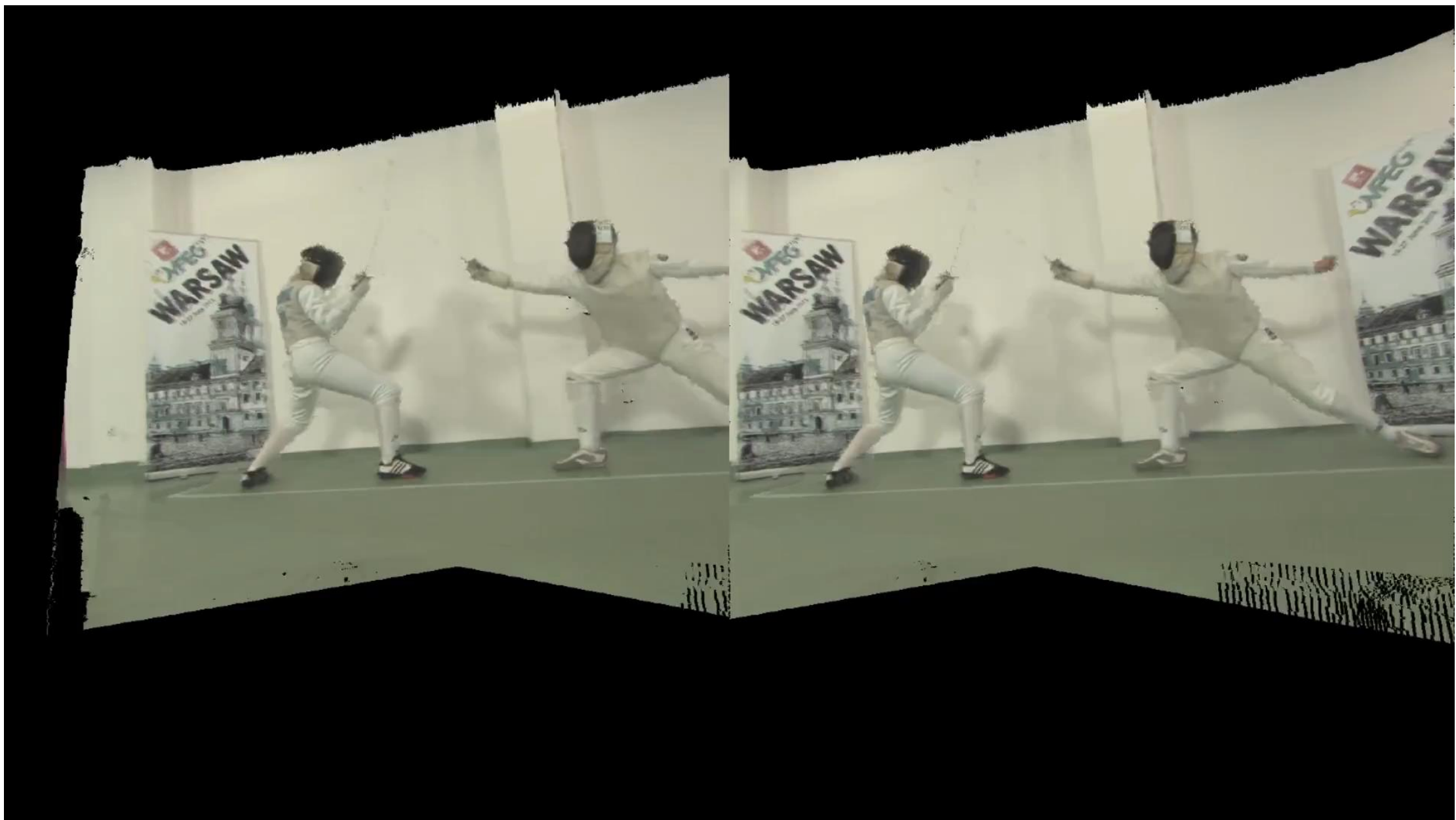


Reference View Synthesizer









(1)



(5)

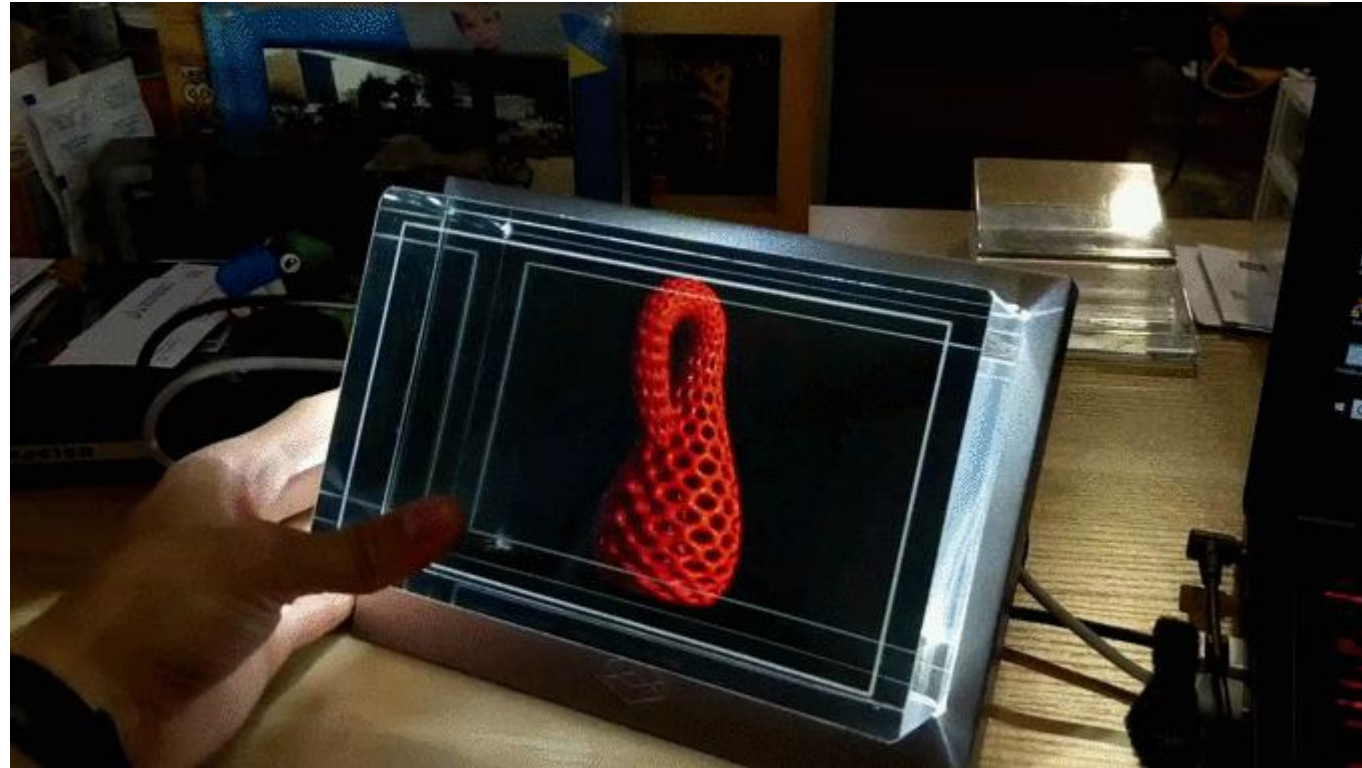


(49)



Displays To Show View Synthesis

- Displays are being developed to show 3D without glasses !
- They display at the same time N directional views
- But, transmitting N views is hard.
 - One movie @ 30 fps in 4k
= $30\text{fps} \times 8 \text{ Millions pixels!}$
 - That's a lot of pixels to transmit.
 - Now instead of 30 fps
 - We need to send $N \times 30$ frames
 - **Impossible.**
- Instead, we send $k \ll N$ frames
- And reconstruct the $N - k$ missing views.



Looking Glass Screen

What I Expect for the exam

- After this lesson I expect that:
 - You have an idea of different ways to input geometry on a computer
 - You can give the explanation of a Hérmitte/Bézier curve
 - You can reexplain and make the drawings for Bicubic surfaces
 - So, no math, but intuitively how they work
 - You know what the Plenoptic function represent and how it is acquired
 - You know the main ideas behind NeRF, Gaussian Splatting, RVS
 - You understand the tradeoffs between the different ways of encoding geometry

Next time



Particle Systems



Fluids Simulations